MINISTRY OF EDUCATION OF THE REPUBLIC OF BELARUS
EDUCATIONAL INSTITUTION
«BREST STATE TECHNICAL UNIVERSITY»
Department of Intelligent Information Technologies

«I admit to the defense»
Head of the department
_____ В.А.Головко
«____»_____ 2027

Application for constructing a maritime vessel’s course based on neural-network classification of geographic zones

EXPLANATORY NOTE TO THE DIPLOMA PROJECT
ДР.ИИ2025.250978-11 81 00

Sheets: 117

Supervisor А.А.Козинский

Completed Li Haoran

Standard control А.А.Козинский

2027

ABSTRACT

Explanatory note: 117 pages, 5 sections, 34 figures, 55 tables, 4 appendices, 42 references. Graphic material: 5 sheets of A1 format.

MARITIME ROUTE PLANNING, NEURAL NETWORK, CLASSIFICATION OF GEOGRAPHIC ZONES, AIS DATA, COST MAP, A* ALGORITHM, DYNAMIC PROGRAMMING, BELLMAN FUNCTION, GENETIC ALGORITHM, FITNESS FUNCTION, COMPLEXITY OF ALGORITHMS.

The aim of the work is to develop an application that constructs a safe and economically reasonable course for a maritime vessel. A residual multilayer perceptron, trained on a corpus of eleven sea regions from which 5,427 mislabelled points were removed, classifies a geographic point into one of four zones; the zones form a traversal cost map, over which A*, Dijkstra, dynamic programming (Bellman function) or a genetic algorithm (fitness function) searches the route. The classifier reaches an accuracy of 96.95 % on 22,712 test points. On six benchmark voyages and one voyage in each of the eleven sea regions of the corpus the three exact methods return the same optimal cost, the genetic algorithm finds a navigable route on fifteen of them, and every great-circle route crosses land. A test on 100 random voyages, checked against an independent land mask, found 19 courses that cross land, mostly where the corpus samples the coast too sparsely; an automatic check of every route against that coastline, built into the planner, corrects 19 of them. In version 1.3 every method has parameters set in the interface (step limits, the parameters of the genetic algorithm, a time limit), a search can be stopped, every result is saved to a JSON file, and complex courses are constructed with an adaptive search area, the fusion of the land mask, a two-level search and intermediate points. The interface is available in English, Russian and Chinese.

Faculty EIS Department IIT

APPROVED
Head of the
department _____
(signature)
“ ” _____ 2026

ASSIGNMENT
for the diploma project

Student Li Haoran

1. Topic of the project Application for constructing a maritime vessel’s course based on neural-network classification of geographic zones

Approved by the order of the university of _____2026 No. ___

2. Deadline for the submission of the completed project by the student 1.01.2027

3. Initial data for the project

General requirements for the application:

- The aim of the work is to develop an application for constructing an optimal and safe route of a maritime vessel on the basis of a classification of geographic zones using neural-network methods.
- The work is carried out using the Python programming language.
- The neural network is implemented with the TensorFlow, Keras or PyTorch libraries.
- The data source is a prepared data set of AIS data (Automatic Identification System), labelled with four classes of geographic zones:
  OPEN_SEA (open sea);
  COASTAL_SEA (coastal sea);
  NEAR_COAST (shore — land not further than 5 km from the coast);
  COASTLINE (land remote from the sea).

Functional requirements for the application:

- Loading of AIS data or of the coordinates of the route points.
- Classification of geographic zones using a pre-trained neural network.
- Construction of a traversal cost map on the basis of the classification (a weight for each zone).
- Search of the optimal route using the A* or the Dijkstra algorithm.
- Visualisation of the constructed route on a map.
- Comparison of the obtained route with the shortest-distance route.
- Visualisation of the constructed route on a map.
- Comparison of the obtained route with the shortest-distance route.
- Export of the results to the GeoJSON, CSV and PDF formats.

Requirements for the database:

- The kind of the database and the means of its implementation are determined by the developer.
- Stored data: coordinates of the points, classified zones, constructed routes, parameters of the routes (algorithm, total cost, execution time).

4. Contents of the explanatory note (list of the questions to be developed)

Introduction

1. System analysis and statement of the problem of creating an application for constructing the course of a maritime vessel
   1.1. Analysis of the subject area and review of existing solutions
   1.2. Statement of the problem of creating an application for constructing the course of a maritime vessel
2. Design of the application for constructing the course of a maritime vessel
   2.1. Development of the architecture of the application
   2.2. Design of the database
   2.3. Development of the model of classification of geographic zones
   2.4. Development of the route construction algorithms

3. Implementation and testing of the application
   3.1. Implementation of the module for loading and pre-processing AIS data
   3.2. Implementation of the neural network for the classification of geographic zones
   3.3. Implementation of the path search algorithms
   3.4. Implementation of the route visualisation module
   3.5. Integration of all modules and testing

4. Technical and economic justification
   4.1. Calculation of the development costs
   4.2. Evaluation of the economic effectiveness

5. Energy and resource saving
   5.1. Analysis of the energy consumption
   5.2. Recommendations for reducing the energy costs

Conclusion

List of abbreviations

List of references

Appendix A. User guide
Appendix Б. Program code of the application
Appendix B. Installation and configuration guide

5. List of the graphic material (with an exact indication of the obligatory drawings and charts):
   1. Statement of the problem of creating an application for constructing the course of a maritime vessel (poster – A1 format)
   2. Architecture of the application (poster – A1 format)
   3. Test results (screenshots of the interface, examples of constructed routes, comparison, poster – A1 format)
   4. Flowchart of the algorithm of classification of geographic zones using a neural network (drawing – A1 format)
   5. Flowchart of the route construction algorithm (drawing – A1 format)

6. Consultants on the project (with an indication of the sections of the project relating to them)
   – on ESPD: [full name, position]
   – on the economic section: [full name, position]

7. Date of issue of the assignment 01 September 2026

8. Calendar schedule of the work on the project for the whole period of design (with an indication of the deadlines and the labour intensity of the individual stages)

   Section 1 (System analysis): 01.09.2026 – 20.09.2026 15%
   Section 2 (Design): 21.09.2026 – 20.10.2026 20%
   Section 3 (Implementation and testing): 21.10.2026 – 10.12.2026 40%
   Section 4 (Economics): 11.12.2026 – 20.12.2026 5%
   Section 5 (Energy saving): 21.12.2026 – 25.12.2026 5%
   Preparation of the project: 26.12.2026 – 15.01.2027 10%
   Preparation for the defence: 16.01.2027 – 31.01.2027 5%

Supervisor _____
(signature)

The assignment is accepted for execution
(date) 01.09.2026

(signature of the student) _____

CONTENTS

INTRODUCTION..... 6
1 SYSTEM ANALYSIS AND STATEMENT OF THE PROBLEM.....8
1.1 Analysis of the subject area and review of existing solutions.....8
1.2 Statement of the problem.....11
2 DESIGN OF THE APPLICATION.....16
2.1 Design of the application architecture.....16
2.2 Design of the database.....19
2.3 Design of the geographic-zone classification model.....21
2.4 Design of the route construction algorithms.....27
3 IMPLEMENTATION AND TESTING OF THE APPLICATION.....40
3.1 Implementation of the AIS loading and pre-processing module.....40
3.2 Implementation of the neural network for zone classification.....41
3.3 Implementation of the path search algorithms.....45
3.4 Implementation of the visualisation module.....49
3.5 Integration of the modules and testing.....53
4 TECHNICAL AND ECONOMIC JUSTIFICATION.....71
4.1 Calculation of the development costs.....71
4.2 Evaluation of the economic effectiveness.....72
5 ENERGY AND RESOURCE SAVING.....76
5.1 Analysis of energy consumption.....76
5.2 Recommendations for reducing energy consumption.....77
CONCLUSION.....80
LIST OF ABBREVIATIONS.....83
LIST OF REFERENCES.....84
Appendix A. User guide.....88
Appendix B. Installation and configuration guide.....100
Appendix C. Results of the test on 100 random voyages.....106
Appendix D. Program text.....117

INTRODUCTION

Maritime transport carries roughly four fifths of world trade by volume, and the fuel burned on that trade is both the largest single item in a ship operator's variable costs and a significant source of greenhouse-gas emissions [28]. The route a vessel actually sails is therefore not a neutral technical detail: it determines fuel consumption, passage time, charter costs, the exposure of the ship to navigational hazards and, ultimately, the safety of the crew and the cargo.

The naive answer to the question "what is the best course between two ports?" is the great circle — the shortest line on the surface of a sphere. For an aircraft at cruising altitude this answer is usually right. For a ship it is almost always wrong, because the shortest line on a sphere takes no account of the fact that most of that sphere is land. A great circle drawn from Kiel to Tallinn crosses Denmark and Sweden; a great circle from Odesa to Novorossiysk crosses the Crimean peninsula. A practical planner must therefore decide, for every point it considers, whether that point is water a vessel may sail through, water it should sail through with caution, or land it must avoid entirely.

Classically that decision is taken by intersecting the candidate route with vector coastline and depth-contour data from electronic navigational charts, as required by the guidelines for voyage planning [1] and by the safety-of-navigation chapter of the SOLAS convention [2]. That approach is accurate where such charts exist and are current, but it is expensive to license and heavy to process, and contour data is sparse in poorly surveyed waters.

This project takes a different route to the same decision. Instead of intersecting geometry, it learns the decision from data. A corpus of 156,838 geo-referenced points, each labelled with one of four geographic zones and each carrying bathymetric and infrastructure attributes, is used to train a neural network that maps a coordinate to a zone. The zones are then given traversal weights, the weights form a cost map over a geographic lattice, and a search — A* [3], Dijkstra [4], dynamic programming [39] or a genetic algorithm [31] — finds the cheapest path across that map. The result is a route that is, by construction, navigable over its whole length, and that can be compared quantitatively with the shortest-distance route it replaces.

The relevance of the work follows from three converging trends. First, open bathymetric and geospatial data — the NOAA and GEBCO grids [24, 25] and OpenStreetMap extracts [26] — have reached a coverage and quality that makes them a viable substitute for licensed chart products in a decision-support role. Second, AIS transponders have made vessel-track data abundant, so a corpus of real positions with real environmental context can be assembled without a dedicated survey. Third, a small neural network can now classify a whole planning lattice of tens of thousands of points in a fraction of a second on an ordinary processor.

The aim of the project is to develop an application for constructing the course of a maritime vessel on the basis of a neural-network classification of geographic zones.

To reach that aim the following tasks were set:

1) analyse the subject area and review existing approaches to maritime route planning and to the classification of geographic zones;
2) formulate the functional and non-functional requirements of the application;
3) design the architecture of the application and the structure of its database;
4) design and train a neural-network model that classifies a geographic point into one of four zones;
5) design and implement the construction of a traversal cost map and the search algorithms that operate over it, including dynamic programming and a genetic algorithm, and compare their evaluation functions;
6) implement the modules for loading and pre-processing AIS data, for classification, for path search, for visualisation and for export;
7) integrate the modules, test the resulting application and evaluate it experimentally against the shortest-distance route;
8) perform the technical and economic justification of the development;
9) analyse the energy consumption of the solution and give recommendations for reducing it.

The object of the study is the process of planning the course of a maritime vessel between two geographic points.

The subject of the study is a method of constructing a safe and economically reasonable route based on a neural-network classification of geographic zones.

The scientific and practical novelty of the work lies in the combination of a learned zone classifier with a classical optimal graph search, in such a way that the learned component supplies the cost model while the search retains its formal guarantee of optimality. The admissibility of the heuristic is preserved by deriving it from the minimum zone weight, and this was confirmed experimentally: on every test voyage A*, Dijkstra and dynamic programming returned exactly the same optimal cost. The same step cost is used as the basis of two evaluation functions — the Bellman function of dynamic programming and the fitness function of a genetic algorithm — which makes the two approaches directly comparable.

The explanatory note consists of an introduction, five sections, a conclusion, a list of abbreviations, a list of references and four appendices. The first section contains the system analysis and the statement of the problem, the second the design of the application, the third its implementation and testing, the fourth the technical and economic justification, and the fifth the analysis of energy consumption. Appendix A is the user guide and appendix B the installation and configuration guide; appendix C records the test of the application on 100 random voyages; appendix D gives the address of the public repository that holds the text of the program.

1 SYSTEM ANALYSIS AND STATEMENT OF THE PROBLEM

1.1 Analysis of the subject area and review of existing solutions

1.1.1 The voyage planning process

Voyage planning is regulated internationally by IMO Resolution A.893(21) [1], which divides the process into four stages: appraisal, planning, execution and monitoring, and by chapter V of the SOLAS convention [2]. The present work addresses the second stage — planning — in which the navigator lays down the intended track on the chart, taking into account the draught of the vessel, the depth of water along the track, the proximity of the shore, traffic separation schemes, weather and the commercial constraints of the voyage.

In the planning stage the navigator repeatedly answers one elementary question: is this particular piece of water safe for this particular ship? Everything else — the choice of waypoints, the decision to take one strait rather than another, the acceptance of a longer track in exchange for deeper water — follows from a sequence of such answers. Automating voyage planning therefore means, first of all, automating that elementary judgement, and only then searching over the space of tracks that it admits.

It is important to state the boundaries of the present work clearly. The application developed here is a decision-support tool. It does not replace the officially approved passage plan, it does not use certified electronic navigational charts, and it must not be used as the sole basis for navigation. Its purpose is to produce a reasoned first approximation of a course, together with a quantitative comparison against the naive shortest-distance route, which a navigator can then refine against official charts and local knowledge.

1.1.2 Approaches to route construction

Three families of approaches to automated maritime route construction can be distinguished in the literature and in practice.

Geometric approaches treat the sea as a polygon with holes and compute a shortest path in the free space, typically over a visibility graph or a navigation mesh built from the coastline. They are exact with respect to the geometry they are given and they produce smooth tracks with few waypoints. Their weakness is that the geometry has to come from somewhere: high-quality vector coastlines are expensive, their generalisation level has to match the scale of

the voyage, and incorporating a continuous quantity such as depth into a polygon model requires slicing the sea into many depth bands, which multiplies the size of the geometry.

Raster approaches overlay a regular lattice on the area of interest, assign a traversal cost to every cell and run a graph search. They are simple, they accept any number of continuous cost factors, and they degrade gracefully where data is sparse. Their weakness is resolution: a strait narrower than one cell is invisible to the planner, and halving the cell size quadruples the number of cells. The present work uses a raster approach and addresses the resolution problem by automatic refinement, described in subsection 2.4.4.

Evolutionary and multi-objective approaches encode a route as a chromosome and optimise it against several objectives at once — fuel, time, safety, comfort — producing a Pareto front of candidate routes rather than a single answer [28]. They handle objectives that a scalar cost cannot express, but they give up the guarantee of optimality, their run time is hard to bound, and the result depends on the random seed. They are best suited to weather routing over several days, where the objectives genuinely conflict.

The present work deliberately stays in the second family and keeps the formal guarantee of optimality [5, 6], because the novel component — the learned cost model — is easier to evaluate when the search around it is exact. If the route is suboptimal it is the cost model that must be blamed, not the search.

1.1.3 Existing software

Table 1.1 compares representative existing solutions with the application developed in this project.

Table 1.1 – Comparison of existing solutions

Solution	**Class**	**Cost model**	**Optimality**	**Availability**
ECDIS route check (e.g. Transas, Wärtsilä)	Commercial, certified	Vector ENC geometry and depth contours	Checks a manual route; does not optimise	Licensed, ship-installed
Weather routing services (e.g. StormGeo, DTN)	Commercial service	Weather, currents, fuel model	Heuristic / multi-objective	Subscription
OpenSeaMap + generic GIS shortest path	Open source	Coastline polygons only	Exact over the given geometry	Free, no depth model
searoute / sea-distance libraries	Open source library	Pre-built maritime network graph	Exact over a fixed network	Free, fixed network, no local detail
Application developed in this project	Open, research / teaching	Learned four-zone classification from bathymetry and OSM context	Exact (A* with an admissible heuristic)	Free, runs offline on an ordinary computer

Two gaps follow from the comparison. Certified systems check a route that the navigator has already drawn, rather than proposing one, and they are not available outside a licensed installation. Open solutions that do propose a route either ignore depth entirely, working from a coastline polygon, or are restricted to a fixed pre-built network of ports and legs, which cannot

express an arbitrary pair of coordinates. The application developed here fills that gap: it proposes a route between arbitrary points, it takes depth and shore proximity into account through a learned model, and it runs without a licence on an ordinary computer.

1.1.4 Classification of geographic zones

The four zones required by the assignment are defined in table 1.2.

Table 1.2 – Definition of the geographic zones

Code	**Meaning**	**Navigable**	**Median depth / height in the corpus as delivered**
OPEN_SEA	Open sea — deep water far from any shoreline	yes	-2156 m
COASTAL_SEA	Coastal sea — navigable water close to the shore	yes	-27 m
NEAR_COAST	Near coast — land not further than 5 km from the shoreline	no	+10 m
COASTLINE	Inland — land remote from the sea	no	+216 m

The two water classes are separated by depth and by distance from the shore; the two land classes are separated by distance from the shore alone. The distinction between them matters to the planner for different reasons. OPEN_SEA and COASTAL_SEA both admit a vessel but carry different risk, so they receive different finite weights. NEAR_COAST and COASTLINE are both impassable, so their weights are identical; they are nevertheless kept apart because the boundary between them is what defines the shoreline, and a model that can place that boundary correctly is a model that understands the coast.

Figure 1.1 shows why the task is not trivial. The two extreme classes are separated by thousands of metres of water column and could be told apart by a threshold on depth alone. The two middle classes, however, occupy almost the same narrow band around sea level: in the corpus as delivered the fifth-to-ninety-fifth percentile range of COASTAL_SEA is -98 … 5 m and that of NEAR_COAST is -6 … 81 m. These two ranges overlap, so no threshold on depth can separate them, and the classifier must draw on spatial context instead.

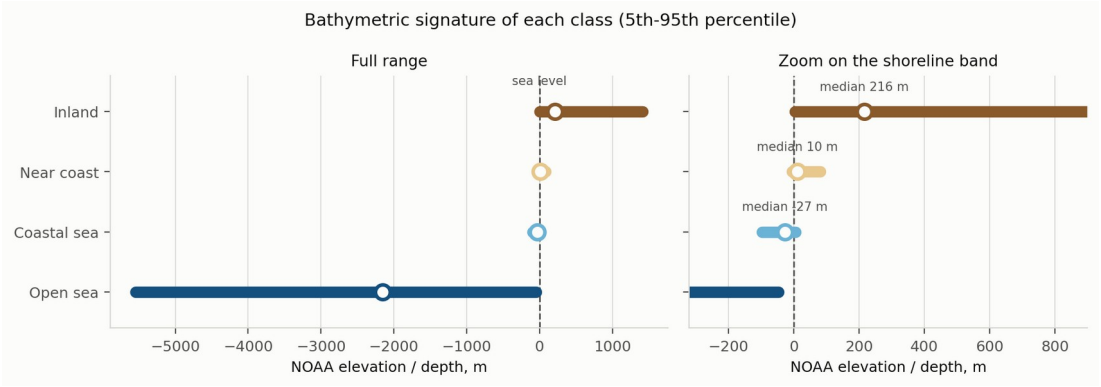


Figure 1.1 – Bathymetric signature of the four classes. Left: the full range. Right: a zoom on the shoreline band, where the two middle classes overlap

1.2 Statement of the problem

1.2.1 General statement

It is required to develop an application that, given a departure point and a destination point on the surface of the Earth, constructs a route between them that lies entirely within navigable water, minimises a traversal cost derived from a neural-network classification of geographic zones, and presents the result together with a quantitative comparison against the shortest-distance route.

Formally, let a geographic point be a pair q = (φ, λ) of latitude and longitude. Let Z = {OPEN_SEA, COASTAL_SEA, NEAR_COAST, COASTLINE} be the set of zones and let f: q → Z be a classifier. Let w: Z → $\mathbb{R}^+ \cup \{\infty\}$ assign a traversal weight to each zone. For a route R = (q_0, q_1, …, q_k) the cost is defined by formula (1.1):

$$C(R) = \Sigma_i \, d(q_{i-1}, q_i) \cdot ½ \cdot [\, w(f(q_{i-1})) + w(f(q_i)) \,], \qquad (1.1)$$

where d is the great-circle distance. The problem is to find a route R* from a given q_{start} to a given q_{end} that minimises C(R) subject to $w(f(q_i)) < \infty$ for every intermediate point. Because the departure and destination are berths chosen by the user, and a quay legitimately lies in a shore zone, the constraint is imposed on the interior of the route only.

1.2.2 Functional requirements

The functions the application shall provide are listed in table 1.3. Requirements F10–F12 were added after the review of the first version of the application by the supervisor, requirement F13 in the same version.

Table 1.3 – Functional requirements

ID	Requirement	Realised in
F1	Load AIS data or route-point coordinates from GeoJSON, CSV or JSON	Subsection 3.1
F2	Classify geographic zones with a pre-trained neural network	Subsections 2.3, 3.2
F3	Build a traversal cost map from the classification, with a weight per zone	Subsections 2.4.1, 3.3
F4	Search the optimal route with the A* or the Dijkstra algorithm; in addition, dynamic programming and a genetic algorithm	Subsections 2.4.2–2.4.3, 2.4.6–2.4.7, 3.3
F5	Visualise the route on an interactive map	Subsection 3.4
F6	Compare the route with the shortest-distance (great-circle) route	Subsections 2.4.5, 3.5.3
F7	Export the results to GeoJSON, CSV and PDF	Subsection 3.4.4
F8	Store points, classified zones, routes and route parameters in a database	Subsections 2.2, 3.5.1
F9	Show the progress of classification and search in real time	Subsection 3.4.2
F10	Set the parameters of every search method in the application — lattice, step limits, parameters of the genetic algorithm, time limit — not in the code	Subsections 2.4.8, 3.4.5
F11	Show the elapsed time of a search and let the user stop it; save the result of every search, successful or not, to a file whose location is shown on the screen	Subsections 2.4.8, 3.3.7, 3.4.5
F12	Construct complex courses: through intermediate points set by the user, out of the rectangle of the two ports, through straits narrower than a cell	Subsections 2.4.9, 3.5.8–3.5.10
F13	Show the interface in English, Russian or Chinese, switched by the user without restarting the application or repeating a search	Subsection 3.4.6

1.2.3 Non-functional requirements

The non-functional requirements and the way each of them was verified are collected in table 1.4.

Table 1.4 – Non-functional requirements and their verification

ID	Requirement	Target	Achieved
N1	Classification accuracy on a held-out test set	≥ 90 %	96.95 %
N2	Macro F1 over the four classes	≥ 0.90	0.9695
N3	Planning time for a voyage of about 1 000 km	≤ 5 s	≤ 2.9 s on the benchmark voyages; 2.1 s on average, up to 53 s on the random test
N4	Interior of the produced route free of land	100 %	17 of 17 test voyages by the classifier; on the random test by an independent land mask 71 of 90 routes without the coastline check and 90 of 90 with it (subsections 3.5.6, 3.5.7)
N5	Optimality of the search	exact	A*, Dijkstra and dynamic programming agree, deviation 0 %
N6	Operation without a network connection	required	on-board chart, vendored client libraries
N7	Hardware	ordinary computer, no GPU	CPU only, 4 GB RAM
N8	Automated test coverage of the modules	all modules	139 tests, all passing
N9	Reaction to the Stop button and to the time limit	≤ 1 s	≤ 0.22 s (subsection 3.5.10)

1.2.4 Input data

The source of training data is a labelled corpus of AIS observations supplied in GeoJSON format. It contains 156,838 point features collected in 28 recording sessions and spanning latitudes from -40.1° to 67.1° and longitudes from -87.9° to 168.3°. Every feature carries the attributes listed in table 1.5.

Table 1.5 – Attributes of a point in the source corpus

Attribute	Type	Meaning
timestamp	string	time of the observation (ISO 8601)
latitude, longitude	real	geographic coordinates, degrees
elevation	real	surface elevation, m (may be absent)
class_code	string	zone label: one of the four classes
class_name	string	human-readable name of the zone
source_session	integer	identifier of the recording session
noaa_depth	real	NOAA depth or height, m (negative below sea level)

Attribute	Type	Meaning
noaa_type	string	“depth” or “height” — which quantity was sampled
osm_piers … osm_industrial	integer	nine counters of OpenStreetMap objects near the point

The corpus is moderately imbalanced: OPEN_SEA accounts for 36.3 % of the points, COASTAL_SEA for 24.9 %, NEAR_COAST for 20.4 % and COASTLINE for 18.4 %. In the corpus as delivered, elevation is missing for 7,066 points and the NOAA sample for 314 points; both gaps are handled by the feature extractor rather than by discarding the records, as described in subsection 2.3.3.

The corpus was delivered in two releases. The second release, supplied by the supervisor, keeps every point of the first one and adds 4,000 points of the Aegean Sea — 1,000 of each class, spread evenly over the rectangle from 34.4° to 41.3° N and from 19.1° to 30.9° E. Together with the 2,347 points of the first release that lie in the same rectangle they form the Aegean Sea region of 6,347 points. All numbers in this note refer to the second release.

The points were not collected in one sea but in eleven separate regions of the world ocean, listed in table 1.6. The regions were identified by clustering the coordinates of the corpus, the Aegean Sea being the region added in the second release, and are described in the configuration by rectangles of latitude and longitude; they are used to report the accuracy of the model region by region (subsection 3.2.6) and to build example routes in every region (subsection 3.5.5).

Table 1.6 – Sea regions of the corpus (after cleaning)

Region	Points	OPEN_SEA	COASTAL_SEA	NEAR_COAST	COASTLINE
North Europe (Baltic and North Sea)	29,693	6,333	9,790	6,616	6,954
South China Sea	19,442	11,516	4,031	2,310	1,585
Japan and North-West Pacific	18,425	11,137	2,329	2,285	2,674
Southern Africa (Cape of Good Hope)	15,770	5,563	5,227	3,128	1,852
Black Sea and Eastern Mediterranean	13,670	4,704	2,864	2,905	3,197
Persian Gulf and Arabian Sea	13,351	6,147	2,548	2,267	2,389
Red Sea and Gulf of Aden	11,162	1,782	3,452	3,150	2,778
Central America (Caribbean coast)	9,721	2,468	1,936	2,860	2,457
Strait of Malacca and Andaman Sea	8,386	3,329	2,091	1,743	1,223
Aegean Sea	6,124	1,413	1,615	1,781	1,315
Bay of Bengal	4,695	1,199	1,098	1,200	1,198
Other (inland samples)	972	63	0	0	909

Coverage outside these regions is thin, and the model should not be expected to classify distant seas reliably. A small share of the labels contradicts the measured height of the point — for example a coastal-sea point several metres above sea level. Such points are removed before training, as described in subsection 2.3.7.

The OpenStreetMap counters are sparse: fewer than one point in a hundred has a non-zero value in any of the nine columns. They are nevertheless retained, because where they are non-zero they are highly informative — a pier or a lighthouse is direct evidence of a shoreline — and because the feature extractor aggregates them over a neighbourhood, which turns a sparse point attribute into a dense contextual one.

1.2.5 Conclusions on the chapter

The analysis of the subject area shows that the practical difficulty of maritime route planning lies not in the search itself, which is a solved problem, but in the cost model that the search operates on. Existing open solutions either omit depth or are confined to a fixed network of legs; certified systems verify a route rather than propose one. The problem has therefore been stated as the construction of a learned four-zone cost model together with an exact search over it, and the functional and non-functional requirements of such an application have been formulated in tables 1.3 and 1.4.

2 DESIGN OF THE APPLICATION

2.1 Design of the application architecture

2.1.1 Architectural style and its justification

The application is built as a layered monolith with a single-page web client. Five layers are distinguished, each depending only on the layers below it: presentation, service, domain, data and export.

The responsibilities of the five layers are summarised in table 2.1. A layered monolith was chosen rather than a set of services for three reasons. The application is used by one operator at a time, so there is no load to distribute. The classifier holds a reference index of about a hundred thousand samples in memory, and splitting it across processes would multiply that memory rather than save it. And the dominant cost of a planning request is a single tight loop — classify the lattice, then search it — which is latency-sensitive and would only suffer from a network hop in the middle. A web client was nevertheless chosen over a desktop toolkit, because the map is the centre of the interface and browser mapping libraries are far ahead of desktop equivalents, and because a browser client runs unchanged on the operating systems found on a ship’s bridge.

Table 2.1 – Layers of the application

Layer	**Responsibility**	**Principal modules**
Presentation	Map, controls, live progress display, charts, interface languages	index.html, i18n.js, app.js, api.js, charts.js
Service	HTTP endpoints, WebSocket channel, chart tiles	web/app.py, web/tiles.py
Domain	Classification façade, planning orchestration, cost map, search	model/inference.py, routing/planner.py, routing/coast_check.py, routing/cost_map.py, routing/astar.py, routing/dijkstra.py, routing/dynamic_programming.py, routing/genetic.py
Data	Loading, validation, feature engineering, persistence	data/ais_loader.py, data/cleaning.py, data/features.py, data/dataset.py, storage/repository.py
Export	GeoJSON, CSV and PDF writers	export/exporters.py

2.1.2 Decomposition into modules

Figure 2.1 shows the layered structure together with the request sequence of a single planning job and the schema of the database.

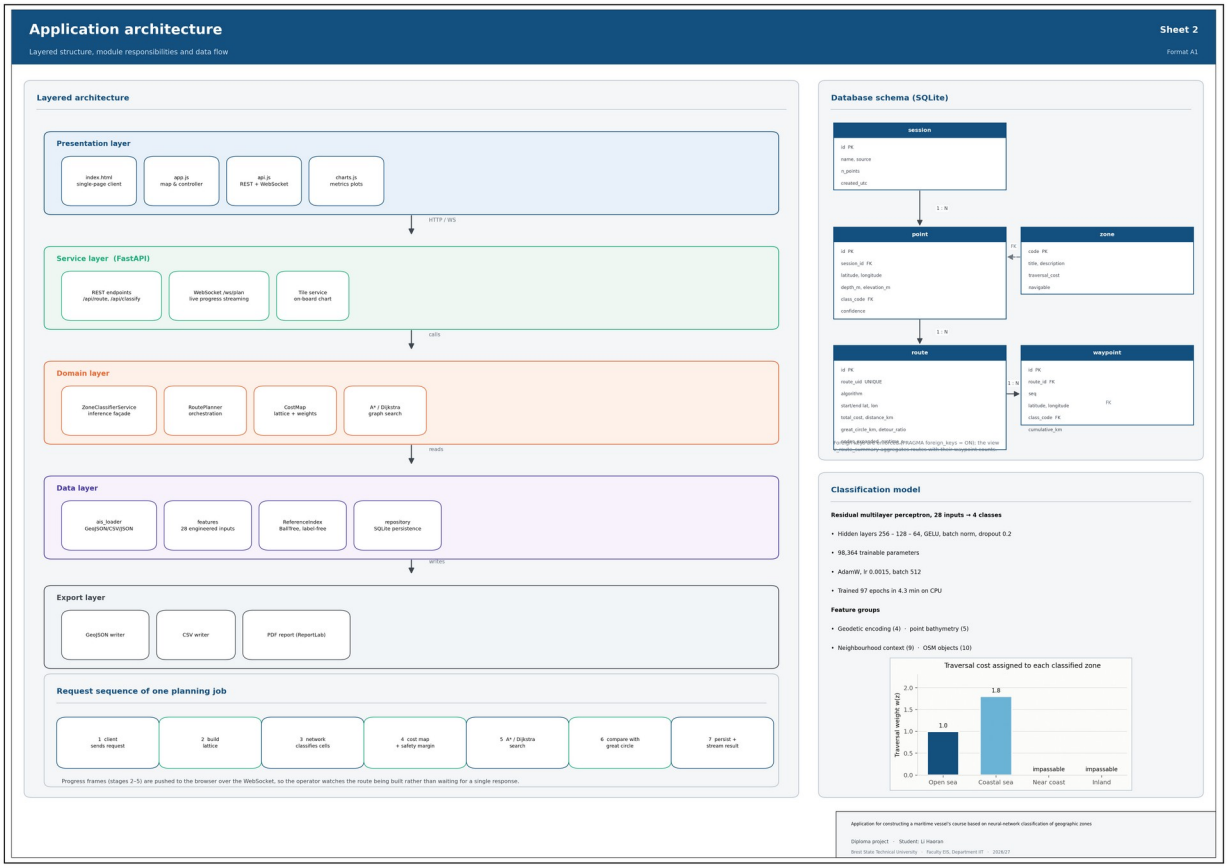


Figure 2.1 – Architecture of the application (sheet 2 of the graphic material)

The dependency rule is enforced by construction: a module of the domain layer never imports from the service layer, and the data layer knows nothing about either. All configurable values — paths, the zone taxonomy, the traversal weights, the hyper-parameters of the network and of the planner — are collected in a single module, config.py, so that no other module contains a hard-coded path, class index or weight.

2.1.3 The planning request sequence

A planning request passes through seven stages:

1) the client sends the departure, the destination and the planning parameters over the WebSocket;
2) the service builds a geographic lattice around the two points with a margin;
3) the classifier evaluates every cell centre of the lattice in batches;
4) the predicted zones are mapped to traversal weights and a safety margin is applied along the shore;
5) the chosen method — A*, Dijkstra, dynamic programming or the genetic algorithm — searches the cheapest path between the snapped endpoints;

6) the great circle between the same endpoints is evaluated on the same cost map for comparison;
7) the route is persisted in the database and streamed back to the client.

Stages two to five push progress frames to the client, so that the operator sees the lattice being classified and the search front advancing rather than waiting for a single response. The channel follows RFC 6455 [22] and complements the REST interface [23]. The design of that channel is described in subsection 2.1.4.

2.1.4 The real-time interaction channel

Interactivity is a requirement of the assignment, and it is not satisfied by a request-response interface alone: classifying a large lattice and searching it can take a second or more, during which a plain HTTP request would leave the interface frozen with no indication of progress. A WebSocket channel was therefore designed alongside the REST interface.

The channel carries the message types listed in table 2.2. The planning job runs in a worker thread; progress callbacks from that thread are marshalled onto the event loop through a queue, and frames are throttled to a configurable minimum interval so that a fast search cannot flood the client with thousands of messages. In the first version the connection was not read while a job ran, so a search could not be interrupted; since version 1.3 the job runs as a separate task of the connection, the connection keeps receiving messages, and a cancel message sets the stop flag of the running search (subsection 2.4.8).

Table 2.2 – Frame types of the real-time channel

Frame	**Direction**	**Content**
hello	server → client	protocol version and the zone palette
plan	client → server	departure, destination, intermediate points, method and its parameters
classify	client → server	coordinates to classify (live pointer read-out)
started	server → client	acknowledgement, chosen algorithm and resolution
progress	server → client	stage, fraction complete, cells done or nodes expanded, elapsed time
overlay	server → client	down-sampled classification of the lattice
result	server → client	the complete route plan — also when no route was found or the search was stopped — and the path of its result file
benchmark	client → server / server → client	request to compare all methods on one cost map; the comparison and the path of its file
cancel	client → server	stop the running search (Stop button)
cancelling	server → client	acknowledgement that the search is being stopped
classified	server → client	zone, confidence and depth of the queried points

Frame	Direction	Content
error	server → client	a human-readable description of the failure
ping / pong	both	keep-alive

The client reconnects automatically with exponential back-off, and it queues frames sent while the socket is down. If the socket cannot be established at all, the client falls back to the equivalent REST endpoint, so the application remains usable — only without live progress — behind a proxy that forbids WebSocket upgrades.

2.2 Design of the database

2.2.1 Choice of the database management system

The assignment leaves the choice of the database to the developer. Three candidates were considered and compared in table 2.3.

Table 2.3 – Comparison of the candidate database management systems

Criterion	SQLite	PostgreSQL + PostGIS	Files (JSON / CSV)
Administration	none — a single file	server, users, backups	none
Transactional integrity	full ACID	full ACID	none
Foreign keys, views	yes	yes	no
Spatial operators	through an extension	native and rich	no
Deployment on a ship	trivial	heavy	trivial
Concurrency needed here	one operator	many clients	—
Suitability for this project	high	excessive	insufficient

SQLite was chosen. The application is a single-operator tool that must run on an ordinary computer, possibly offline and possibly with no administrator available; SQLite requires no server, stores the whole database in one file that can be copied as a backup, and still provides full transactional SQL with foreign keys, indexes and views. The spatial operators of PostGIS are not needed, because the only spatial query the application performs — the nearest-neighbour lookup in the reference index — is served by an in-memory ball tree that is far faster than a database round trip. Should a multi-user deployment ever be required, the repository pattern used in the data layer confines the change to a single module.

2.2.2 Logical model

Six relations and one view make up the schema, as table 2.4 shows. The zone table is a reference relation seeded at start-up from the configuration, so that the traversal weights stored in the database can never drift away from the weights the planner actually uses.

Table 2.4 – Relations of the database

Relation	**Purpose**	**Principal attributes**
session	One ingestion of AIS data or one interactive session	id, name, source, n_points, created_utc
point	A geographic point with its classification result	id, session_id, latitude, longitude, depth_m, class_code, confidence
zone	Reference table of the four zones	code, title, description, traversal_cost, navigable, colour
route	A planned route with its aggregated parameters	id, route_uid, algorithm, start/end coordinates, total_cost, distance_km, great_circle_km, detour_ratio, nodes_expanded, runtime_s, feasible, coast_check, status, parameters, result_file
waypoint	An ordered point of a route	id, route_id, seq, latitude, longitude, class_code, cumulative_km, bearing_deg
model_run	Traceability of the deployed classifier	id, version, accuracy, macro_f1, cohen_kappa, parameters, trained_utc
v_route_summary	View: routes with their waypoint counts	route_uid, algorithm, distance_km, detour_ratio, n_waypoints

The attributes stored for a route are exactly those required by the assignment — the algorithm used, the total cost and the execution time — together with the quantities needed to reproduce and to compare the route: the endpoints, the lattice resolution, the number of expanded nodes, the great-circle distance and the serialised comparison against it.

2.2.3 Integrity and normalisation

The schema is in the third normal form. Every relation has a surrogate primary key; route additionally carries a unique business key, route_uid, which is the identifier exposed to the client and used in exported file names. Referential integrity is declared and enforced — the connection sets PRAGMA foreign_keys = ON — with cascade deletion from route to waypoint and from session to point, so that removing a route cannot leave orphaned waypoints behind.

Domain constraints are declared in the schema rather than only in the application: latitude is constrained to [−90, 90], longitude to [−180, 180], confidence to [0, 1], the algorithm to the five supported values and the boolean columns to zero or one. Non-finite values — an infinite cost on an infeasible route, a not-a-number detour ratio — are converted to NULL before insertion, so that SQL aggregates over these columns remain meaningful.

Three indexes support the access patterns that actually occur: point by session, point by coordinates, and route by creation time for the history list. No further index was added, because the remaining queries are either primary-key lookups or full scans of a small relation.

2.3 Design of the geographic-zone classification model

2.3.1 Formulation of the learning task

The task is a four-way classification of a geographic point. The natural input is the pair of coordinates, but coordinates alone carry almost no information about navigability: the same latitude and longitude would have to be memorised individually, and the model would not generalise one metre beyond the training corpus. The design problem is therefore the design of the features, not the design of the network.

Three requirements were imposed on the feature set. It must be computable for an arbitrary coordinate, not only for a point of the corpus, because the planner has to classify lattice cells that were never observed. It must not leak the target, directly or indirectly. And it must be reproducible in a deployment where the corpus is absent, so that the same features can be derived from public data sources.

2.3.2 The reference index

These requirements are satisfied by a label-free reference index. The index stores, for every training point, only its coordinates, its bathymetric sample, its elevation and its nine OpenStreetMap counters — never its class label. It is queried by a ball tree under the haversine metric, which returns the k nearest samples to an arbitrary coordinate in logarithmic time [16], the distance being computed by the haversine formula [17].

The index plays exactly the role that a NOAA or GEBCO bathymetry raster and an OpenStreetMap extract would play in a production deployment: it answers the question “what does the sea floor look like around this point, and what man-made objects are near it?”. Using it at inference time is therefore not a shortcut but a stand-in for a public data source, and the same features can be computed anywhere on Earth once that source is attached.

Three measures guarantee that no target information leaks through the index:

- the index stores no labels, so a leak would have to be indirect;
- the index is built from the training split alone — validation and test points are never inserted into it, so a test point cannot retrieve itself or a neighbour that was chosen with knowledge of its label;

- during the construction of the training features a point never queries itself: the nearest neighbour, which would be the point itself at distance zero, is discarded.

The corpus is split before the index is built, which is the order that makes the second measure possible. The split is stratified by class, in the proportion 70 : 15 : 15, giving 105,987 training, 22,712 validation and 22,712 test points.

2.3.3 The feature vector

Twenty-eight features in four groups are derived for every point.

Table 2.5 – Groups of input features

Group	Count	Features and their motivation
Geodetic encoding	5	sin φ, cos φ, sin λ, cos λ and the normalised absolute latitude. Trigonometric encoding is used instead of raw degrees so that the representation is continuous across the antimeridian and across the poles; a raw longitude would place −179° and +179° at opposite ends of the input range although they are neighbours.
Point measurements	5	signed logarithm of the bathymetric sample, its missing-value indicator, the height/depth type flag, signed logarithm of the elevation, its missing-value indicator. The signed logarithm $\mathrm{sign}(x)\cdot\ln(1+\lvert x\rvert)$ compresses a range of almost ten kilometres into a span the network can use while preserving the sign, which is the single most important bit of the input.
Neighbourhood context	9	mean, standard deviation, minimum, maximum and range of the depth of the eight nearest samples; the fraction of those samples that lie above sea level; agreement between the sign of the point and the sign of its neighbourhood; the logarithm of the distance to the nearest sample; the vertical gradient. This group is what separates the two middle classes: a point at −5 m surrounded by land is coastal water, the same depth surrounded by water is not.
OpenStreetMap objects	9	marine infrastructure (piers, lighthouses, breakwaters, harbours), natural coastal objects (beaches, cliffs) and inland water (rivers, lakes), each taken at the point and averaged over the neighbourhood; industrial objects at the point; the total and the density of objects in the neighbourhood.

The missing-value indicators deserve a comment. Rather than discarding the 6,023 points with no elevation and the 314 with no bathymetric sample, the extractor imputes the value from the interpolated neighbourhood and raises a binary flag. The network therefore learns both from the imputed value and from the fact that it was imputed, which is strictly more information than either discarding the record or imputing silently.

2.3.4 Architecture of the network

The architecture and the hyper-parameters adopted are collected in table 2.6. The input is a short, dense, tabular vector, which rules out convolutional and recurrent architectures; a multilayer perceptron is the appropriate family [7]. A plain perceptron was found to converge slowly on the boundary between the two middle classes, so residual blocks [8] were adopted:

each block applies a linear map, batch normalisation [9], a GELU non-linearity [10] and dropout [12], and adds a projected skip connection.

Table 2.6 – Architecture and hyper-parameters of the classifier

Parameter	**Value**	**Justification**
Architecture	Residual MLP	tabular input; skip connections speed convergence on the hard boundary
Input dimension	28	see table 2.5
Hidden layers	256 – 128 – 64	a narrowing funnel; wider layers gave no measurable gain
Activation	GELU	smooth, avoids dead units
Normalisation	Batch normalisation	the input features differ in scale by orders of magnitude
Dropout	0.2	regularisation; higher values under-fit
Output	4 logits → softmax	four mutually exclusive classes
Trainable parameters	98,364	small enough for sub-millisecond batched inference on a CPU
Loss	Cross-entropy, class-weighted, label smoothing 0.02	the corpus is imbalanced; smoothing limits over-confidence
Optimiser	AdamW [11], lr 0.0015, weight decay 0.0001	decoupled weight decay regularises better than L2 in Adam
Batch size	512	a compromise between gradient noise and speed
Schedule	ReduceLROnPlateau, factor 0.5	halves the rate when validation loss stalls
Early stopping	patience 15 epochs	restores the best weights by validation loss
Random seed	42	reproducibility

Class weighting is applied as the inverse of the class frequency, normalised to a mean of one, so that the under-represented COASTLINE class is not sacrificed to the over-represented OPEN_SEA class. Label smoothing [13] of 0.02 keeps the network from driving its logits to saturation, which matters here because the confidence of the prediction is displayed to the operator and should be informative rather than uniformly close to one.

2.3.5 Forward pass of the network

The classifier is a feed-forward network: the input vector passes through a batch-normalisation layer, three hidden blocks and a linear output layer, and each layer depends only on the one before it. A network of this kind with enough hidden neurons can approximate any continuous mapping on a bounded domain [34], which is the theoretical basis for using it to classify a vector of features. Before it enters the network, every feature is standardised to zero mean and unit variance with the statistics of the training split, so that depths in metres, distances in kilometres and object counts enter on the same scale.

A layer of m neurons computes an affine transformation of its input, as formula (2.1) shows:

$$z = W \cdot h + b \tag{2.1}$$

where $h \in \mathbb{R}^n$ is the input of the layer, $W \in \mathbb{R}^{m \times n}$ the matrix of weights, $b \in \mathbb{R}^m$ the vector of biases. Each neuron forms a weighted sum of all n inputs; the weights and biases are the parameters that training adjusts. They are initialised by the Kaiming uniform scheme, which chooses the spread of the initial weights so that the variance of the signal is preserved from layer to layer.

The output of the layer is normalised over the mini-batch — batch normalisation [9], formula (2.2):

$$\hat{z} = \gamma \cdot (z - \mu_B) / \sqrt{(\sigma^2_B + \varepsilon)} + \beta \tag{2.2}$$

where μ_B and σ^2_B are the mean and the variance of z over the current mini-batch of 512 examples, $\varepsilon = 10^{-5}$ prevents division by zero, and γ and β are a learned scale and shift. During training the statistics of the mini-batch are used and running averages are accumulated; at inference the running averages are used, so the result for a point does not depend on the other points of the batch.

The non-linearity is the Gaussian error linear unit GELU [10], formula (2.3):

$$GELU(z) = z \cdot \Phi(z) \tag{2.3}$$

where Φ is the distribution function of the standard normal distribution. GELU passes large positive values almost unchanged, suppresses large negative ones and is smooth in between, so the gradient never vanishes abruptly as it does for the rectified linear unit. Without a non-linearity a stack of layers would reduce to one affine map, and the classes could be separated only by hyperplanes.

During training, dropout [12] sets each output of the main path to zero with probability 0.2 and multiplies the remaining outputs by $1/(1 - 0.2)$; the network cannot rely on any single neuron, which reduces over-fitting. At inference dropout is switched off. The four operations form one residual block [8], whose output is given by formula (2.4):

$$h_{l+1} = Dropout(GELU(BN(W_l \cdot h_l + b_l))) + P_l \cdot h_l \tag{2.4}$$

where P_l is a linear projection of the input onto the width of the block. The widths $28 \rightarrow 256 \rightarrow 128 \rightarrow 64$ all differ, so each of the three blocks has its own projection. The term $P_l \cdot h_l$

gives the gradient a short path back to the earlier layers, which is what made the boundary between the two middle classes converge faster than in a plain perceptron.

The output layer maps the last hidden vector to four numbers $z_1, \ldots, z_4$ (logits), which the softmax function turns into class probabilities, formula (2.5):

$$p_k = exp(z_k) / \Sigma_j exp(z_j), \quad k = 1, \ldots, 4 \tag{2.5}$$

The predicted zone is the class with the largest probability, and that probability is shown to the operator as the confidence of the prediction. The whole network has 98,364 trainable parameters.

2.3.6 Training of the network

The network is trained by supervised learning: the zone y of every training point is known, and the parameters θ — all weights, biases, γ and β — are adjusted to minimise a loss that measures the disagreement between the predicted probabilities and the known zones. The loss is the class-weighted cross-entropy with label smoothing, formula (2.6):

$$L(\theta) = - \Sigma_i \Sigma_k \alpha_k \cdot q_{ik} \cdot ln\, p_{ik} / \Sigma_i \alpha_{yi} \tag{2.6}$$

where i runs over the examples of a mini-batch, k over the four classes, p_{ik} is the probability (2.5) of class k for example i, and q_{ik} is the smoothed target: $1 - \varepsilon + \varepsilon/4$ for the true class and $\varepsilon/4$ for the others, with $\varepsilon = 0.02$ [13, 38]. The class weight $\alpha_k = N / (4 \cdot N_k)$, normalised to a mean of one, where N is the number of training points and N_k the number of those of class k, compensates for the imbalance of the corpus.

The gradient of the loss with respect to every parameter is computed by back-propagation [33]: the chain rule is applied layer by layer from the output back to the input, so that all the partial derivatives are obtained at the cost of about one additional pass through the network. The norm of the gradient is clipped to 5.0, which protects the first epochs, when the loss is large, from excessive steps. The parameters are then updated by the AdamW optimiser [11, 29], formulas (2.7) and (2.8):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{2.7}$$

$$\theta_t = \theta_{t-1} - \eta \cdot [\, \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon) + \lambda \cdot \theta_{t-1} \,] \tag{2.8}$$

where g_t is the gradient at step t; m_t and v_t are running averages of the gradient and of its square with $\beta_1 = 0.9$ and $\beta_2 = 0.999$; $\hat{m}_t$ and $\hat{v}_t$ are the same averages corrected for their zero initialisation; $\delta = 10^{-8}$ prevents division by zero; $\eta = 0.0015$ is the learning rate and $\lambda = 0.0001$ the weight decay. Every parameter thus receives its own step size, inversely proportional to the typical size of its gradient. AdamW applies the weight decay to the parameters directly rather than through the gradient, which regularises better than an L2 term in the loss [11].

One pass over the 105,987 training points, in 208 mini-batches, is an epoch. After every epoch the loss is computed on the validation split, which takes no part in the updates. When the validation loss has not improved for more than 6 epochs the learning rate is halved; when it has not improved for 15 epochs training stops and the weights of the epoch with the lowest validation loss are restored (early stopping [37]). The test split is used once, for the final evaluation in subsection 3.2.3.

Besides the neural network the application uses two further groups of artificial-intelligence methods. The reference index of subsection 2.3.2 is a nearest-neighbour model: it describes the surroundings of an arbitrary point by the measurements of the closest known points. The route search of section 2.4 uses the problem-solving methods of AI [5] — heuristic search, dynamic programming and evolutionary search.

2.3.7 Cleaning of the training data

A corpus labelled from several sources contains labelling errors, and a classifier trained on them learns the errors together with the rule [32]. In the supplied corpus the most reliably detectable error is a contradiction between the class of a point and its measured height: a sea point that lies several metres above sea level, or a land point that lies below it. Every point carries two independent height measurements, both positive above sea level: the NOAA relief value (attribute noaa_depth) and the elevation recorded by the data collector (attribute elevation). Each of them is checked separately. With $C_w = \{\text{OPEN_SEA, COASTAL_SEA}\}$ the set of water classes and $C_l = \{\text{NEAR_COAST, COASTLINE}\}$ the set of land classes, a point is removed when condition (2.9) holds for at least one of its measurements:

$$(y \in C_w \ \text{and} \ h_k > \tau) \quad \text{or} \quad (y \in C_l \ \text{and} \ h_k < -\tau), \quad k \in \{noaa, elev\} \tag{2.9}$$

where y is the label, h_k the k-th measured height and τ a tolerance that absorbs tides and the vertical error of the relief models. A missing measurement is not checked, and a point without any measurement is kept. The number of points removed at different tolerances is given in table 2.7.

Table 2.7 – Points removed by the consistency check

Tolerance τ, m	**0**	**2**	**5**	**10**	**20**
Points removed	7,546	6,523	5,427	4,124	2,558
Share of the corpus	4.81 %	4.16 %	3.46 %	2.63 %	1.63 %

The tolerance of 5 m was adopted. A smaller value begins to remove genuine beach and tidal-flat points, whose height legitimately differs from zero by a metre or two; a larger one keeps obvious errors such as a coastal-sea point on a ten-metre bank. At 5 m the check removes 5,427 points, 3.46 % of the corpus: 1,297 OPEN_SEA and 2,111 COASTAL_SEA points above sea level, and 1,707 NEAR_COAST and 312 COASTLINE points below it. The conflict is revealed by the NOAA value alone for 3,900 points, by the collector's elevation alone for 332 and by both for 1,195. 151,411 points remain, which is exactly the number of clean points stated by the supervisor for the second release of the corpus; this confirms that the rule reproduces the intended cleaning. The check is applied before the corpus is split, so the validation and test splits are cleaned as well. Its effect on the model is analysed in subsection 3.2.5.

2.4 Design of the route construction algorithms

2.4.1 The cost map

The search operates on a regular lattice in geographic coordinates, built around the bounding box of the departure and the destination with a margin of two degrees. The margin matters: without it, a route that has to go around a peninsula lying outside the bounding box could not be found.

Every cell centre is classified, and the predicted zone is mapped to a traversal weight through table 2.8. The two land classes receive an infinite weight, which the implementation represents by a large finite constant so that the weights can be stored in an ordinary numeric array.

Table 2.8 – Traversal weights

Zone	**Weight w(z)**	**Interpretation**
OPEN_SEA	1.0	reference cost of one kilometre of open-water passage
COASTAL_SEA	1.8	navigable, but shallower, busier and closer to hazards; one kilometre here is treated as costly as 1.8 km of open sea
NEAR_COAST	∞	land — impassable
COASTLINE	∞	land — impassable

The ratio of 1.8 between coastal and open water is a policy parameter rather than a physical constant. It was chosen so that the planner accepts a detour of up to roughly eighty per

cent of the length of a coastal leg in order to reach open water, which corresponds to the practice of standing well off a lee shore, while still allowing coastal passage where no open-sea alternative exists — as in the Danish straits, where the whole route is necessarily coastal. The parameter is exposed in the configuration, and the effect of changing it is examined in subsection 3.5.4.

A safety margin is then applied: every water cell orthogonally adjacent to a land cell has its weight multiplied by 1.6. This produces a penalised belt one cell wide along every shoreline, so that the optimiser prefers to stand off the coast where it can, without forbidding coastal passage where it cannot. The belt is computed by an iterative dilation of the land mask, which costs a handful of array operations and no additional classification.

2.4.2 The cost of an edge and the A* heuristic

The lattice is treated as an eight-connected graph. The cost of moving from cell u to an adjacent cell v is the great-circle length of the step multiplied by the mean of the two cell weights, as formula (2.10) shows:

$$c(u, v) = d_{gc}(u, v) \cdot \tfrac{1}{2} \cdot [\, w(z_u) + w(z_v) \,] \tag{2.10}$$

Averaging the two weights rather than taking the weight of the destination makes the cost symmetric, so that the cost of a route does not depend on the direction in which it is sailed. The poly-line the search returns follows the lattice and is therefore stepped; where a smoother track is wanted it is simplified by the Douglas–Peucker algorithm [18] within a tolerance the operator sets, which removes the staircase without moving the route off navigable water.

For A* [3] the heuristic is the great-circle distance from the current cell to the goal, multiplied by the smallest weight present anywhere on the map; it is given by formula (2.11):

$$h(v) = w_{min} \cdot d_{gc}(v, goal) \tag{2.11}$$

This heuristic is admissible. Any route from v to the goal has a length of at least d_{gc}(v, goal), because the great circle is the shortest path on a sphere, and every kilometre of it costs at least w_{min}, because no cell weight is smaller. Hence h(v) never exceeds the true remaining cost, and A* is therefore guaranteed to return the optimal path. The heuristic is also consistent, since the triangle inequality holds for great-circle distances, so no node has to be re-expanded. The guarantee was verified experimentally: on all seventeen test voyages A* and Dijkstra returned identical costs, with a maximum deviation of 0 %.

2.4.3 Choice of the search algorithms

Both algorithms required by the assignment are implemented, as two independent procedures rather than as one procedure with the heuristic switched off, so that their node expansion counts and run times can be compared on identical cost maps. Following the recommendation of the supervisor, two further methods were added: dynamic programming and a genetic algorithm; they are designed in subsections 2.4.6 and 2.4.7. The properties of the four methods are compared in table 2.9.

Table 2.9 – Properties of the implemented search methods

Property	**Dijkstra**	**A***	**Dynamic programming**	**Genetic algorithm**
What is minimised	g(v)	f(v) = g(v) + h(v)	V(v), Bellman equation	fitness F of a whole route
Optimality	guaranteed	guaranteed, h admissible	guaranteed	not guaranteed
Time	O(E log V)	O(E log V), fewer nodes	O(K · E), K sweeps	O(G · P · L)
Limit of the work (parameter)	maximum number of steps	maximum number of steps; heuristic weight	maximum number of sweeps	number of generations
Result	one route	one route	cost-to-go of every cell	one route
Use in the application	verification	default	verification, comparison	comparison

Here V is the number of cells, E the number of moves between them, K the number of sweeps of value iteration, G the number of generations, P the size of the population and L the length of a route in cells.

The great-circle route is implemented as a fifth, degenerate “algorithm”, so that it can be requested, stored, displayed and exported through exactly the same interfaces as the four real ones, which keeps the comparison in subsection 2.4.5 free of special cases.

2.4.4 Automatic refinement of the lattice

A raster planner cannot see a channel narrower than one cell. At a resolution of 0.25° a cell is about sixteen kilometres wide at the latitude of Denmark, while the navigable channel of the Great Belt is narrower than that; the planner therefore reports, correctly with respect to its own model but wrongly with respect to the world, that no route exists from the North Sea into the Baltic.

This failure mode was discovered during testing and is addressed by automatic refinement. When the search finds no path, the planner does not report failure: it multiplies the lattice step by 0.55, widens the margin around the two ports by two degrees, rebuilds the cost map and searches again, up to three times, down to a floor of 0.04°. The widening lets the planner

find a passage lying outside the first rectangle, such as the Skagerrak on a voyage from the North Sea into the Baltic. Only if the finest lattice still admits no path is the request reported as infeasible, and the message then names the resolution that was reached, so that the operator can distinguish a genuinely landlocked request from a resolution limit. The refinement is reported to the client as a distinct progress stage, so that the additional waiting time is explained rather than merely experienced.

Since version 1.3 the planner first determines where the obstacle lies: when the water around both ports reaches the edge of the lattice, the passage may lie outside the rectangle, and the area is widened while the cell size is kept; otherwise the lattice is refined. This and the other measures for complex courses are described in subsection 2.4.9.

Refinement removes the failure in which no route is found. The opposite failure — a route that is found but crosses land which the classifier has taken for water — is caught by a second safeguard added after the test of subsection 3.5.6: every route is checked against an independent coastline, and a route that crosses land is searched again with the offending cells blocked or on a finer lattice. It is described in subsection 3.5.7.

2.4.5 Comparison with the shortest-distance route

The comparison required by the assignment is performed on the same cost map as the optimised route, which is what makes it meaningful: both routes are judged by the same classifier, at the same resolution, at the same moment.

The great circle between the endpoints is sampled at 160 points; each sample is located in the lattice and its zone is read off. The comparison then reports the length of both routes, the additional distance accepted by the optimised route, how many samples of the great circle fall on land, and a verdict. Samples are counted both in total and over the interior of the route — excluding the two berths — so that the figure is directly comparable with the corresponding count for the optimised route, whose endpoints are also berths.

A route that crosses land has no meaningful finite cost. Where that is the case the comparison reports the cost as undefined rather than as a large number derived from the impassability constant, and the interface and the PDF report show “not navigable” instead. This was a deliberate correction made during implementation: the first version reported a cost of some hundreds of millions, which is arithmetically consistent but conveys nothing to the operator.

2.4.6 Dynamic programming: the objective function and the Bellman equation

Every search method in the application rests on an evaluation function of the discrete step — the cost (2.10) of one move between neighbouring cells. How the methods turn this step

cost into a decision is determined by a second function, built on top of it; designing that function is the main decision in each algorithm. In dynamic programming and the theory of optimal control this function is the Bellman function, or value function [39, 40]; in the genetic algorithm it is the fitness function.

Both methods solve the same optimisation problem. A route is a sequence of cells $P = (v_0, v_1, \ldots, v_n)$, in which v_0 is the cell of the departure, v_n the cell of the destination and every v_{t+1} one of the eight neighbours of v_t. The cost of a route is the sum of the step costs (2.10) along it, and the objective function of the planning problem is formula (2.12):

$$J(P) = \Sigma_{t=0}^{n-1} c(v_t, v_{t+1}) \rightarrow \min, \quad v_0 = \text{start}, \; v_n = \text{goal}, \; v_{t+1} \in N(v_t) \qquad (2.12)$$

where $c(u, v) = dgc(u, v) \cdot ½ \cdot [w(z_u) + w(z_v)]$ is the cost of one step, dgc(u, v) the great-circle length of the step in kilometres and w(z) the weight of the zone z of a cell: 1 for open sea, 1.8 for coastal sea, multiplied by 1.6 in the belt along the shore (subsection 2.4.1), and infinite for the near-coast zone and for land, so that a route through them has an infinite cost and is not admissible. J is measured in weighted kilometres; for a route that lies entirely in open sea it equals the length of the route. Its minimum $J^* = \min J(P)$ over all admissible routes is the optimal cost that A*, Dijkstra and dynamic programming find, and the reference against which the genetic algorithm is judged.

In the dynamic-programming formulation the state of the vessel is a cell v of the lattice and a control is one of the eight moves to a neighbouring cell u. The value V(v) is the smallest total cost with which the destination can be reached from v. By Bellman's principle of optimality, every tail of an optimal route is itself optimal, so V satisfies the Bellman equation (2.13):

$$V(goal) = 0, \quad V(v) = \min_{u \in N(v)} [\, c(v, u) + V(u) \,] \qquad (2.13)$$

where N(v) is the set of navigable neighbours of v and c(v, u) is the step cost (2.10). No move from or into a land cell is admitted — its step cost is infinite — so the value of a land cell stays infinite.

The equation is solved by value iteration. V is set to zero in the goal and to infinity everywhere else; each sweep replaces the value of every cell by the right-hand side of (2.13). After k sweeps V(v) equals the cheapest cost over routes of at most k moves, so the iteration stops as soon as a sweep changes nothing, and the values are then exact. A sweep updates all cells at once with array operations: for each of the eight directions the lattice of values is shifted by one cell and added to the lattice of step costs.

In formulas, value iteration computes the sequence of functions (2.14):

$$V_{k+1}(v) = \min \{ V_k(v),\ \min_u \in N(v) [c(v, u) + V_k(u)] \} \quad (2.14)$$

starting from $V_0(\text{goal}) = 0$ and $V_0(v) = \infty$ for every other cell, and stops at the first sweep k for which $V_{k+1} = V_k$. Since an optimal route visits no cell twice, at most n − 1 sweeps are needed on a lattice of n navigable cells; in practice the number of sweeps equals the number of moves of the longest optimal route to the destination. The fixed point satisfies the Bellman equation, and V(start) is the value of the objective function (2.12) on the optimal route: V(start) = J*. The route is then read off the value function by the greedy policy (2.15):

$$v_{t+1} = arg\ min_{u \in N(v_t)} [\ c(v_t, u) + V(u)\ ], \quad v_0 = start \quad (2.15)$$

Starting from the departure, the vessel always moves to the neighbour through which the remaining cost is smallest; with an exact V every move lowers the value, and the walk ends in the destination. The cost of the route equals V(start) and is the same optimum that A* and Dijkstra find. Unlike them, dynamic programming computes the cost-to-go of every cell, not only of the cells near the optimal route; that is more work for one query, but once V is known the optimal route to the same destination from any other departure is obtained without a new search.

2.4.7 The genetic algorithm: the fitness function

A genetic algorithm [30, 31] imitates natural selection. It keeps a population of candidate solutions, each encoded as a chromosome; evaluates every candidate by a fitness function; and builds the next generation from the fitter candidates by crossover, which combines parts of two parents, and mutation, which changes a child at random. The method is used for path planning on grids with obstacles [35] and for ship weather routing [36].

A chromosome here is a list of up to fourteen intermediate waypoints, each a cell of the lattice. The route it encodes is the poly-line from the departure through the waypoints to the destination; each straight segment is rasterised into a chain of neighbouring cells, so the decoded route is a path in the same graph that the other methods search. The waypoints are kept ordered along the line from the departure to the destination, which rules out loops. Written as route(χ), the decoded lattice path is evaluated with the objective function (2.12), in which land, instead of being forbidden, carries a large finite weight (2.16):

$$\tilde{w}(z) = w(z) \text{ for water}; \quad \tilde{w}(z) = W = 200 \text{ for the near-coast zone and land} \quad (2.16)$$

The fitness of a chromosome χ is then the cost of its route with these weights, formula (2.17):

$$F(\chi) = \Sigma_{(u,\,v)\,\in\,route(\chi)}\; d_{gc}(u, v) \cdot ½ \cdot [\; \tilde{w}(z_u) + \tilde{w}(z_v)\;] \qquad (2.17)$$

where the sum runs over the steps (u, v) of route(χ). The fitness is minimised: the smaller F, the fitter the chromosome. The finite land weight is the main design decision of the algorithm. With an infinite weight every route that touches land would have the same fitness, and the population would receive no signal about which of two infeasible routes is closer to being feasible. With a finite W a route that crosses three land cells is worse than one that crosses one, so evolution moves towards navigable routes, while even a single land cell costs more than any navigable detour on the voyages considered. Because F uses the same step cost as the exact methods, the result can be compared directly with the optimum; the relative difference is the optimality gap of the algorithm.

For a navigable chromosome the fitness coincides with the objective function of its route, F(χ) = J(route(χ)) ≥ J*, so the quality of the genetic algorithm is measured by the optimality gap (2.18):

$$\Delta = (\; F(\hat{\chi}) - J^{*}\;) \,/\, J^{*} \cdot 100\ \% \qquad (2.18)$$

where $\hat{\chi}$ is the best chromosome of the last generation and J* the optimal cost found by A* or dynamic programming. The fitness enters the algorithm in three places: tournament selection takes, of three random chromosomes, the one with the smallest F; the two chromosomes with the smallest F pass unchanged to the next generation, so that the best fitness never increases from one generation to the next; and the search stops when the best F has not decreased for 60 generations, or after 300 generations.

The operators and parameters are listed in table 2.10. The segment repair mutation is specific to route planning: it finds the first stretch of land that a segment crosses and inserts a waypoint at the navigable cell nearest to its middle, so that the route bends around a headland step by step.

Table 2.10 – Operators and parameters of the genetic algorithm

Element	**Choice**
Initial population	80 chromosomes: the direct line and random lists of 1–6 waypoints from an ellipse around the two ports
Selection	tournament of 3: the fittest of three random individuals
Crossover	one-point, along the departure–destination axis, probability 0.9
Mutation	probability 0.35; one of five moves: segment repair, shift, insertion, deletion of a waypoint, moving a waypoint off land
Elitism	the 2 best individuals pass unchanged to the next generation
Termination	300 generations, or 60 generations without improvement
Land weight W	200

Element	Choice
Random seed	42

Example. On the voyage Kiel → Tallinn (lattice of 0.1375°, 15,936 cells) value iteration (2.14) converges after 181 sweeps and gives V(start) = J* = 1,817.34 weighted km for a route 1,249.3 km long; A* and Dijkstra return the same cost. The best chromosome of the genetic algorithm after 290 generations has nine waypoints and $F(\hat{\chi}) = 1{,}898.92$, so by formula (2.18) the optimality gap is $\Delta = 4.49$ % (table 3.6).

The two evaluation functions are compared in table 2.11.

Table 2.11 – Bellman function and fitness function

Aspect	Bellman function V(v)	Fitness function F(χ)
Objective	V(start) = J* = min J(P), formula (2.12)	F(χ) → min; Δ = (F − J*)/J*, formula (2.18)
Argument	a state: one cell	a whole route
Built from	step cost c(v, u), recursively	step cost with a finite land weight, summed
Treatment of land	excluded: infinite value	penalised: finite weight W
How it is used	greedy policy (2.15)	tournament selection, elitism and the stopping rule
Result	exact optimum	approximate, depends on the seed
Extension to weather or fuel	only if the cost stays additive per step	any function of the whole route

Dynamic programming is exact but requires the cost to be a sum of independent step costs. The genetic algorithm gives no guarantee, but its fitness can be any function of the whole route — the number of course changes, the fuel for a speed profile, the weather along the track — without any change to the search. The experimental comparison is given in subsections 3.3.5 and 3.3.6.

2.4.8 Parameters of the search methods and control of a running search

In the first version of the application the parameters of the lattice and of the search methods were constants of the configuration module: changing, for example, the number of generations of the genetic algorithm meant editing the code, and a long search could not be interrupted. Both were pointed out by the supervisor in the review of that version. In version 1.3 every request carries a set of search parameters, designed as one table in the module routing/parameters.py (table 2.12). The interface builds its parameter panels from that table; the server validates the values against it — an unknown name is rejected, a value outside its range is clamped — and the values actually used are stored with the route and written to its result file, so

that every result can be reproduced. The defaults equal the constants of the first version, so a request without parameters gives exactly the same route as before.

Table 2.12 – Parameters of the search methods

Method	Parameter	Default	Range	Effect
all	lattice cell budget	400,000	10^4 – $4 \cdot 10^6$	largest lattice that is classified
all	margin around the points	2°	0 – 15°	search area around the rectangle of the points
all	diagonal moves	on	on / off	8 or 4 neighbours of a cell
all	near-shore band; penalty	1 cell; 1.6	0 – 3; 1 – 5	extra cost of water next to land
all	refinement; maximum refinements	on; 3	on / off; 0 – 6	finer lattice when no route exists
all	coastline check	on	on / off	check and correction of the route (3.5.7)
all	land mask in the correction; from the first lattice	on; off	on / off	fusion of the land mask into the cost map
all	land share that blocks a cell	0.75	0.1 – 1	threshold θ of formula (2.20)
all	adaptive search area	on	on / off	widen instead of refine (2.4.9)
all	two-level search; corridor half-width	on; 50 km	on / off; 5 – 500 km	fine lattice in a corridor
all	time limit	none	0 – 3,600 s	the search stops after this time
A*	maximum number of steps	10^6	0 – $5 \cdot 10^7$	cells taken from the queue; 0 = no limit
A*	heuristic weight ε	1	1 – 5	weighted A*, formula (2.19)
Dijkstra	maximum number of steps	10^6	0 – $5 \cdot 10^7$	as for A*
DP	maximum number of sweeps	until convergence	0 – 10^6	sweeps of the Bellman equation
GA	population P; generations G	80; 300	10 – 2,000; 1 – 50,000	size and length of the evolution
GA	early stop	60	0 – 50,000	generations without improvement; 0 = never
GA	tournament; crossover; mutation; elite	3; 0.9; 0.35; 2	2 – 20; 0 – 1; 0 – 1; 0 – 50	operators of table 2.10
GA	maximum waypoints; land weight W; seed	14; 200; 42	2 – 80; 1 – 10^5; any	chromosome and fitness

The step limit is the parameter that the supervisor named as an example. One step of A* or Dijkstra is one cell taken from the priority queue and expanded; one step of dynamic programming is one sweep of the Bellman equation; the number of generations plays the same role for the genetic algorithm. When the limit is reached before the destination, the search ends with the outcome step limit, not no path (table 2.13). The two must be distinguished: a raised limit can turn the first into a route, whereas the second is a property of the lattice. The planner therefore does not refine the lattice after a step limit — a finer lattice would need even more steps — and the interface tells the user which parameter to raise. Dijkstra expands every cell closer to the start than the destination, so a long or winding course needs far more steps than A*; the

experiment of subsection 3.5.10 shows that a limit of 1,000 steps is not enough for a voyage round Denmark, while 10,000 is.

A dynamic-programming run stopped by its sweep limit still yields a valid route when the departure already has a finite value. After k sweeps V(v) is the cheapest cost over routes of at most k moves, so for the neighbour u chosen by the greedy policy (2.15) $c(v, u) + V(u) \le V(v)$ holds; with positive step costs the value strictly decreases along the walk, which therefore reaches the destination. The route is optimal only if the iteration had converged, and the message says so.

The heuristic weight ε turns A* into weighted A*, which orders the queue by formula (2.19):

$$f(v) = g(v) + \varepsilon \cdot h(v), \quad C \le \varepsilon \cdot C^*, \quad \varepsilon \ge 1 \tag{2.19}$$

where C is the cost of the route found and C* the optimum. With a consistent heuristic the bound holds for every $\varepsilon \ge 1$, and the larger ε, the more greedily the search runs towards the destination and the fewer cells it expands. On a very large lattice this trades a guaranteed, bounded loss of quality for time; $\varepsilon = 1$ is the exact A* of subsection 2.4.2.

A running search can be stopped by the user, and it stops by itself when the time limit set in the interface passes. The mechanism is a run-control object shared by the web server and the worker thread that performs the search: a thread-safe flag, which the server sets when the client sends a cancel message, and the start time of the run. Every long loop of the planner tests it at a fixed interval — every 2,000 expansions of A* and Dijkstra, every sweep of value iteration, every generation of the genetic algorithm, every batch of the classification of the lattice and every round of the coastline correction. When the flag is set or the time is over, an exception unwinds all nested loops at once; the planner catches it and returns an ordinary result with the outcome cancelled or time limit. A stopped search is therefore reported, stored and saved like any other. The possible outcomes are listed in table 2.13.

Table 2.13 – Outcomes of a search

Outcome	Meaning	Suggested action
found	a route was found	—
no path	the search space was exhausted without reaching the destination	via points, finer lattice, larger margin
step limit	the maximum number of steps, sweeps or generations was reached first	raise the limit of the method
time limit	the time limit passed	raise the limit or use a coarser lattice
cancelled	the user pressed Stop	—

Every search, whatever its outcome, and every comparison of the methods are written to a result file for further processing. JSON was chosen rather than CSV, as the supervisor also suggested: a CSV file holds a single table, whereas the result consists of parts of different shape

— the request, the parameters, the outcome and its timing, the lattices tried, the coastline check and the list of points — and JSON keeps them in one self-describing document that the standard library of every common language reads without a special parser. The name of the file contains the coordinates of the points, the method, the outcome, the search time and the time of the run, for example route_N54.3233E10.1228_N59.4370E24.7536_dijkstra_step_limit_5.44s_20261014-134803.json, so that the files of one voyage sort together and can be told apart without being opened. The files are written to the folder outputs/results of the application, and the absolute path of the file is shown on the screen after every search. The structure of the file is given in table 2.14.

Table 2.14 – Structure of the result file

Key	Content
format, application, route_id	identification of the file and of the program version
request	departure, destination, intermediate points, method, resolution, speed
parameters	all parameters of table 2.12 used by the method
status, status_text, found, message, warning	outcome (table 2.13) and the messages shown to the user
timing	search time in seconds, start and end time (UTC)
search	steps made, their meaning for the method, the step limit, the lattice, the lattices tried
route	length, great-circle distance, detour ratio, cost, passage time, zone profile, coastline check
endpoints, n_points, points	the given points and every point of the route: latitude, longitude, zone, weight, cumulative distance, course

2.4.9 Construction of complex courses

The supervisor also asked that the application construct routes for more complex arrangements of the points. Testing the first version on such courses (subsection 3.5.8) showed four ways in which a complex course fails, and version 1.3 answers each of them with a separate measure.

1) The passage lies outside the search area. The lattice covers the rectangle of the two ports with a margin. A voyage from the Red Sea to the Persian Gulf must leave that rectangle twice — through Bab-el-Mandeb in the south and round the Arabian Peninsula in the east. The first version refined the lattice on every failure and widened the margin by only two degrees, so it never reached the passage and reported no route after four ever larger lattices. Version 1.3 first determines where the obstacle lies. The navigable cells are divided into connected components by one labelling pass, O(V). The search failed, so the two ports lie in different components; if both components reach the edge of the lattice, the water bodies continue beyond it and the connecting passage may lie outside, so the margin is widened by half the

larger side of the rectangle and the cell size is kept. If one of them is enclosed inside the rectangle — a gulf whose mouth is closed on this lattice — a wider area cannot help, and the lattice is refined.

2) The classifier takes land for water. The correction of subsection 3.5.7 blocks the land under the route round by round. On a complex course the first route may cross hundreds of kilometres of land — 234 km on the voyage Kapellskär – Oulu — and blocking it piece by piece does not converge. Version 1.3 therefore begins the correction by fusing the independent land mask into the existing cost map, by formula (2.20):

$$w'(v) = \infty, \text{ if } z(v) \text{ is navigable and } s(v) \geq \theta; \quad w'(v) = w(v) \text{ otherwise} \qquad (2.20)$$

where $s(v)$ is the share of the 3×3 sub-samples of cell v that lie on land of the mask and θ the land share that blocks a cell (0.75 by default). The zones of the classifier are kept and the network is not asked again, so the fusion costs milliseconds. Cells within 10 km of a port — at least one and a half cells — are exempt, so that a port in a fjord or a river mouth narrower than a cell is not cut off from the sea. The rounds of subsection 3.5.7 then continue on the fused map. The mask is not used from the first lattice by default, because on a coarse lattice it closes narrow straits and the lattice must be refined more often; the parameter Land mask from the first lattice switches it on.

3) The passage is narrower than the cells. Refining the whole rectangle multiplies the number of cells by $(1 / 0.55)^2 \approx 3.3$ per step, and the classification of the cells dominates the time of a request. Version 1.3 uses a two-level search instead: the route found on the coarse lattice defines a corridor; the fine lattice, 2.5 times finer, covers the same rectangle, but only the cells within the corridor half-width W of the coarse route (50 km by default) are classified, and the others are treated as land. The number of classified cells is estimated by formula (2.21):

$$N_1 \approx 2W \cdot L / (\Delta\varphi \cdot \Delta\lambda) \quad \text{instead of} \quad N_0 = A / (\Delta\varphi \cdot \Delta\lambda) \qquad (2.21)$$

where L is the length of the coarse route, A the area of the rectangle and $\Delta\varphi$, $\Delta\lambda$ the sides of a cell in kilometres; on a long voyage N_1 is a few per cent of N_0. If the corridor gives no route without land, the whole area is refined as in the first version, and the better of the two results is kept.

4) The user knows the passage. A navigator often knows on which side of an island or through which strait the course must go. The route can therefore be forced through intermediate points set on the chart. It is planned leg by leg with the same method and parameters; every leg gets its own lattice, and the legs are joined into one route.

The time limit and the Stop button act on the whole route. This also answers the question how to set one's own route: any departure, destination and intermediate points can be set with the mouse or typed in (appendix A).

The cost of the measures is small where they are not needed: a route that passes the coastline check on the first lattice is not affected by any of them, so the routes of the test on 100 random voyages that were correct before are unchanged. Their effect on complex courses is measured in subsection 3.5.8.

2.4.10 Conclusions on the chapter

The architecture of the application has been designed as a five-layer monolith with a single-page client and a WebSocket channel for live progress. A relational schema of six relations and one view has been designed and justified, and SQLite has been selected as the storage engine. The classification model has been designed around a twenty-eight-component feature vector derived from a label-free reference index, with three explicit measures against target leakage; the forward pass and the training of the network have been described mathematically, and a rule for removing mislabelled training points has been designed. The cost model, four search methods — A*, Dijkstra, dynamic programming and a genetic algorithm — and the automatic refinement of the lattice have been designed, with the Bellman function and the fitness function as the evaluation functions of the last two, and the admissibility of the A* heuristic has been proved.

In response to the review of the first version, the parameters of every method — the step limits, the heuristic weight, the parameters of the genetic algorithm, the lattice and a time limit — have been made settable in the application; a running search can be stopped, and the result of every search is saved to a JSON file. Four measures for complex courses have been designed: an adaptive search area, the fusion of the land mask into the cost map, a two-level search in a corridor, and intermediate points.

3 IMPLEMENTATION AND TESTING OF THE APPLICATION

The application is implemented in Python 3.11. The neural network uses PyTorch [14]; scikit-learn [15] supplies the standardiser, the ball tree and the evaluation metrics; FastAPI and uvicorn serve the HTTP and WebSocket interfaces; ReportLab produces the PDF report; the client is written in plain JavaScript with the Leaflet mapping library. The application comprises about 9,900 lines of Python, including the documentation comments that accompany every module, class and function, about 2,000 lines of client code and a SQL schema of 209 lines. The source code is published in a public Git repository together with installation instructions.

3.1 Implementation of the AIS loading and pre-processing module

3.1.1 Reading the source formats

The loader accepts GeoJSON, CSV and JSON. The corpus supplied for this project is a GeoJSON file of 119 MB, too large to parse comfortably as a single document, so the loader uses a streaming parser that yields one feature at a time; the peak memory is proportional to the number of records retained rather than to the size of the file. Where a feature carries coordinates both in its geometry and in its properties, the geometry is authoritative. CSV input is tolerant of the usual column spellings: lat, latitude, y and their longitude counterparts are all recognised.

3.1.2 Validation and cleaning

Every record passes a validation stage that drops rows with missing or out-of-range coordinates and reports what was dropped and why; on the supplied corpus no record was dropped. The 156,838 valid points then pass the consistency check of subsection 2.3.7, implemented in the module cleaning.py, which removes 5,427 mislabelled points and leaves 151,411. The same module assigns every point to its sea region; the region of every test point is stored with the prepared dataset so that the accuracy can later be reported region by region.

3.1.3 Feature extraction

The extractor is vectorised over the whole frame: the ball tree is queried once for all points, and the neighbourhood statistics are computed as array operations over the resulting

matrix. Preparing the dataset — loading, cleaning, splitting, building the reference index and extracting the twenty-eight features of every point — takes about sixteen seconds.

3.2 Implementation of the neural network for zone classification

3.2.1 The network

The network of subsections 2.3.4–2.3.5 is implemented as two PyTorch modules: a residual block, which applies formulas (2.1)–(2.4), and the classifier, which stacks three blocks and a linear output layer. Softmax (2.5) is applied outside the network, inside the loss function during training and explicitly at inference.

3.2.2 Training

Training ran for 97 epochs in 4.3 minutes on four CPU cores. The lowest validation loss (0.175327) was reached at epoch 82; fifteen epochs without improvement followed, early stopping ended the run, and the weights of the best epoch were restored. The dynamics of the run are shown in figure 3.1.

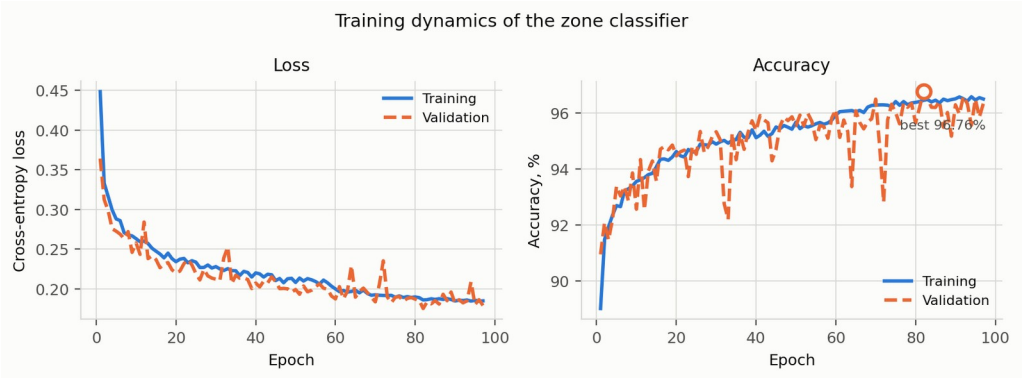


Figure 3.1 – Training dynamics: loss and accuracy on the training and validation splits

The curves show no over-fitting: the validation loss follows the training loss downwards and stays below it in 78 of the 97 epochs — an expected consequence of dropout and batch normalisation, which are active during training and switched off during evaluation. The best validation accuracy was 96.76 %.

3.2.3 Results of the classification

Table 3.1 reports the metrics on the held-out test split, which took no part in training, in validation or in the construction of the reference index.

Table 3.1 – Classification quality on the test set

Metric	**Value**
Number of test points	22,712
Accuracy	96.95 %
Macro F1	0.9695
Weighted F1	0.9695
Cohen’s kappa	0.9582
Macro ROC-AUC (one vs rest)	0.9985
Mean confidence of the predicted class	0.9504

Cohen’s kappa [19] of 0.9582 corrects for the agreement that would arise by chance on an imbalanced corpus; a value above 0.8 is usually read as almost perfect agreement. The macro ROC-AUC [20] of 0.9985 shows that the ranking produced by the network is nearly perfect even where the arg-max decision is wrong. The quality on each class is reported in table 3.2.

Table 3.2 – Per-class quality on the test set

Class	**Precision**	**Recall**	**F1**	**Support**
OPEN_SEA	0.9835	0.9639	0.9736	8,348
COASTAL_SEA	0.9433	0.9721	0.9575	5,548
NEAR_COAST	0.9703	0.9709	0.9706	4,537
COASTLINE	0.9768	0.9755	0.9761	4,279

3.2.4 Analysis of the errors

The distribution of the errors over the classes is shown in figure 3.2.

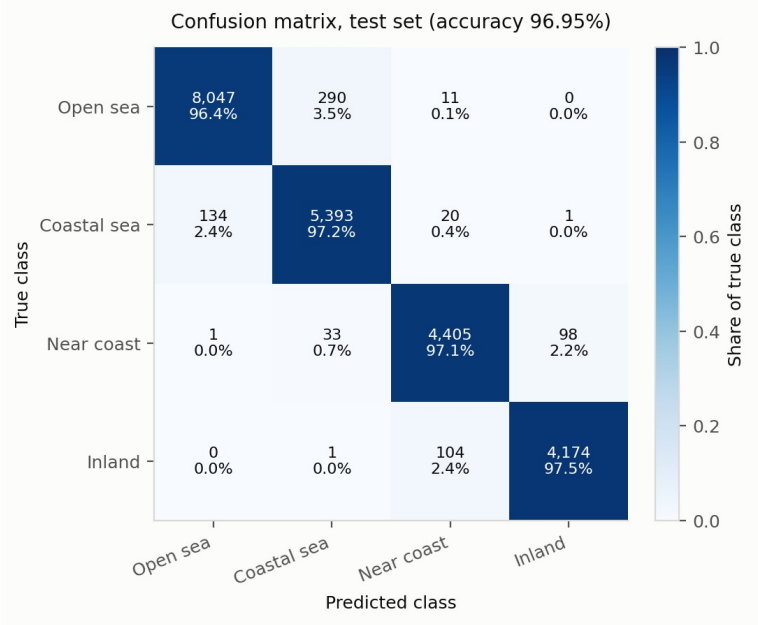


Figure 3.2 – Confusion matrix on the test set

The dominant error is the confusion of OPEN_SEA with COASTAL_SEA: 290 open-sea points were labelled coastal and 134 coastal points were labelled open sea. It is also the least consequential error, because both classes are navigable: it changes the weight of a cell from 1.0 to 1.8 or back, which may bend a route but cannot make it unsafe. The second group lies on the shoreline: 98 near-coast points were labelled inland and 104 inland points near-coast. Both classes are impassable, so this confusion has no effect on the routes.

The errors that matter are those that cross the water–land boundary. There are 32 water points classified as land and 35 land points classified as water, together 0.29 % of the test set. The dangerous direction, land classified as water, occurs in 0.15 % of cases. The safety margin of subsection 2.4.1 mitigates it: an isolated misclassified land cell is surrounded by correctly classified land, and the penalised belt around that land keeps the optimiser away from the hole.

3.2.5 Effect of the data cleaning

The effect of the cleaning of subsection 2.3.7 was measured by comparing the model trained on the uncleaned corpus with the model trained on the cleaned one (table 3.3). The first model was evaluated on all points of its own test split, and separately on the points that the cleaning rule keeps and on those it would remove.

Table 3.3 – Model trained before and after cleaning

Model and test points	Points	Accuracy	Water ↔ land errors
Uncleaned model, whole test split	23,526	96.74 %	189
of these: consistent labels	22,715	97.11 %	79
of these: contradictory labels	811	86.31 %	110
Cleaned model, cleaned test split	22,712	96.95 %	67

The 811 contradictory points, 3.4 % of the old test split, account for 110 of its 189 water–land errors: on these 110 points the model sided with the physical measurement against the wrong label, so the metric was penalising the model for being right. After cleaning, the overall accuracy barely changes, but the number of water–land errors falls to 67, 0.29 % of the test points; these are the errors that decide whether a route is safe.

3.2.6 Accuracy by sea region

A single overall accuracy can hide a region where the model is weak. The test split was therefore also evaluated region by region (table 3.4).

Table 3.4 – Accuracy of the classifier in the eleven sea regions

Region	Test points	Accuracy	Water ↔ land errors
North Europe (Baltic and North Sea)	4,533	97.95 %	5
South China Sea	2,857	93.45 %	5
Japan and North-West Pacific	2,747	99.20 %	1
Southern Africa (Cape of Good Hope)	2,353	94.14 %	3
Black Sea and Eastern Mediterranean	2,072	96.19 %	20
Persian Gulf and Arabian Sea	2,023	97.73 %	4
Red Sea and Gulf of Aden	1,663	99.46 %	4
Central America (Caribbean coast)	1,433	97.35 %	4
Strait of Malacca and Andaman Sea	1,315	98.02 %	3
Aegean Sea	868	97.93 %	7
Bay of Bengal	689	94.63 %	11
Other (inland samples)	159	100.00 %	0
All regions	22,712	96.95 %	67

The accuracy ranges from 93.5 % in the South China Sea, where the two water classes are hardest to tell apart, to 99.5 % in the Red Sea. The largest share of the water–land errors, 20 of 67, falls into the Black Sea and Eastern Mediterranean region, whose steep coasts leave only a narrow band of shallow water. In the Aegean Sea, the region added in the second release of the corpus, the model reaches 97.9 % on 868 test points with 7 water–land errors, although the many islands and narrow channels make it one of the most intricate coastlines of the corpus. Its share of water–land errors, 0.81 %, is nevertheless about 2.7 times the average of the test split and the third highest of all regions, after the Bay of Bengal and the Black Sea and Eastern Mediterranean.

3.3 Implementation of the path search algorithms

3.3.1 The cost map

The cost map is built by classifying the flattened lattice in batches, reshaping the predictions into the lattice, mapping zones to weights and dilating the land mask to produce the safety belt. The batch size is chosen so that a large lattice is classified in at least twenty batches of at least 1 024 cells, which gives the client a steady stream of progress frames. For the Kiel–Tallinn voyage at 0.25° the first lattice has 37 × 76 = 2,812 cells and is built in 0.44 s, almost all of it classification.

3.3.2 A* and Dijkstra

Both algorithms use a binary heap as the priority queue and a closed set to avoid re-expansion. Step lengths are pre-computed once per lattice row and per direction, since the great-circle length of a one-cell step depends on the latitude but not on the longitude; the inner loop of the search therefore contains no trigonometry.

3.3.3 An optimisation of the heuristic

The first implementation evaluated the heuristic by calling the vectorised haversine function with scalar arguments. A* then expanded about 2.5 times fewer nodes than Dijkstra and yet ran about 1.6 times slower: the heuristic is evaluated for every generated node, and the overhead of a NumPy call on scalars exceeded the cost of the expansions it saved. The heuristic was rewritten with the math module and with the per-row trigonometry pre-computed; over four

hundred random cells the two versions differ by at most $9.4 \cdot 10^{-13}$ km. After the change A* became 1.233× faster than Dijkstra on average, as theory predicts.

3.3.4 Results of the routing benchmark

Six voyages in European waters were used as the main benchmark. For each, the cost map is built once and shared by all methods, so that the run times measure the search alone. The results of the exact search are collected in table 3.5.

Table 3.5 – Routing benchmark over six voyages

Voyage	A*, km	Great circle, km	Extra	A* nodes	Dijkstra nodes	A*, s
Kiel–Tallinn	1249	1052	+18.8 %	1,856	4,171	0.012
Helsinki–Copenhagen	1152	883	+30.5 %	991	1,123	0.007
Gdansk–Stockholm	636	555	+14.6 %	210	356	0.003
Rotterdam–Lisbon	2077	1794	+15.8 %	305	779	0.004
Bergen–Reykjavik	1746	1458	+19.8 %	587	920	0.006
Brest–Gibraltar	1423	1364	+4.3 %	61	354	0.003

A* expands on average 2.5 times fewer nodes than Dijkstra and runs 1.233 times faster; the advantage ranges from 1.13× on Helsinki–Copenhagen to 5.8× on Brest–Gibraltar, where the heuristic is most informative. Both algorithms return the identical optimal cost on every voyage (maximum deviation 0 %), which confirms the admissibility proved in subsection 2.4.2. All 6 great-circle routes cross land, while every route produced by the application is navigable, at a cost of 17.29 % additional distance on average. At the resolutions of table 3.5 the longest complete request, including the construction and refinement of the cost map, took 2.9 s.

3.3.5 Dynamic programming

Value iteration (subsection 2.4.6) is implemented without an explicit loop over cells. The step costs (2.10) of the eight directions are computed once as eight arrays of the size of the lattice; one sweep of the Bellman equation (2.13) is then eight array shifts, additions and element-wise minima (listing 3.1).

Listing 3.1 – One sweep of value iteration on the Bellman equation

```
value = np.full((n_rows, n_cols), np.inf)
value[goal] = 0.0
while True:
    best = value.copy()
    for (dr, dc), step in costs.items():      # eight directions
```

```
        np.minimum(best, step + shift(value, dr, dc), out=best)
    best[goal] = 0.0
    if not np.any(best < value - 1e-9):      # nothing improved: V is exact
        break
    value = best
```

The value function of the Odesa–Novorossiysk voyage is shown in figure 3.3. The iso-cost lines run around the Crimean peninsula, and the greedy policy (2.15) follows the steepest descent of V from the departure to the destination.

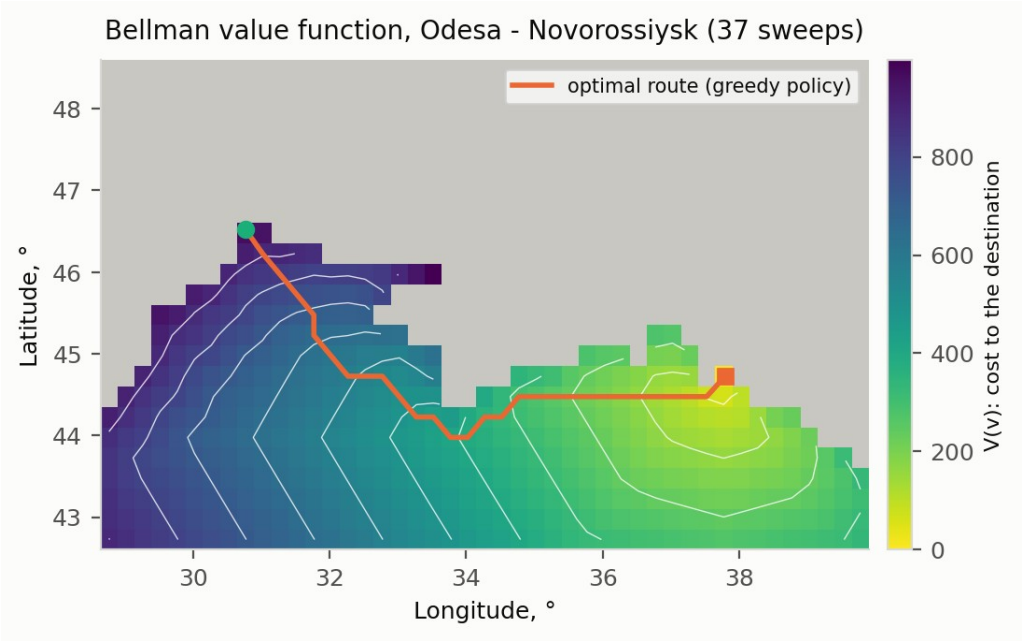


Figure 3.3 – Bellman value function V and the optimal route, Odesa → Novorossiysk

On all seventeen test voyages the cost of the dynamic-programming route equals the A* optimum (maximum deviation 0 %), as it must, since both methods are exact on the same graph. Dynamic programming performs far more elementary updates than A* expands nodes — it computes the value of every reachable cell, including those far from the route — but the updates are array operations, so its run time is of the same order as that of A* (table 3.6).

3.3.6 The genetic algorithm

The genetic algorithm of subsection 2.4.7 is implemented in genetic.py. Segment costs are cached, because the same segment occurs in many individuals and generations, and each segment is rasterised with array operations. The effort and the quality of the three search methods on the benchmark voyages are compared in table 3.6.

Table 3.6 – A*, dynamic programming and the genetic algorithm

Voyage	A*, s	DP, s	DP updates	GA, s	GA evaluations	GA above optimum
Kiel–Tallinn	0.012	0.041	1,178,491	0.41	23,281	4.5 %
Helsinki–Copenhagen	0.007	0.008	74,420	0.14	7,201	3.6 %
Gdansk–Stockholm	0.003	0.004	10,025	0.09	7,281	3.4 %
Rotterdam–Lisbon	0.004	0.006	48,345	0.11	6,721	3.7 %
Bergen–Reykjavik	0.006	0.010	57,240	0.25	11,521	7.9 %
Brest–Gibraltar	0.003	0.004	15,756	0.08	5,681	0.0 %

The genetic algorithm found a navigable route on every benchmark voyage, on average 3.8 % above the optimum, and on the eleven regional voyages of subsection 3.5.5 on average 1.07 % above it. It is 15–45 times slower than A*. Its convergence on the Kiel–Tallinn voyage is shown in figure 3.4 for three random seeds.

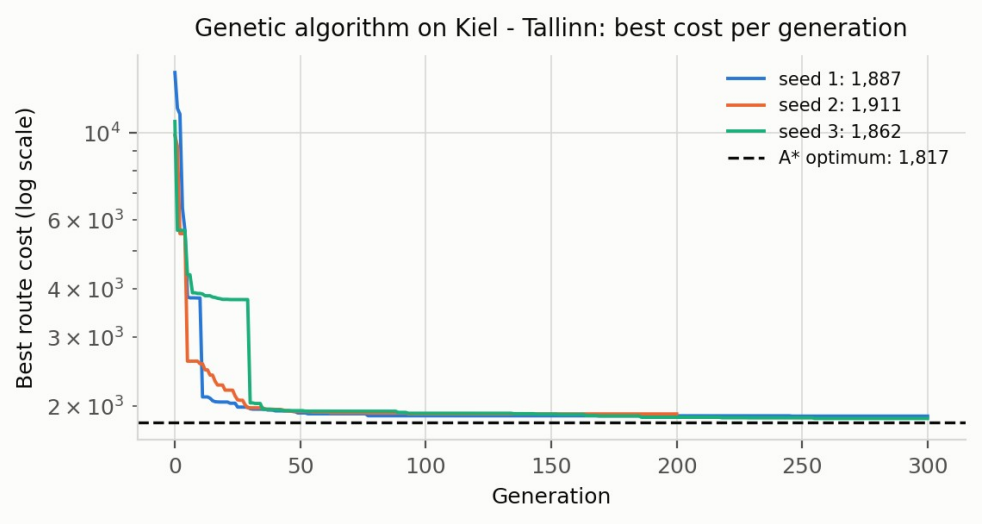


Figure 3.4 – Best route cost per generation of the genetic algorithm, Kiel → Tallinn

The figure shows the typical behaviour of the method. All three runs fall steeply during the first generations, while the population discards routes that pay the land penalty, and then improve slowly; they end 2–5 % above the optimum. The third run spends about thirty generations on a route twice as expensive as the optimum before a mutation of its waypoints opens a better passage: every small change of that route makes it worse, and the large change that is needed is unlikely under mutation. On 2 regional voyages such a trap was not escaped at all. Between Rotterdam and Gdańsk the algorithm converged on the Øresund, which the lattice closes by one cell, while the optimal route passes through the Great Belt; between Piraeus and Thessaloniki, on the 0.1° lattice of the Aegean Sea, no individual found a land-free passage among the islands within the generation limit. In a maze-like sea with narrow passages the fitness

function has local optima that the Bellman function does not have. For the static problem of this project dynamic programming and A* are therefore preferable; the genetic algorithm is retained because its fitness function can include criteria that are not additive per step, as discussed in subsection 2.4.7.

3.3.7 Step limits, the time limit and the Stop button

The run control of subsection 2.4.8 is implemented in routing/control.py as a class with three operations: cancel, called by the web server when a cancel message arrives; check, called by the search loops; and the elapsed time, shown by the timer of the interface. The flag is a threading.Event, so cancel may be called from any thread. A check costs one test of the flag and one reading of the clock, well under a microsecond, and is made every 2,000 expansions, so its overhead on A* and Dijkstra is below one per cent. The step limits are tested on every expansion, which costs one integer comparison.

The classification of the lattice is the longest stage of a request, and the check is made between its batches; when a run can be stopped, a batch is limited to 1,024 cells, about a tenth of a second of classification. The reaction was measured on long searches with a time limit of one second and with the Stop button pressed after one second: the search ended 0.01–0.22 s after the limit and 0.22 s after the button (subsection 3.5.10), within the target of one second (table 1.4).

3.4 Implementation of the visualisation module

3.4.1 The client

The client is a single HTML page with four scripts: a transport layer that wraps the REST calls and the WebSocket, a small plotting module for the metric charts, the texts of the interface in three languages (subsection 3.4.6), and the controller. Leaflet is stored in the repository rather than loaded from a content delivery network, so that the interface starts with no internet connection. The main window after a route has been built is shown in figure 3.5.

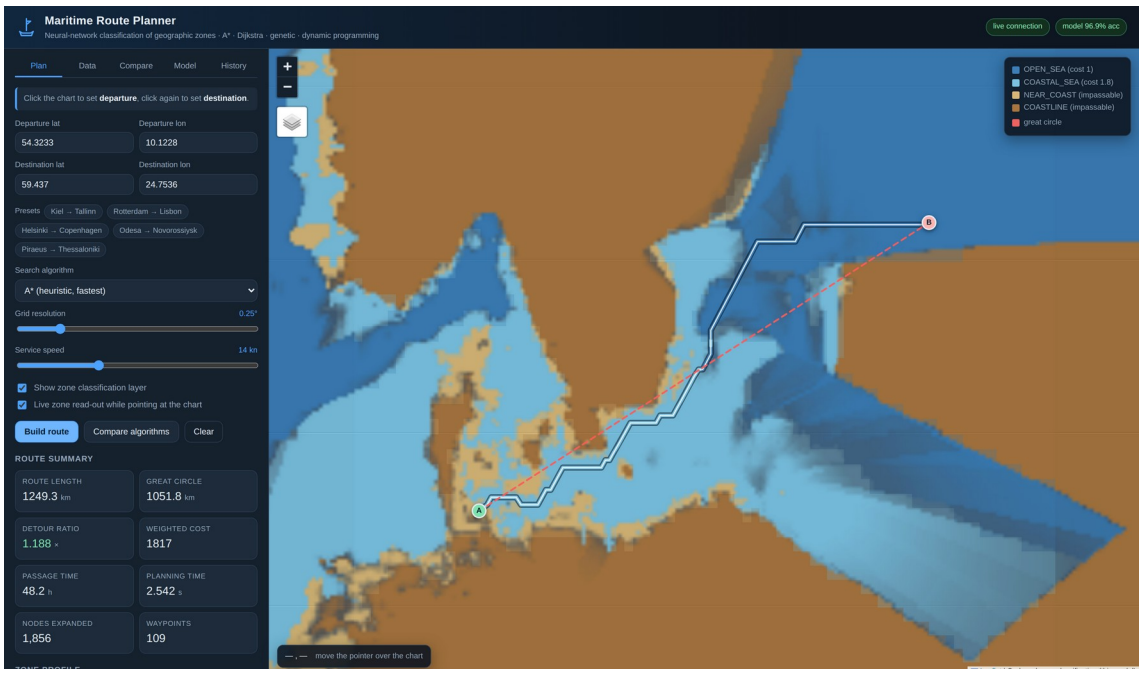


Figure 3.5 – The main window after a route has been built (Kiel → Tallinn)

The four zones are ordered around sea level, so they are encoded by a diverging scheme — water blue against land ochre — rather than by an arbitrary set of hues [27]. The route is drawn segment by segment and coloured by the zone of each segment; the great circle is drawn as a red dashed line. The classification layer is drawn underneath as translucent cells, down-sampled by the server by majority vote to at most six thousand cells.

3.4.2 Real-time interaction

Three mechanisms make the interface interactive in real time.

- Progress streaming. While a job runs, the panel names the current stage and shows a progress bar with the figures behind it — cells classified, nodes expanded by A*, sweeps of value iteration or generations of the genetic algorithm.
- Live zone read-out. As the pointer moves over the chart, its position is sent over the socket, and the zone, the confidence and the depth under the pointer are displayed at the bottom-left corner within a few milliseconds.
- Direct manipulation. The departure and the destination are set by clicking the chart and can be dragged; the coordinate fields and the markers are kept synchronised.

A progress frame captured during the classification of the lattice is shown in figure 3.6.

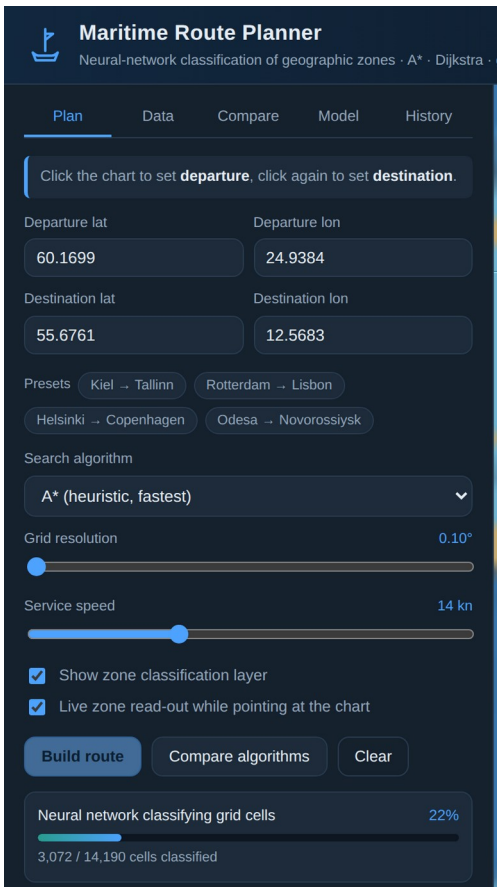


Figure 3.6 – Live progress during the classification of the lattice

3.4.3 The on-board chart

The client requests map tiles from the OpenStreetMap tile service when the machine is online. A vessel at sea frequently is not, so the server also renders its own tiles: each tile is a lattice of coordinates classified by the network and painted with the zone palette, with low-confidence cells blended towards grey. If four tile requests to the public service fail, the client switches to the on-board chart by itself. Every screenshot in this note was taken in this offline mode.

3.4.4 Export

The three export formats required by the assignment are listed in table 3.7.

Table 3.7 – Export formats

Format	Content	Intended use
GeoJSON [21]	Route line, every waypoint with its zone and cumulative distance, the great-circle reference line, styling properties	opens directly in QGIS, geojson.io and other GIS software
CSV	Waypoint table: sequence, coordinates, zone, weight, cumulative distance, bearing	further analysis in a spreadsheet
PDF	Voyage summary, zone profile, comparison with the shortest route, information on the model, the waypoint list	a document for the voyage file

Non-finite values are converted to null in GeoJSON, because JSON has no representation for infinity and a naive serialiser would emit a document that strict parsers reject.

3.4.5 Parameter panels, timer, Stop and result files

The parameter panels are not written in the page: the client requests the description of the parameters from the server (GET /api/parameters) and builds the fields from it — label, unit, range, default and an explanation shown when the pointer rests on the label. The panel shows the common group and the group of the method selected above it, marks every changed value, counts the changes in its title and restores the defaults on request; the changed values are sent with every request and remembered in the browser.

While a search runs, the progress block shows a timer, the time limit, if one is set, and a Stop button; the buttons that start a new search are disabled. After the search the outcome is shown with a hint which parameter to change, a table shows every lattice the planner tried, and a box shows the absolute path of the result file with a Copy path button and a download link. The latest result files and their folder are also listed on the Data tab. The intermediate points are added with Shift and a click or in an explicit mode, drawn as numbered markers that can be dragged or removed, and listed under the coordinates. The comparison of the methods runs over the same channel, with progress, timer and Stop, and draws the routes of all methods on the chart. These views are shown in figures A.3–A.6 of appendix A.

3.4.6 Interface languages

The interface is shown in English, Russian or Chinese; the language is switched with the buttons EN, RU and 中文 in the header. The first time the page opens, the language of the browser is taken if it is one of the three, otherwise English; the choice is then remembered in the browser. All texts of the interface are kept in one script, i18n.js, as three dictionaries of 307 keys each, with the labels and explanations of every algorithm parameter. The elements of the page carry the key of their text in an attribute, and the controller takes every text it generates —

banners, tables, progress, tooltips, the legend — from the dictionary of the current language, with placeholders for the numbers. A key missing in Russian or Chinese would be shown in English.

The server and the result files stay in English, so that a result file, a report and the log do not depend on the language of the person who made the search. The messages the server sends during and after a search (the stage, the outcome, the lattices tried) are translated in the client by a table of phrase patterns; a phrase not in the table is shown as received. The zone classes are shown by their codes in English, as in this note, and by their names in the other two languages. On a switch the controller redraws what is on the screen — the route summary, the comparison table, the parameter panels, the model and the lists — from the data it keeps, without a new request to the server. The automated tests check that every key used by the page has a text in all three languages, that every parameter has a Russian and a Chinese label, and the translation of the server phrases. The Russian and the Chinese interface are shown in figures A.7 and A.8 of appendix A.

3.5 Integration of the modules and testing

3.5.1 Integration

The modules are integrated through two façades. The classification service is a thread-safe singleton, so that the reference index — 105,848 samples, the training points that carry a depth or an elevation measurement — is held in memory once, and inference is serialised by a lock. The planner composes the classifier, the cost map and the four search methods and is the only module that knows the order of the stages. Persistence is confined to the repository, the only module that issues SQL; when an older database is opened, the repository upgrades its schema so that routes of the two new methods can be stored.

3.5.2 The test suite

In total 139 automated tests were written with pytest. They cover every module of the application and run in about 45 seconds on the computer of the experiments. The composition of the suite is given in table 3.8.

Table 3.8 – Composition of the test suite

Test module	Tests	What is verified
test_geodesy.py	7	great-circle distance against reference values, bearings, interpolation, densification, simplification
test_cleaning.py	7	removal of sea points above and land points below sea level, the tolerance, conflicts revealed by the elevation alone, missing values, the Aegean region
test_data_loading.py	12	column aliases, malformed input, validation, feature matrix, missing values, index round-trip and self-exclusion
test_routing.py	11	lattice, coarsening, agreement of A* and Dijkstra, obstacle avoidance, absence of a path, snapping, cheaper zones, overlay tiling
test_dynamic_programming.py	5	equality with the A* optimum, the Bellman equation in every cell, land excluded, unreachable goal
test_genetic.py	7	eight-connected rasterisation, gap in a wall, never below the optimum, reproducibility, monotone best fitness
test_storage_export.py	14	GeoJSON, CSV and PDF export, database round-trip, deletion, storage of the new methods, upgrade of an old database
test_model_and_api.py	23	probabilities, known locations, determinism, route safety, refinement, every HTTP endpoint, upload, tiles, the WebSocket planning session
test_coast_check.py	15	verdicts of the land mask, the unchecked port stretches, blocking of cells, automatic correction of a route over a missed island by A*, Dijkstra and dynamic programming, the warning of an uncorrectable route, the unchanged benchmark map, storage, upgrade of an old database, the PDF line
test_search_control.py	30	stop flag and time limit, step limits of A*, Dijkstra, dynamic programming and the genetic algorithm, weighted A*, validation of the parameters, outcome of the planner, land-mask fusion, corridor map, adaptive area, via points, comparison with step limits and Stop, result files, the WebSocket cancel and benchmark actions
test_i18n.py	8	the three dictionaries of the interface have the same keys, every key used by the page and the scripts is translated, every parameter has a Russian and a Chinese label, placeholders, language switch and translation of the server phrases

Several tests encode properties rather than particular numbers. The equality of the A*, Dijkstra and dynamic-programming costs is asserted directly, so a change that breaks the admissibility of the heuristic fails the suite; the test of dynamic programming checks the Bellman equation (2.13) in every cell; and the WebSocket test drives a complete planning session.

3.5.3 Experimental comparison with the shortest-distance route

Table 3.9 reports the comparison for the Kiel–Tallinn voyage, which is representative of a route through confined waters. The route was requested at the default resolution of 0.25° and planned on the 0.1375° lattice produced by the automatic refinement (subsection 3.5.4).

Table 3.9 – Optimised route against the great circle, Kiel → Tallinn

Criterion	**Great circle**	**Optimised route (A*)**
Length	1,052 km	1,249 km
Additional length	—	+18.8 %
Interior sample points over land	57 of 158	0 of 107
Share of the interior over land	36.1 %	0.0 %
Weighted traversal cost	undefined (crosses land)	1,817
Navigable	no	yes
Estimated passage time at 14 kn	(not sailable)	48.2 h

The corresponding view of the interface, with all five methods compared on one cost map, is shown in figure 3.7.

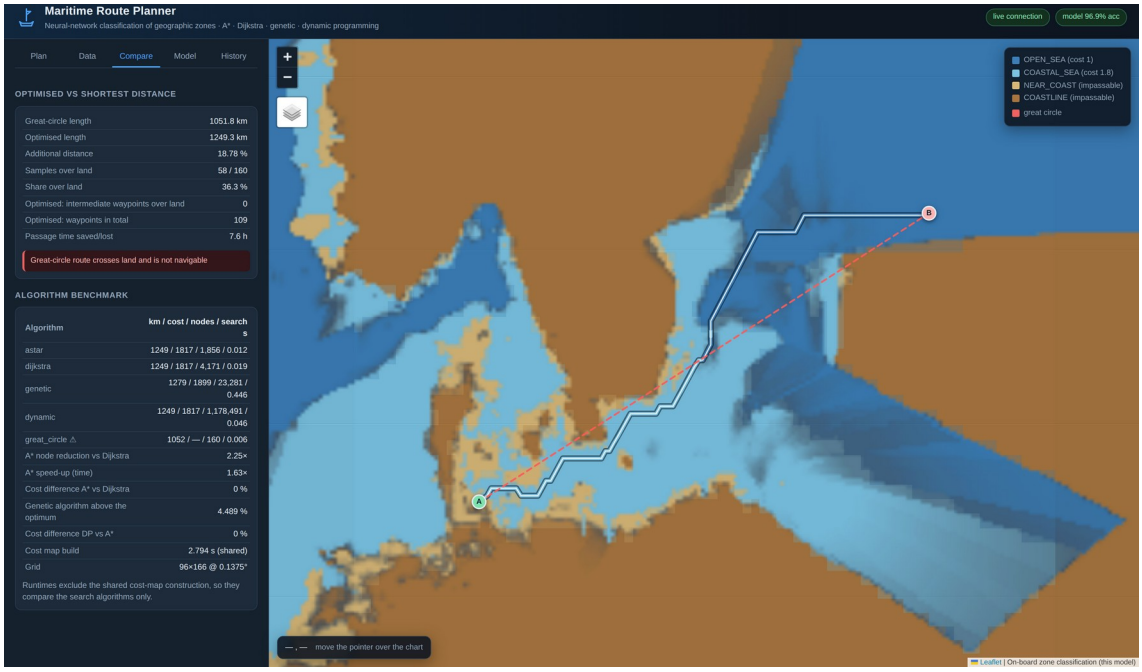


Figure 3.7 – Comparison of the methods on one shared cost map

The great-circle route is 198 km shorter, and it is worthless: 36 % of it runs over Denmark, southern Sweden and Estonia. The additional distance is not an inefficiency of the optimiser but the price of a route that exists, which is why the cost of the great circle is reported as undefined rather than as a number.

3.5.4 Sensitivity to the parameters

Lattice resolution. A coarser lattice is faster but may lose a narrow strait. Requested at 0.5° and at 0.25°, the Kiel–Tallinn voyage finds no path on the first lattice, because the Danish straits are narrower than a cell; the automatic refinement rebuilds the map at 0.275° and at 0.1375° respectively, with a wider margin, and succeeds. The routes returned are 1,226 km and 1,249 km, against 1,280 km when 0.1375° is requested directly with the default margin: a finer lattice follows the shoreline more closely and does not necessarily give a shorter route, because it also resolves more of the coastal cells that the planner avoids.

The coastal weight. Raising the weight of COASTAL_SEA pushes the track away from the shore at the price of length; lowering it gives a shorter but more coastal track. At 1.8 the Kiel–Tallinn route stands off the Swedish coast in the Baltic proper while accepting coastal passage where no alternative exists. The parameter is exposed in the configuration.

3.5.5 Example routes in the eleven sea regions

As recommended by the supervisor, one realistic port-to-port voyage was planned in each of the eleven sea regions of the corpus with all methods of the application. The routes are shown in figure 3.8 and the numbers are given in table 3.10.

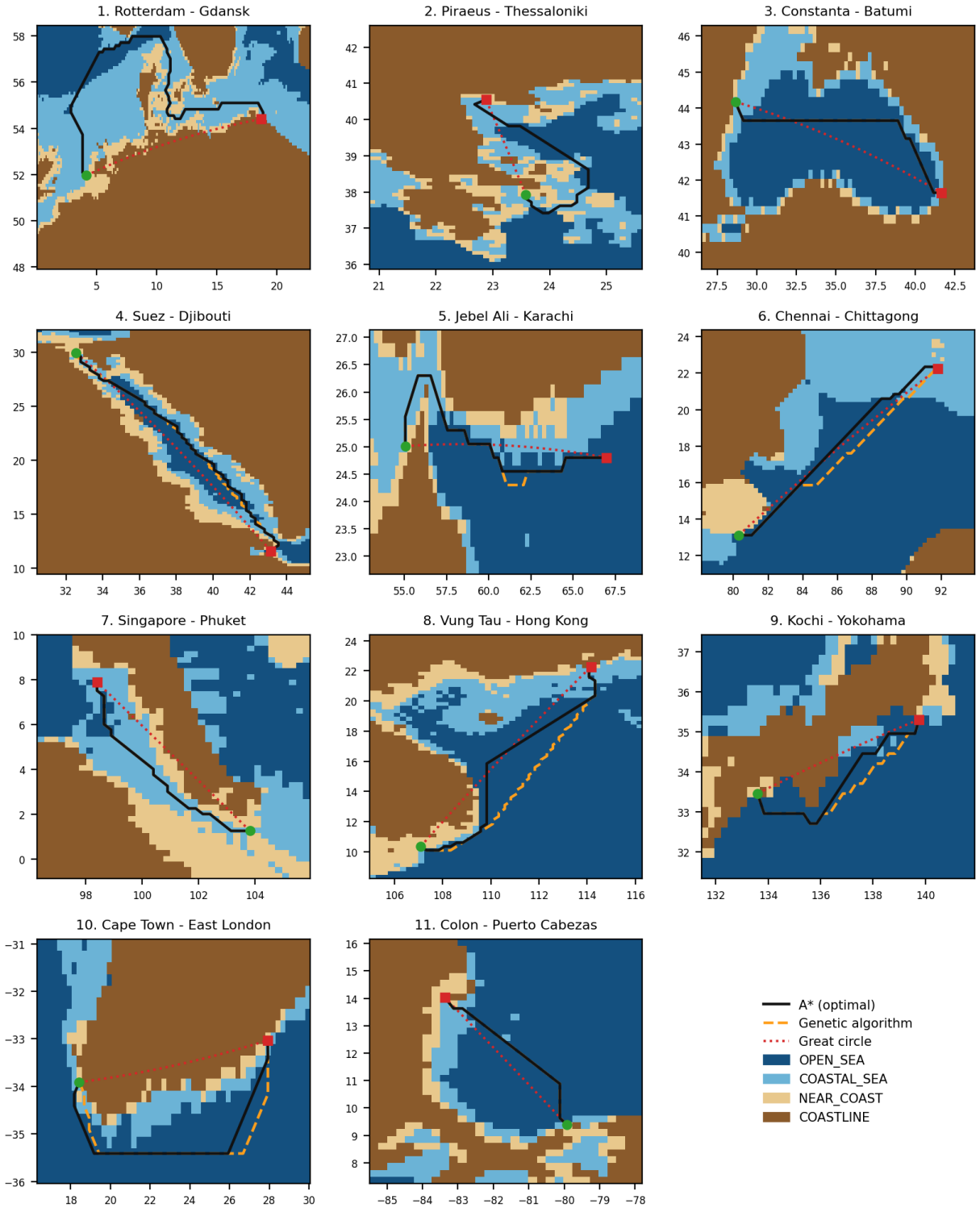


Figure 3.8 – Example routes in the eleven sea regions: A* (solid), genetic algorithm (dashed) and great circle (dotted) over the classified lattice

Table 3.10 – Example voyages in the eleven sea regions

Voyage	A*, km	Great circle, km	Great circle over land	A* = DP	GA above optimum
Rotterdam – Gdansk	2039	1003	96.9 %	yes	failed
Piraeus – Thessaloniki	532	299	33.1 %	yes	failed
Constanta – Batumi	1190	1092	1.3 %	yes	0.00 %
Suez – Djibouti	2510	2313	13.8 %	yes	1.15 %
Jebel Ali – Karachi	1396	1201	10.6 %	yes	1.43 %
Chennai – Chittagong	1659	1586	14.4 %	yes	0.19 %
Singapore – Phuket	1061	950	67.5 %	yes	0.00 %
Vung Tau – Hong Kong	1771	1526	33.1 %	yes	0.19 %
Kochi – Yokohama	763	601	46.9 %	yes	0.18 %
Cape Town – East London	1150	885	97.5 %	yes	6.46 %
Colon – Puerto Cabezas	681	639	5.6 %	yes	0.00 %

A* found a navigable route in all 11 regions, dynamic programming found a route of exactly the same cost in every region, and the genetic algorithm succeeded in 9 of them. All 11 great circles touch land — from 1.3 % of the samples between Constanta and Batumi to 97.5 % between Cape Town and East London, where the great circle runs across the southern tip of Africa. The safe routes are on average 28.28 % longer than the great circles. The choice of the voyages also exposed the limits of the data: ports outside the covered coasts, such as Durban or the Pacific side of Panama, lie in areas that the model classifies as land, so the voyages were chosen within the covered areas.

The Aegean Sea. The new region is one of the most intricate of the corpus, and it showed a weakness of the default lattice. At 0.25° the A* route from Piraeus leaves the Saronic Gulf by a diagonal step into the South Euboean Gulf: both cells are water, but the step passes over Attica, which is narrower than a cell. The waypoints alone do not reveal this; sampling the route every 2 km does — 17 of its 193 samples (8.8 %) lie on land, against at most 1.9 % on every other voyage, where such samples only touch the five-kilometre near-coast zone. The Aegean voyage is therefore planned at 0.1°, and the preset of the client selects this step. The route, 532 km long and 78 % longer than the great circle, then leaves the Saronic Gulf to the south, rounds Cape Sounion, passes the Kafireas Strait between Euboea and Andros and runs north along the outer side of Euboea into the Thermaic Gulf; none of its 246 samples lies on land according to the classifier. The great circle runs over the mainland of Greece for 33 % of its length.

The same voyage also shows a limit of the data. The corpus samples the Aegean rectangle with about one point per eleven kilometres, and on some islands — Kea, Naxos and Skiathos among them — there is no land sample at all, only sea points around them. The

classifier, which describes a point by its sampled neighbourhood, takes such an island for coastal sea, and the route above crosses Kea. Islands of this size become visible to the model only when the corpus contains land points on them.

3.5.6 Testing on 100 random voyages

The voyages above were chosen by hand. To find out whether the application ever lays a course incorrectly, it was also tested on 100 voyages drawn at random with a fixed seed: about nine in each sea region, between water points of the corpus 150–1200 km apart. Each voyage was planned as a user would plan it — A* on the default 0.25° lattice with automatic refinement. The verdict is not taken from the neural network itself: every route is sampled every 1 km and every sample is tested against the 1 km land mask that the package global-land-mask [42] derives from the GLOBE elevation model [41]. The first and the last 5 km are ignored, because a port lies on the coast. A route is correct when no sample falls on land; it grazes the coast when every land run is shorter than 3 km — at the resolution of the mask a shoreline, a narrow channel or an islet touched; it crosses land, and the course is laid incorrectly, when a continuous land run of 3 km or more occurs. The results are summarised in table 3.11; every voyage is listed in table C.1 and drawn in figures C.1–C.11 of appendix C, and the script route_audit.py reproduces them.

Table 3.11 – Results of the test on 100 random voyages

Region	Voyages	Correct	Grazing	Crossing land	No route
North Europe	10	7	1	2	0
Red Sea and Gulf of Aden	9	6	0	3	0
Aegean Sea	9	4	2	3	0
Black Sea and Eastern Mediterranean	9	7	0	0	2
Persian Gulf and Arabian Sea	9	5	0	4	0
Bay of Bengal	9	9	0	0	0
Strait of Malacca and Andaman Sea	9	3	2	4	0
South China Sea	9	8	0	1	0
Japan and North-West Pacific	9	6	0	2	1
Southern Africa	9	9	0	0	0
Central America	9	2	0	0	7
All regions	100	66	5	19	10

The course was correct on 66 voyages and grazed the coast on 5; on 19 voyages it crosses land, and on 10 no route was found. Incorrect courses therefore exist and are not rare. Their

causes were attributed automatically. For a land run, the classifier is asked at the land samples themselves: when it answers “water” there, the classifier has taken land for water; when it answers “land”, the route has stepped diagonally between two water cells across land narrower than a cell. For a voyage without a route, the land mask is searched for a sea connection between the two ports. The voyages with an error were then planned again on a 0.1° lattice (table 3.12).

Table 3.12 – Causes of the errors and the effect of a 0.1° lattice

Error	**Cause**	**Voyages**	**Corrected at 0.1°**
Crossing land	the classifier takes land for water	16	5
Crossing land	diagonal step across land narrower than a cell	3	2
No route	no open sea connection between the ports	7	—
No route	a sea connection exists but the classified lattice closes it	3	0

The classifier is the dominant cause: in 16 of the 19 crossings it takes for water a piece of land that the corpus samples too sparsely — islands such as Masirah off Oman, Singapore, the Danish islands and the islands of the Aegean Sea, the northern tip of Honshu and the Horn of Africa, where voyage 79 crosses 185 km of Somalia. Such places lie at the edges of the sampled areas or between sampled points, and the classifier, which describes a point by its sampled neighbourhood, fills them with water. The lattice is responsible for 3 crossings, among them a cut across the tip of the Musandam peninsula at the Strait of Hormuz. A 0.1° lattice corrects 7 of the 19 crossings, but it cannot correct a wrong classification, and in 3 cases it closes the passage completely. Of the 10 voyages without a route, 7 have no open sea connection: most join the Caribbean coast of Panama with its Pacific coast, which only the Panama Canal connects, so the refusal is correct; the others join the Black Sea with the Mediterranean through the Bosporus, which is narrower than both the land mask and the finest lattice of 0.04°, a limitation of a raster planner. In the remaining 3 a connection exists but the classifier closes it, in the sparsely sampled Gulf of Panama and between the Pacific coast of Honshu and the Sea of Japan. A request took 2.1 s on average and at most 53 s, when the refinement went down to the finest lattice. Examples of incorrect courses are shown in figure 3.9.

The test shows that the safety of the course depends first of all on how densely the corpus samples the coast: where land is not sampled, the classifier fills it with water, and the exact search then uses the gap. Three remedies follow. Land samples should be added exactly where the errors occurred — the test lists them. Every planned route should be checked against an independent coastline in the same way as in this test, and corrected or rejected when it crosses land, which turns the test into a safeguard of the application. In island seas a finer lattice should be used from the start. The second remedy has been built into the application; it is described in the next subsection.

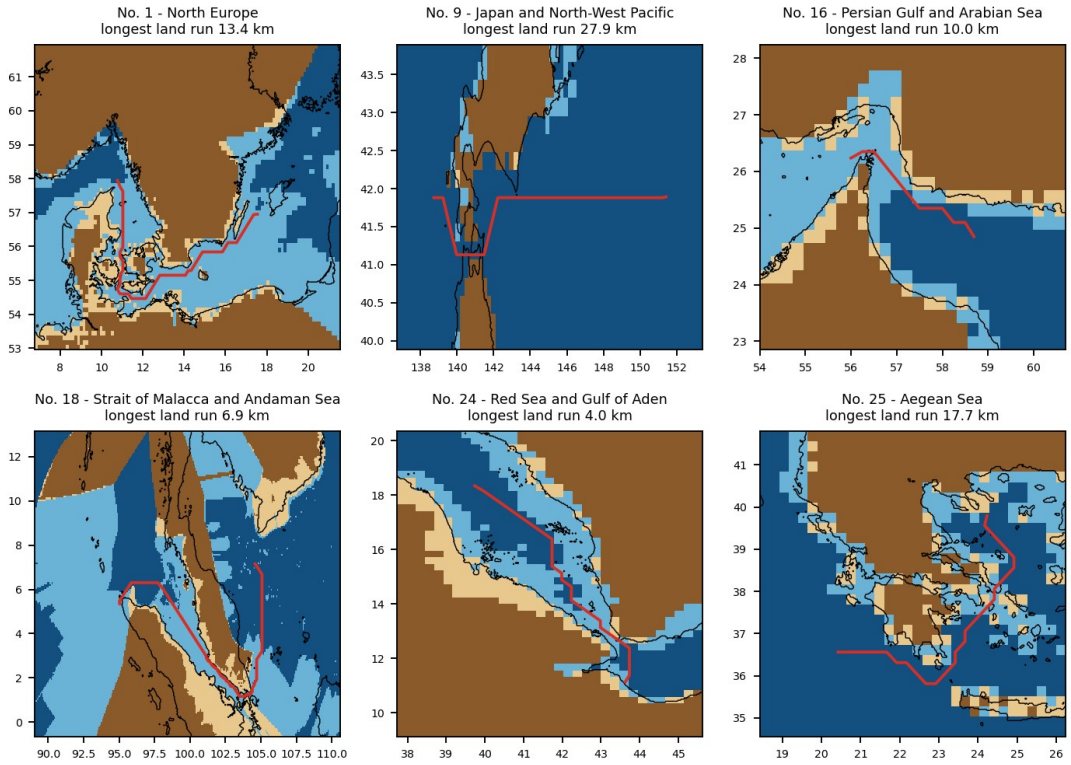


Figure 3.9 – Incorrect courses of the test: classified lattice, coastline of the independent land mask (black line) and the route (red)

3.5.7 Automatic coastline check of the route

Following the second remedy, the check of the test was built into the planner itself, as the module routing/coast_check.py of version 1.2 of the application. After every search the route is sampled every kilometre and each sample is looked up in the same independent land mask [41, 42]; the first and the last 5 km are not checked. A continuous land run of 3 km or more makes the course incorrect, and the planner then corrects it automatically:

1) the cells under the land samples, the route vertices at both ends of the offending legs and every navigable cell within two cells of the run whose centre lies on land of the mask are made impassable on a copy of the cost map, and A* searches again; the round is repeated, up to eight times, until the route no longer crosses land. Water cells are not blocked, so a real strait stays open, and the classification itself is not changed;
2) when the blocked cells close the only passage, or the rounds do not remove the land, the voyage is planned again on a finer lattice — 0.1°, then 0.55 of the step — which is checked in the same way;

3) Dijkstra, dynamic programming or the genetic algorithm, when one of them was chosen, is then run on the corrected map, so that the route shown is that of the selected method; the heuristic genetic algorithm gets up to three more rounds on its own route. A route that still crosses land is returned with a warning, and its land samples are marked on the chart.

The result is shown in a coloured banner above the route summary — passed, corrected automatically, or a warning (appendix A) — and it is written to the PDF report, to the GeoJSON export and to a new column of the route table; an older database receives the column when it is opened. A port given by a city-centre coordinate lies on land, and the straight connector from it to the first navigable cell may cross the shore; land that begins at the quay and ends within that connector is reported as the port approach and is not treated as an error. Without the package global-land-mask the check is skipped and the planner works as before. The check is covered by fifteen automated tests (table 3.8).

The test of subsection 3.5.6 was repeated with the check switched on; table 3.13 compares the two runs. All 19 courses that crossed land were corrected automatically, and none needs the warning. No voyage that was correct before became worse. The corrected routes are on average 62 km longer than the incorrect ones, the price of going round the land. The voyages without a route are unchanged: the check corrects routes that exist, it cannot open a connection that the lattice closes. Every changed voyage is listed in table C.2 of appendix C, and examples are shown in figure 3.10.

Table 3.13 – The test on 100 random voyages without and with the coastline check

Result	Without the check	With the check
Correct course	66	75
Grazing the coast (land runs under 3 km)	5	15
Crossing land	19	0
No route	10	10
Routes corrected automatically	–	19
of them on a finer lattice	–	3
Routes with a warning	–	0
Mean extra distance over the great circle, %	23.0	26.0
Mean request time in the repeated test, s	3.56	3.60

The check costs little time. A route that passes is checked in a few milliseconds; only a corrected one needs additional searches and possibly a new lattice. The repeated test was run on another computer, about 1.6 times slower than the one of subsection 3.5.6, so its times are compared only with each other: the mean request time rose from 3.56 s to 3.60 s, by about 1 %. The check also changed four of the five preset voyages of the user interface. Without it, Helsinki – Copenhagen ran 108 km over the skerries of the Finnish coast, Rotterdam – Lisbon 41 km across Cabo da Roca, Kiel – Tallinn 25 km over the peninsulas of northern Estonia and Piraeus –

Thessaloniki 12 km across the island of Kea; with the check every preset passes, with A*, Dijkstra and dynamic programming alike, in at most 4.8 s on that computer.

One reservation must be stated. After the correction the final route is judged by the same mask that guided the correction, so “correct” in the right-hand column of table 3.13 means agreement with the mask rather than an independent confirmation; the left-hand column remains the independent evaluation of the planner and the classifier. The mask itself has a resolution of about one kilometre, so a channel narrower than that appears as land, and a small islet may be missing. The check therefore does not make the application a navigation system: it removes the gross errors of the classifier, which were the dominant cause of the incorrect courses.

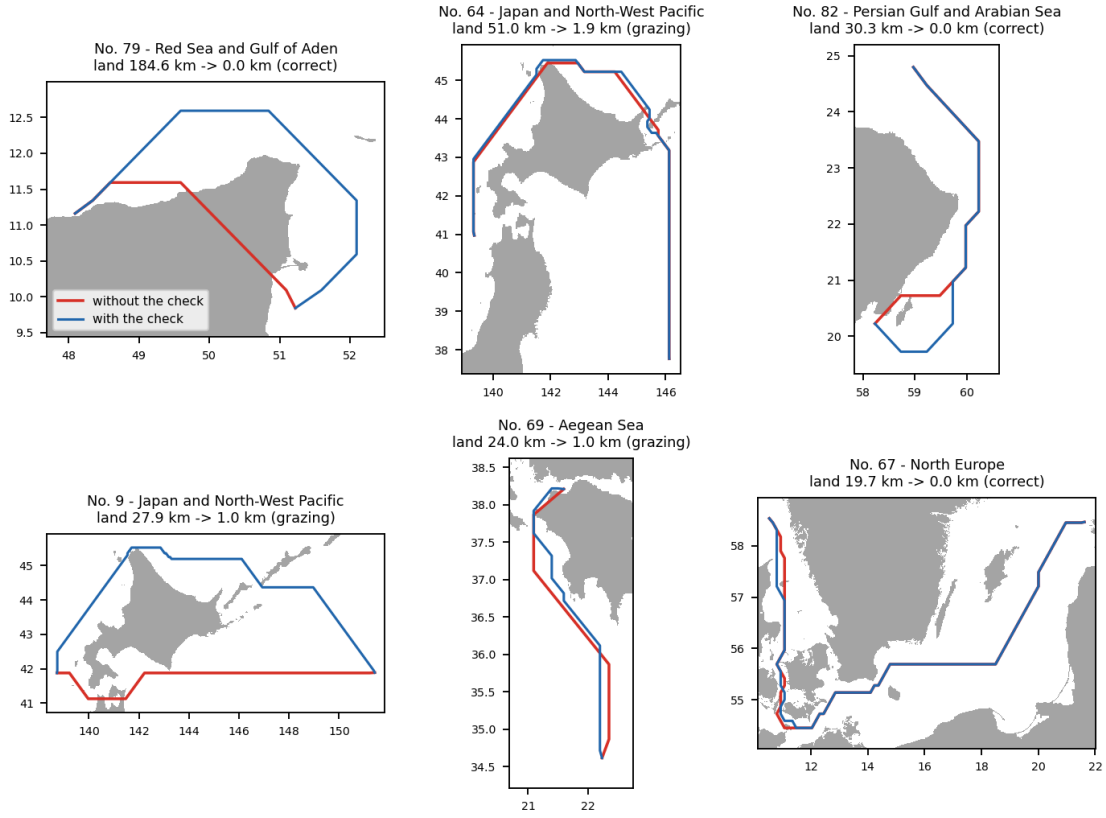


Figure 3.10 – Courses of the test that crossed land (red) and the same voyages with the automatic coastline check (blue), on the land of the independent mask (grey)

3.5.8 Complex courses

Seven courses were chosen because each combines several difficulties: a voyage round Denmark through the Danish straits and the Skagerrak, two voyages through the Baltic and the Øresund, a voyage between the islands of the Aegean Sea, a voyage from the Red Sea round the Arabian Peninsula into the Persian Gulf, a voyage through the Singapore Strait and the South

China Sea, and a voyage up the Gulf of Bothnia through the Kvarken. Each was planned with A* by the planner of version 1.2 and by that of version 1.3, with the default parameters; the routes were judged by the coastline check. The results are given in table 3.14.

Table 3.14 – Complex courses planned by versions 1.2 and 1.3

Course	What makes it complex	v1.2: outcome	km	s	v1.3: outcome	km	s	lattice s
Gdańsk → Bergen	round Denmark: Danish straits, Kattegat, Skagerrak	grazing	1,736	5.7	grazing	1,736	1.5	2
Helsinki → Copenhagen	Gulf of Finland, Baltic, Øresund	grazing	1,150	0.7	correct	1,151	0.6	1
Rotterdam → Gdańsk	North Sea, Skagerrak, Danish straits, Baltic	correct	2,048	3.9	grazing	1,995	3.9	3
Piraeus → Rhodes	between the Aegean islands	grazing	604	1.0	grazing	604	1.1	3
Jeddah → Dubai	Red Sea, Bab-el-Mandeb, Arabian Sea, Hormuz	no route	–	126.4	correct	4,423	5.5	2
Singapore → Hong Kong	Singapore Strait, South China Sea	correct	2,823	0.7	correct	2,823	0.6	1
Kapellskär → Oulu	Gulf of Bothnia, the Kvarken	crosses land 10 km	739	11.1	crosses land 5 km	737	20.3	6

Version 1.2 gives a route without land on 5 of the seven courses, version 1.3 on 6. The decisive difference is the voyage Jeddah – Dubai: version 1.2 refined the lattice four times, down to 0.0416°, kept the passage outside the search area and reported after 126 s that no route exists; version 1.3 recognised that the water of both ports reaches the edge of the lattice, widened the area at the same cell size and found the route of 4,423 km in 5.5 s. On the other courses the routes of the two versions are of almost the same length; the voyage Gdańsk – Bergen is planned 3.7 times faster, because the land-mask fusion corrects the first route at once, without a new lattice.

The voyage Kapellskär – Oulu is the limit of the measures. Its first route crosses 234 km of land: in the Gulf of Bothnia the corpus has few samples, and the classifier takes large parts of the Finnish coast for water. The corrections shorten the land stretch from 9.6 km to 4.8 km but cannot remove it, because in the Kvarken all water lies within a few kilometres of the shore, which the classifier calls the impassable NEAR_COAST zone. The route is shown with the warning and its land is marked on the chart; the remedy is land and water samples of the Gulf of Bothnia in the corpus, not the search.

3.5.9 Comparison of the methods on one course and their complexity

The supervisor suggested comparing the results of the different algorithms for one course. For every course of table 3.14 the five methods were run on one shared cost map — the map of the corrected A* route — with the default parameters; the comparison is the Compare

algorithms function of the interface. The routes are drawn in figure 3.11 and the figures are collected in table 3.15.

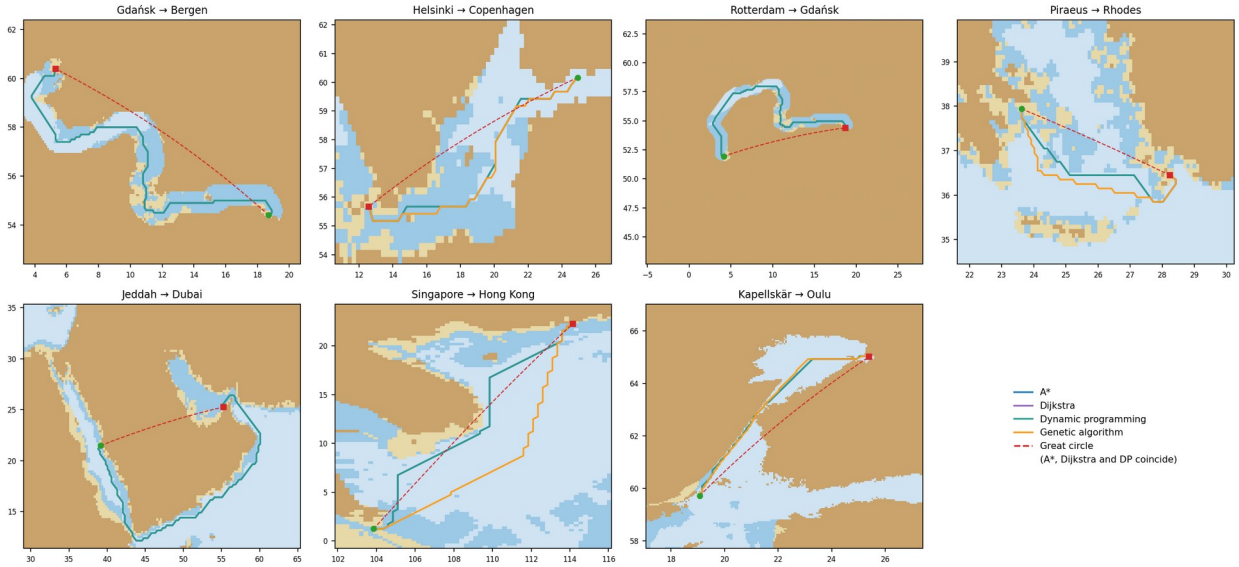


Figure 3.11 – Routes of A*, Dijkstra, dynamic programming, the genetic algorithm and the great circle on the complex courses

Table 3.15 – The methods on the complex courses

Course	Lattice	A*, s	A* steps	Dijkstra steps	DP sweeps	DP, s	GA	GA above opt.	GA, s
Gdańsk → Bergen	0.1°	0.03	2,172	2,185	205	0.10	no route	–	1.11
Helsinki → Copenhagen	0.25°	0.01	624	632	61	0.02	found	3.0 %	0.47
Rotterdam → Gdańsk	0.1°	0.04	2,559	2,604	225	0.53	no route	–	1.21
Piraeus → Rhodes	0.1°	0.01	890	2,526	73	0.02	found	2.7 %	0.38
Jeddah → Dubai	0.25°	0.03	2,528	3,233	169	0.07	step limit	–	3.06
Singapore → Hong Kong	0.25°	0.03	2,244	3,970	98	0.03	found	0.7 %	0.26
Kapellskär → Oulu	0.055°	0.03	2,887	8,228	231	0.19	found	0.7 %	0.73

A*, Dijkstra and dynamic programming return the same optimal cost on every course, as they must. Dijkstra expands 1.0–2.9 times more cells than A*; the ratio is smallest where the course turns away from the destination — round Denmark or round Arabia — because there the great-circle heuristic is least informative. The genetic algorithm finds a route on 4 of the 7 courses; it fails on Gdańsk → Bergen, Rotterdam → Gdańsk, Jeddah → Dubai. These are exactly the courses that must turn away from the destination: the chromosome keeps its waypoints ordered along the line between the ports (subsection 2.4.7), which excludes a route that runs

backwards for a while, and no setting of the population or the number of generations changes that (subsection 3.5.10). Intermediate points remove the obstacle, because every leg is simple.

Table 3.16 sets the measured work against the theoretical complexity of table 2.9 for the course Jeddah – Dubai, on a lattice of 3,951 navigable cells. The worst-case bound of A* and Dijkstra, $(V + E) \log_2 V$ with $E \le 8V$, assumes that every cell is expanded; the searches stop at the destination and expand a small part of the lattice. For dynamic programming the worst case is V sweeps of E updates; the iteration converged after S = 169 sweeps — the number of moves of the longest shortest path to the destination — so its share of the bound is S / V, but in absolute terms its work is the largest, because every sweep updates every cell; it is fast only because a sweep is a few array operations. The genetic algorithm does a fixed amount of work set by its parameters: the initial population and G generations of P routes of about L cells each, whatever the course; it stops earlier only after a run of generations without improvement.

Table 3.16 – Theoretical complexity and measured work, Jeddah → Dubai

Method	Time complexity	Work measured	Unit of work	Worst-case bound	Share of the bound
A*	O((V + E) log V)	20,224	cell expansions × 8 moves	$(V + E) \log_2 V$ = 424,859	4.76 %
Dijkstra	O((V + E) log V)	25,864	cell expansions × 8 moves	$(V + E) \log_2 V$ = 424,859	6.09 %
Dynamic programming	O(S * E), S <= V	5,341,752	Bellman updates × 8 moves	$V \cdot E$ = 124,883,208	4.28 %
Genetic algorithm	O(G * P * L)	24,081	evaluations × L cells	$((G + 1) \cdot P + 1) \cdot L$ = 24,081	100.00 %

3.5.10 Step limits and parameters of the methods

The example of the supervisor was reproduced on the course Gdańsk – Bergen on a lattice of 0.1° with 7,776 navigable cells: A* and Dijkstra were run with a maximum of 100 to 1,000,000 steps (table 3.17, figure 3.12). A* needs 5,351 steps and Dijkstra 6,728; with 100 or 1,000 steps both stop with the outcome step limit, from 10,000 on both find the optimal route, and a larger limit changes nothing, because the search ends at the destination. The step limit therefore bounds the work without affecting a search that fits into it.

Table 3.17 – Step limits on the course Gdańsk → Bergen

Maximum number of steps	A*: outcome	A*: steps	A*, s	Dijkstra: outcome	Dijkstra: steps	Dijkstra, s
100	step limit	100	0.002	step limit	100	0.002
1,000	step limit	1,000	0.008	step limit	1,000	0.007
10,000	found	5,351	0.094	found	6,728	0.055
100,000	found	5,351	0.054	found	6,728	0.054
1,000,000	found	5,351	0.059	found	6,728	0.057

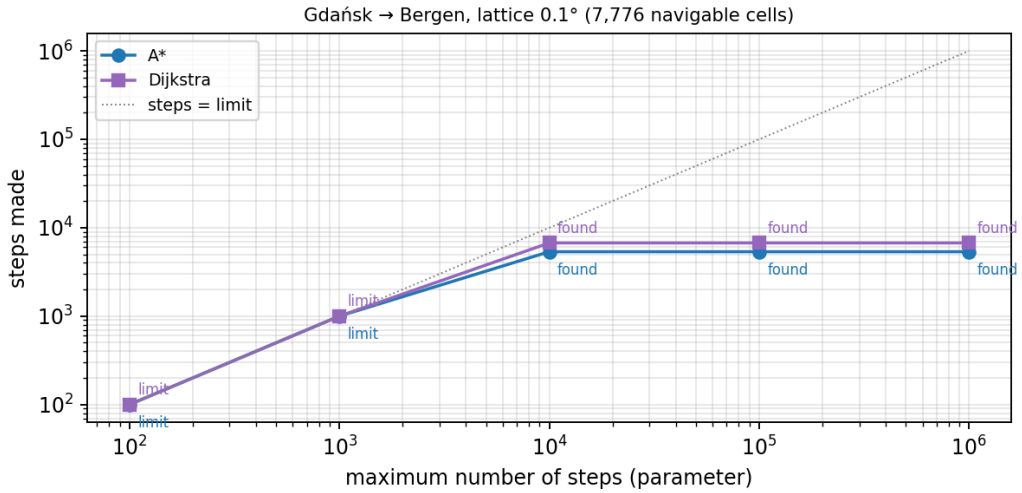


Figure 3.12 – Steps made against the maximum number of steps, Gdańsk → Bergen

The parameters of the genetic algorithm were studied on the course Helsinki → Copenhagen, which the algorithm solves in one leg: the population and the number of generations were varied together from 20 × 100 to 320 × 1,200, with at most 6, 14 or 30 waypoints, three random seeds each (figure 3.13). With the default parameters (80 × 300, 14 waypoints) the route is on average 2.6 % above the optimum in 0.8 s; with 320 × 1,200 it is 0.8 % above it, in 3.6 s. More individuals and generations improve the route slowly and cost time roughly in proportion to their product, as the complexity O(G · P · L) predicts; too few waypoints prevent a route that has to bend round many headlands. On the courses that run backwards for a while no setting helped, while intermediate points did (table 3.18).

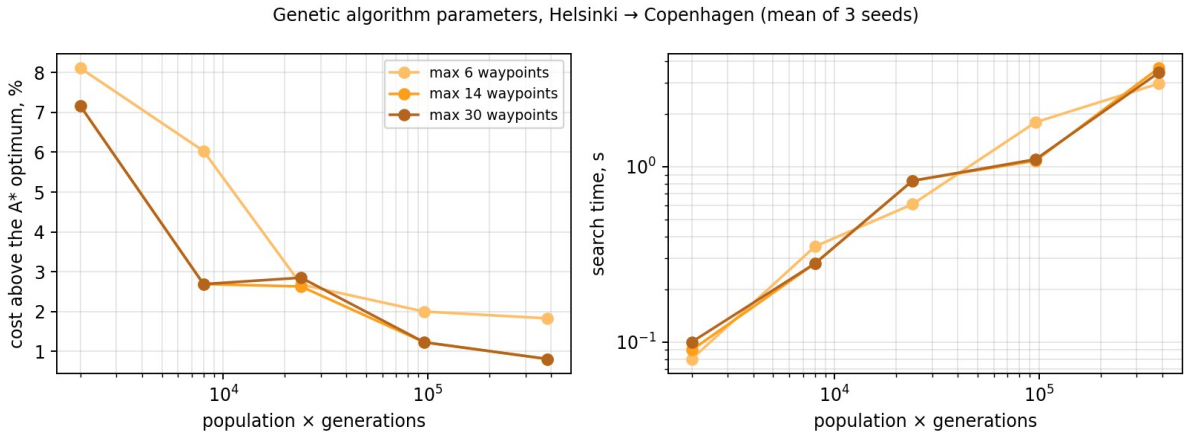


Figure 3.13 – Effect of the population, the number of generations and the number of waypoints on the genetic algorithm

Table 3.18 – Courses without and with intermediate points

Course	Method	Direct: outcome	km	s	Via points	With via points: outcome	km	s
Gdańsk → Bergen	A*	grazing	1,736	1.4	1	grazing	1,761	1.3
Gdańsk → Bergen	GA	no route	–	1.4	1	no route	–	1.1
Jeddah → Dubai	A*	correct	4,423	5.0	3	grazing	4,441	2.5
Jeddah → Dubai	GA	no route	–	9.7	3	correct	4,492	5.3

Finally, the reaction to the time limit and to the Stop button was measured on searches that would take much longer: dynamic programming and Dijkstra on fine lattices of the course Jeddah – Dubai and the genetic algorithm with fifty thousand generations (table 3.19). Each search was stopped within a fraction of a second, its outcome was reported and its result file written.

Table 3.19 – Reaction to the time limit and to the Stop button

Course	Method	Lattice	Stopped by	Outcome	Ended after, s	Reaction, s
Jeddah → Dubai	DP	0.05°	time limit 1 s	time limit	1.00	0.01
Jeddah → Dubai	Dijkstra	0.04°	time limit 1 s	time limit	1.22	0.22
Gdańsk → Bergen	GA	0.05°	time limit 1 s	time limit	1.22	0.22
Jeddah → Dubai	Dijkstra	0.04°	Stop after 1 s	cancelled	1.22	0.22

To make sure that the measures do not spoil the simple courses, the 100 random voyages of subsection 3.5.6 were planned again with version 1.3: 79 correct, 11 grazing the coast, 2 crossing land and 8 without a route, in 4.7 s on average; 6 voyages changed their verdict against version 1.2. Four of them now pass without touching the coast. Two voyages that version 1.2 reported as having no route — from the Pacific coast of Honshu into the Sea of Japan and from the Mediterranean into the Black Sea — now obtain a route through the wider search area, but it leads through a strait that the 1 km mask does not resolve (the Bosporus is narrower than one kilometre), so it is returned with the warning. No route that was correct before became incorrect.

3.5.11 Limitations

The following limitations are stated explicitly, because a decision-support tool whose limits are not stated is more dangerous than one that is absent.

- The application is not a certified navigation system and does not use official electronic navigational charts; it must not be the sole basis for navigation.
- The cost model takes no account of weather, currents, tides, ice, traffic separation schemes or the draught of the particular vessel.

- The corpus covers eleven sea regions; outside them the classification is unreliable and may turn open sea into land. Inside them, land that the corpus samples too sparsely — small islands such as Kea or Naxos, narrow peninsulas — is taken for sea; on 19 of 100 random voyages the course crossed land for this reason or because of the lattice (subsection 3.5.6). The automatic coastline check corrects 19 of them (subsection 3.5.7), but it relies on a land mask of about one kilometre, which cannot resolve narrower channels.
- The resolution of the lattice limits the narrowest channel that can be found, and a diagonal step may cut across a peninsula narrower than a cell; automatic refinement and a finer step for island seas mitigate this but cannot eliminate it.
- The genetic algorithm can converge on a strait that the lattice closes and gives no guarantee of optimality.
- The chromosome of the genetic algorithm orders its waypoints along the line between the ports, so a course that must run backwards for a while — round Denmark, round Arabia — is found only with intermediate points. Where the classifier takes large stretches of coast for water, as in the Gulf of Bothnia, the corrections reduce but do not remove the land on the route; the route then carries a warning.

3.5.12 Conclusions on the chapter

All modules required by the assignment have been implemented and integrated. After the removal of mislabelled points the classifier reaches 96.95 % accuracy on a held-out test set, above the target of ninety per cent, with water–land errors in 0.29 % of points. A*, Dijkstra and dynamic programming return identical optimal costs on all seventeen test voyages; the genetic algorithm finds a navigable route on fifteen of them, a few per cent above the optimum. Every route produced by the exact methods is navigable over its whole interior as judged by the classifier on the lattice, while none of the corresponding great-circle routes is. On 100 random voyages checked against an independent land mask the course was correct or only grazed the coast on 71, crossed land on 19 — mostly where the corpus samples the coast too sparsely — and was not found on 10, 7 of them without an open sea connection. With the automatic coastline check built into the planner, 19 of the 19 incorrect courses are corrected and the course is correct or grazes the coast on 90 voyages. One hundred and one automated tests cover the modules and pass.

In version 1.3 every method has parameters set in the interface, a search can be stopped and is stopped by a time limit within 0.22 s, and the result of every search is saved to a JSON file. Of seven complex courses version 1.3 lays 6 without land, against 5 for version 1.2; the voyage

Jeddah – Dubai, impossible for version 1.2, is planned in 5.5 s. The methods were compared on each course together with their theoretical complexity. 139 automated tests pass.

4 TECHNICAL AND ECONOMIC JUSTIFICATION

This chapter estimates the cost of developing the application and evaluates the economic effect of putting it into service. All monetary values are given in Belarusian roubles (BYN) at the price level assumed for the 2026/27 academic year. Every rate used below is stated explicitly, because the conclusion of the chapter depends on those rates far more than on the method of calculation.

4.1 Calculation of the development costs

4.1.1 Labour intensity

The calendar plan of the project runs from 1 September 2026 to 31 January 2027, that is five months. With an average monthly working-time fund of 167.3 hours, the total labour intensity is 836.5 hours for one engineer-programmer. Table 4.1 distributes that time over the stages in the proportions given in the assignment.

Table 4.1 – Labour intensity by stage

Stage	**Share**	**Hours**	**Labour cost, BYN**
1 System analysis and statement of the problem	15 %	125.5	3 000.00
2 Design of the application	20 %	167.3	4 000.00
3 Implementation and testing	40 %	334.6	8 000.00
4 Technical and economic justification	5 %	41.8	1 000.00
5 Energy and resource saving	5 %	41.8	1 000.00
6 Preparation of the documentation	10 %	83.7	2 000.00
7 Preparation for the defence	5 %	41.8	1 000.00
Total	100 %	836.5	20 000.00

4.1.2 Cost items

The monthly salary of an engineer-programmer is taken as 4 000.00 BYN, which gives an hourly rate of 23.91 BYN. The remaining items are computed from that base at the rates in force in the Republic of Belarus: additional salary at 15 % of the basic salary, contributions to the

Social Protection Fund at 34 % and compulsory accident insurance at 0.6 % of the total payroll, and overheads at 60 % of the basic salary.

The complete estimate is given in table 4.2.

Table 4.2 – Development cost estimate

No.	Cost item	Basis of calculation	Amount, BYN
1	Basic salary	836.5 h × 23.91 BYN/h	20 000.00
2	Additional salary	15 % of item 1	3 000.00
3	Social Protection Fund contributions	34 % of items 1 + 2	7 820.00
4	Compulsory accident insurance	0.6 % of items 1 + 2	138.00
5	Amortisation of the workstation	3 500 BYN over 5 years, 5 months, 90 % use	262.50
6	Electricity	100.4 kWh × 0.2969 BYN/kWh	29.80
7	Software licences	the entire stack is open source	0.00
8	Materials and consumables	printing, storage media	180.00
9	Overheads	60 % of item 1	12 000.00
10	Other expenses	5 % of items 1–9	2 171.52
–	Total cost of development	–	45 601.82

Item 7 deserves a comment, because it is the largest single saving in the estimate rather than the smallest entry. Python, PyTorch, scikit-learn, FastAPI, SQLite, ReportLab and Leaflet are all distributed under permissive or copyleft open-source licences that permit commercial use without payment. An equivalent development on a proprietary numerical platform, or against a licensed chart-data library, would have added several thousand roubles of annually renewed licence fees to this estimate and to every year of operation thereafter.

Item 5 also reflects a design decision rather than an accident: the application trains and runs on an ordinary processor, so no graphics accelerator had to be bought or rented. This is examined further in chapter 5.

4.2 Evaluation of the economic effectiveness

4.2.1 Sources of the economic effect

The application produces an economic effect from two sources.

1) Avoided licence fees. The application performs, for the class of tasks it covers, the work of a commercial route-planning module. Where such a module would otherwise be licensed per seat, that fee is avoided.

2) Saved planning labour. Constructing and checking a course by hand on a chart takes a watch officer tens of minutes per voyage. The application produces a first approximation within a few seconds, together with a quantitative comparison against the shortest-distance route, leaving the officer to verify and refine rather than to construct.

A saving of fuel is deliberately not claimed. The routes the application produces are on average 17.29 % (benchmark voyages) to 28.28 % (regional voyages) longer than the great circle, and the application models neither weather nor currents nor the hydrodynamics of a particular hull. Claiming a fuel saving on this basis would not be defensible. The value of the application lies in producing a route that can actually be sailed, and in producing it quickly, not in shortening one that already could be.

4.2.2 Scenario analysis

The size of the effect depends almost entirely on assumptions about the scale of deployment and about the price of the software it displaces, neither of which can be known precisely in advance. A single point estimate would therefore convey false precision. Three scenarios are evaluated instead, with the annual cost of maintenance taken as a share of the development cost and the hourly rate of a watch officer taken as 32.87 BYN (5 500 BYN per month).

The assumptions of the three scenarios are collected in table 4.3, and the results they lead to in table 4.4.

Table 4.3 – Assumptions of the three scenarios

Assumption	**Pessimistic**	**Base**	**Optimistic**
Seats replaced	4	12	20
Licence fee avoided per seat, BYN/year	1 200	1 500	1 800
Vessels served	4	8	12
Voyages planned per vessel per year	30	40	45
Planning time saved per voyage, min	15	25	35
Annual maintenance, share of development cost	12 %	10 %	8 %

Table 4.4 – Results of the scenario analysis

Indicator	Pessimistic	Base	Optimistic
Saving of planning labour, BYN/year	986	4 383	10 356
Avoided licence fees, BYN/year	4 800	18 000	36 000
Gross annual effect, BYN	5 786	22 383	46 356
Annual maintenance, BYN	5 472	4 560	3 648
Net annual effect, BYN	314	17 823	42 708
Return on the development cost	0.7 %	39.1 %	93.7 %
Payback period, years	not achieved	2.56	1.07
Net present value over 5 years at 12 %, BYN	−44 470	+18 647	+108 349

4.2.3 Interpretation

The scenarios differ by two orders of magnitude in their outcome, and the reason is instructive. The development cost is fixed and is incurred once; the marginal cost of an additional installation is essentially zero, since the application is a single Python package with no per-seat licence and no server infrastructure. The economics of the project are therefore entirely a question of how many seats it is deployed to.

In the pessimistic scenario — four seats, a small operator, a cheap displaced licence — the net effect barely exceeds the cost of maintaining the application, and the development does not pay for itself within any reasonable horizon. In the base scenario, which corresponds to a training centre together with the operations office of a small fleet, the investment is recovered in about two and a half years and the five-year net present value is positive. In the optimistic scenario the payback falls to just over a year.

The practical conclusion is that the project is economically justified under the base and optimistic scenarios, and that the decisive factor is the breadth of deployment rather than any technical parameter of the application. It follows that effort spent on making the application easy to install and to operate — the single-file database, the absence of a server requirement, the offline chart, the installation guide in appendix B — is not incidental polish but the part of the work with the greatest economic leverage.

Two further sources of value are not quantified above because they cannot be expressed reliably in money, but they should be recorded. The first is safety: a route that has been checked against a depth-aware model before it is sailed reduces the probability of a grounding, whose cost is measured in hundreds of thousands of roubles and potentially in lives. The second is educational: the application, its source code and this explanatory note form a complete worked

example of applying machine learning to a classical optimisation problem, which has value in teaching independently of any operational deployment.

4.2.4 Conclusions on the chapter

The cost of developing the application has been estimated at 45 601.82 BYN, of which 20 000.00 BYN is the basic salary of one engineer-programmer over 836.5 hours. No licence fees were incurred, because the entire technology stack is open source, and no accelerator hardware was required. The economic effect has been evaluated in three scenarios; under the base scenario the net annual effect is 17 823 BYN, the payback period is 2.56 years and the five-year net present value at a discount rate of 12 % is positive.

5 ENERGY AND RESOURCE SAVING

Energy saving is treated here at two levels: the energy consumed by the application itself, which is small but was influenced by several explicit design decisions, and the energy consumed by the vessels whose routes the application plans, which is larger by many orders of magnitude and which the application can influence only indirectly.

5.1 Analysis of energy consumption

5.1.1 Energy consumed in training the model

The classifier was trained on the processor of an ordinary workstation, without a graphics accelerator. Training ran for 97 epochs in 255 s. Taking the power drawn by four loaded processor cores together with the rest of the system as 65 W, the energy consumed by a complete training run is obtained from formula (5.1):

$$E = 65\ W \times 255\ s = 16.6\ kJ \approx 4.6\ W{\cdot}h \tag{5.1}$$

At the Belarusian emission factor for electricity of about 0.40 kg of CO_2 per kilowatt-hour, a training run corresponds to roughly 1.8 g of CO_2 — comparable with boiling a tablespoon of water. The figure is reported not because it is large but because it is the correct order of magnitude to have in mind: for a model of this size, energy is not a constraint, and design decisions should be taken on other grounds.

That conclusion is itself the result of a design decision. A model of 98,364 parameters over 28 tabular features trains in 4.3 minutes on a processor. Had the same task been approached by, for example, feeding raster tiles of bathymetry to a convolutional network, training would have required a graphics accelerator drawing 250–400 W for hours, and the energy of a single run would have risen from watt-hours to kilowatt-hours — a factor of several hundred. The feature-engineering effort described in subsection 2.3.3 is, among other things, an energy saving measure.

5.1.2 Energy consumed in operation

Table 5.1 reports the energy of the operations the application performs, computed from the measured run times at the same 65 W under load and 12 W at idle.

Table 5.1 – Energy consumption of the application

Operation	Time	Energy	Remark
Complete training run	255 s	4.6 W·h	performed once
Genetic-algorithm search of one route	≤ 1 s	≤ 0.018 W·h	search time 15–45 times that of A*
Preparation of the dataset	16 s	0.29 W·h	performed once
Planning one route, typical / longest	0.1 s / 2.9 s	0.002 / 0.053 W·h	about 480,000 typical routes per kW·h
Classifying one point (live read-out)	< 5 ms	< 0.0001 W·h	negligible
Rendering one chart tile	0.35 s	0.006 W·h	cached after the first request
Server at idle	24 h	288 W·h	dominates if the server is left running

The table contains the most important single fact of this chapter: the energy cost of an idle server over one day exceeds the energy cost of ten thousand planned routes. For this application, and for small machine-learning services generally, the useful work is not what consumes the energy; the machine waiting to do it is.

5.1.3 Consumption of other resources

The storage and network resources the application consumes are listed in table 5.2.

Table 5.2 – Consumption of storage and network resources

Resource	Volume	Remark
Trained model weights	0.4 MB	small enough to ship inside the repository
Reference index	2.0 MB	compressed arrays of the training points
Database after 100 routes	≈ 0.5 MB	single SQLite file
Client payload on first load	≈ 0.35 MB	including the vendored map library
Network traffic per planning request	≈ 0.3 MB	mostly the classification overlay
External network traffic	0 in on-board mode	no tile service, no content delivery network

5.2 Recommendations for reducing energy consumption

5.2.1 Measures already implemented

Six measures that reduce energy or resource consumption were incorporated in the course of the work.

1) A compact model over engineered features rather than a large model over raw rasters. This removes the need for accelerator hardware entirely and reduces the energy of a training run by roughly two orders of magnitude.
2) Early stopping with restoration of the best weights. Training stops when the validation loss ceases to improve, so no energy is spent on epochs that do not improve the model.
3) Optimisation of the A* heuristic. Replacing the vectorised call with pre-computed scalar trigonometry made the search about 2.0 times faster; since the search is the only part of a planning request that scales with the size of the problem, this is a proportional energy saving on every request.
4) Caching of chart tiles. A tile is rendered once and then served from memory, so panning and zooming cost nothing after the first pass.
5) Batched inference. The lattice is classified in batches of 16 384 coordinates rather than one at a time, which amortises the per-call overhead of the framework over thousands of points.
6) An embedded database and vendored client libraries. No database server process runs alongside the application, and no request leaves the machine to fetch a script or a map tile in on-board mode, which removes both the idle power of a second process and the network traffic.

5.2.2 Further recommendations

- Do not leave the server running when it is not in use. Given the figures in table 5.1, this single measure dominates every other software optimisation: starting the application on demand saves more energy than any conceivable improvement to the search.
- Reuse the cost map across requests with the same bounding box and resolution. The construction of the cost map is about ninety-five per cent of the energy of a planning request, and an operator exploring alternative destinations in the same sea area rebuilds it needlessly each time.
- Cache the classification of the lattice on disk between sessions. The classification of a given cell does not change unless the model is retrained, so a persistent cache keyed by the model version would eliminate the dominant cost for repeated areas.
- Retrain only when the corpus has materially changed. Retraining is cheap in absolute terms but it is also almost always unnecessary; the metadata file records the training date and the corpus statistics so that this can be judged rather than guessed.

- Quantise the model to eight-bit integers if it is ever deployed to an embedded bridge computer. For a model of this size the gain on a workstation would not repay the loss of accuracy, but on a low-power device the trade-off reverses.
- Serve the application from a single low-power machine for several workstations rather than installing it on each. The application is stateless between requests apart from its database, so one instance can serve a whole operations office at essentially the idle power of one computer.

5.2.3 Resource saving in the subject area

The energy consumed by the application is negligible beside the energy consumed by the vessels it plans for. A medium-sized cargo vessel burns of the order of twenty to forty tonnes of fuel per day at sea; one day of steaming, about 250–450 MW·h of fuel energy, therefore exceeds a whole year of running this application continuously by two to three orders of magnitude.

It follows that any influence the application has on the route is far more significant, in energy terms, than its own consumption. Two honest observations must be made here. The routes produced are longer than the great circle — by 17 % on the benchmark voyages and 28 % on the regional ones — so the application does not save fuel relative to an idealised straight line. But that straight line does not exist: all seventeen great circles of the experiments cross land. Against the route that would otherwise be sailed, the application avoids corrections at sea, keeps the track out of shallow congested water, and makes the cost of every detour explicit.

The cost model itself is the natural place to extend this. The traversal weight w(z) is currently a risk weight; nothing in the design prevents it from becoming a fuel weight, combining zone risk with a weather and current field and with the speed–consumption curve of the particular vessel. The search, the interface, the export and the database would be unchanged. This is the most promising direction for further work.

5.2.4 Conclusions on the chapter

The energy consumed by the application has been measured and is small: 4.6 W·h for a complete training run and about 0.009 W·h for a typical planned route, so that a kilowatt-hour is sufficient for about 110 000 routes. Six energy-saving measures were implemented in the course of the work, of which the compact model over engineered features is by far the most significant, and six further recommendations have been formulated. The analysis also shows where attention should not be directed: the idle power of the server exceeds the energy of the useful work by a wide margin, and the energy of the vessel exceeds both by several orders of magnitude.

CONCLUSION

In the course of the diploma project an application has been developed for constructing the course of a maritime vessel on the basis of a neural-network classification of geographic zones. All tasks set in the introduction have been completed.

The analysis of the subject area showed that the difficulty of automated maritime route planning lies in the cost model rather than in the search, and that existing open solutions either ignore depth or are confined to a fixed network of legs, while certified systems verify a route rather than propose one. On that basis the problem was stated as the construction of a learned four-zone cost model together with an exact search over it, and nine functional and eight non-functional requirements were formulated, to which four functional and one non-functional requirement were added after the review of the first version.

The application was designed as a five-layer monolith with a single-page web client and a WebSocket channel for live progress. A relational schema of six relations and one view was designed and SQLite was selected and justified as the storage engine. A classification model was designed around a twenty-eight-component feature vector derived from a label-free reference index, with three explicit measures against target leakage, and its forward pass and training were described mathematically. A rule for removing training points whose label contradicts the measured height was designed; it removed 5,427 of 156,838 points. The cost model and four search methods were designed: A* with an admissible heuristic, Dijkstra, dynamic programming built on the Bellman function, and a genetic algorithm built on a fitness function; the two evaluation functions share the same step cost and were compared.

All modules were implemented and integrated, and the following results were obtained:

- the classifier reaches 96.95 % accuracy, a macro F1 of 0.9695, Cohen’s kappa of 0.9582 and a macro ROC-AUC of 0.9985 on a held-out test set of 22,712 points, against a target of ninety per cent;
- the errors of the classifier are concentrated where they are least harmful: most confuse the two navigable classes with each other, and after the cleaning of the data only 67 test points (0.29 %) are confused across the water–land boundary, against 189 before it;
- the accuracy in the eleven sea regions of the corpus ranges from 93.5 % in the South China Sea to 99.5 % in the Red Sea, and reaches 97.9 % in the Aegean Sea added in the second release of the corpus;
- A* expands 2.5 times fewer nodes than Dijkstra and runs 1.233 times faster, while both return the identical optimal cost on every test voyage (maximum deviation 0 %), which confirms the proof of admissibility experimentally; dynamic programming returns the same optimal cost on all seventeen test voyages;

- the genetic algorithm finds a navigable route on fifteen of the seventeen voyages, on average a few per cent above the optimum; on the remaining two it is trapped by a strait that the lattice closes or by the islands of the Aegean Sea, which shows the weakness of evolutionary search in a maze-like sea;
- all seventeen great-circle routes of the experiments cross land and are not navigable, whereas the interior of every route produced by the application lies entirely in navigable water as judged by the classifier on the lattice, at a cost of 17.29 % (benchmark) and 28.28 % (regional voyages) additional distance on average;
- a test on 100 random voyages checked against an independent 1 km land mask found the course correct or only grazing the coast on 71 voyages, crossing land on 19 and not found on 10 (7 of them without an open sea connection); in 16 of the crossings the classifier took for water land that the corpus samples too sparsely. A check of every route against that coastline was therefore built into the planner: it blocks the offending cells and searches again, if necessary on a finer lattice, and corrected 19 of the 19 courses;
- at the default resolution a complete planning request, including the construction and where necessary the refinement of the cost map, takes at most 2.9 s on the benchmark voyages, against a target of five seconds; on the random voyages it takes 2.1 s on average but up to 53 s when the refinement descends to the finest lattice, and the automatic coastline check adds about 1 % to the mean time;
- after the review of the first version, the parameters of every method — step limits, heuristic weight, the parameters of the genetic algorithm, the lattice and a time limit — were made settable in the application; a search can be stopped and is stopped by its time limit within 0.22 s, and every search is saved to a JSON file whose location is shown on the screen; the interface can be switched between English, Russian and Chinese;
- for complex courses an adaptive search area, the fusion of the land mask into the cost map, a two-level search in a corridor and intermediate points were introduced; of seven complex courses 6 are laid without land, against 5 before, and the voyage Jeddah – Dubai, reported as impossible before, is planned in 5.5 s;
- 139 automated tests cover every module and pass; the source code, with documentation comments for every module, class and function, is published in a public repository.

Three findings of the work are worth stating separately, because they were not anticipated at the design stage. The first is the resolution failure of a raster planner in narrow straits and its remedy by automatic refinement: at a coarse lattice the Danish straits are invisible and the route is reported as impossible, while a finer and wider lattice recovers it. The second is that an asymptotically superior algorithm can be defeated by the constant factor of its own

heuristic: A* expanded 2.5 times fewer nodes than Dijkstra and nevertheless ran slower until the heuristic was rewritten without vectorised scalar calls. The third, found only by end-to-end testing, is that storing the reference coordinates as 32-bit floats made a point fail to match itself by half a metre, so that data from the training corpus was classified with a neighbourhood the network had never seen; identifying the query point by distance rather than by a flag removed the defect.

The technical and economic justification estimated the cost of development at 45 601.82 BYN and evaluated the economic effect in three scenarios; under the base scenario the payback period is 2.56 years and the five-year net present value at 12 % is positive. The analysis of energy consumption showed that a training run costs 4.6 W·h and a planned route about 0.01 W·h, so that the energy of the application is negligible beside the idle power of the machine that hosts it and beside the fuel of the vessel it plans for.

The practical value of the work consists in a complete, documented and tested application that runs offline on an ordinary computer without licence fees, and that can be used as a decision-support tool for voyage planning, as a teaching aid, and as a reference implementation for further research into learned cost models for maritime routing.

The following directions for further work are proposed:

1) attaching a real bathymetric grid (NOAA or GEBCO) [24, 25] and an OpenStreetMap extract [26] in place of the reference index, which would extend the coverage of the model beyond the regions represented in the corpus;
2) extending the traversal weight from a pure risk weight to a fuel weight that combines zone risk with a weather and current field and with the speed–consumption curve of the vessel [28] — the search, the interface, the export and the database would remain unchanged;
3) replacing the uniform lattice with an adaptive one, fine near coasts and coarse in open water; the two-level search of version 1.3 is a first step, since it classifies fine cells only in a corridor around the coarse route;
4) introducing the draught of the particular vessel as an input, so that the boundary between navigable and impassable water becomes vessel-specific rather than universal;
5) calibrating the confidence of the classifier, so that the reported probability can be used as a formal measure of the risk of a cell rather than only as an indication to the operator.

LIST OF ABBREVIATIONS

Abbreviation	Expansion	Abbreviation	Expansion
A*	A-star, an informed graph search algorithm	GIS	Geographic Information System
ACID	Atomicity, Consistency, Isolation, Durability	GPU	Graphics Processing Unit
AdamW	Adaptive Moment Estimation with decoupled Weight decay	HTTP	HyperText Transfer Protocol
AIS	Automatic Identification System	IMO	International Maritime Organization
API	Application Programming Interface	JSON	JavaScript Object Notation
BN	Batch Normalisation	MLP	Multilayer Perceptron
BYN	Belarusian rouble (ISO 4217 currency code)	NOAA	National Oceanic and Atmospheric Administration
CDN	Content Delivery Network	NPV	Net Present Value
CPU	Central Processing Unit	OSM	OpenStreetMap
CRS	Coordinate Reference System	PDF	Portable Document Format
CSV	Comma-Separated Values	PNG	Portable Network Graphics
CVD	Colour Vision Deficiency	REST	Representational State Transfer
DP	Dynamic Programming	ROC-AUC	Area Under the Receiver Operating Characteristic curve
ECDIS	Electronic Chart Display and Information System	ROI	Return On Investment
ENC	Electronic Navigational Chart	SOLAS	International Convention for the Safety of Life at Sea
ESPD	Unified System of Program Documentation	SQL	Structured Query Language
GA	Genetic Algorithm	UI	User Interface
GEBCO	General Bathymetric Chart of the Oceans	UTC	Coordinated Universal Time
GELU	Gaussian Error Linear Unit	XML	Extensible Markup Language
GeoJSON	Geographic JavaScript Object Notation	–	–

LIST OF REFERENCES

1. Guidelines for Voyage Planning : Resolution A.893(21) : adopted on 25 November 1999 / International Maritime Organization. — London : IMO, 2000. — (A 21/Res.893). — URL: https://wwwcdn.imo.org/localresources/en/KnowledgeCentre/IndexofIMOResolutions/AssemblyDocuments/A.893(21).pdf (date of access: 24.09.2026).
2. SOLAS : Consolidated Edition 2024 : International Convention for the Safety of Life at Sea, 1974, its Protocol of 1988, and amendments in force as of 1 July 2024. — London : International Maritime Organization, 2024. — 604 p. — ISBN 978-92-801-1793-6.
3. Hart, P. E. A Formal Basis for the Heuristic Determination of Minimum Cost Paths / P. E. Hart, N. J. Nilsson, B. Raphael // IEEE Transactions on Systems Science and Cybernetics. — 1968. — Vol. 4, No. 2. — P. 100–107. — DOI: 10.1109/TSSC.1968.300136.
4. Dijkstra, E. W. A note on two problems in connexion with graphs / E. W. Dijkstra // Numerische Mathematik. — 1959. — Vol. 1, No. 1. — P. 269–271. — DOI: 10.1007/BF01386390.
5. Russell, S. Artificial Intelligence: A Modern Approach / S. Russell, P. Norvig. — 4th ed. — Hoboken : Pearson, 2021. — 1136 p. — ISBN 978-0-13-461099-3.
6. Introduction to Algorithms / T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. — 4th ed. — Cambridge, MA : The MIT Press, 2022. — 1312 p. — ISBN 978-0-262-04630-5.
7. Goodfellow, I. Deep Learning / I. Goodfellow, Y. Bengio, A. Courville. — Cambridge, MA : The MIT Press, 2016. — 800 p. — ISBN 978-0-262-03561-3. — URL: https://www.deeplearningbook.org (date of access: 24.09.2026).
8. Deep Residual Learning for Image Recognition / K. He, X. Zhang, S. Ren, J. Sun // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). — Las Vegas, 2016. — P. 770–778. — DOI: 10.1109/CVPR.2016.90.
9. Ioffe, S. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift / S. Ioffe, C. Szegedy // Proceedings of the 32nd International Conference on Machine Learning. — PMLR, 2015. — (Proceedings of Machine Learning Research ; Vol. 37). — P. 448–456. — URL: https://proceedings.mlr.press/v37/ioffe15.html (date of access: 24.09.2026).
10. Hendrycks, D. Gaussian Error Linear Units (GELUs) / D. Hendrycks, K. Gimpel // arXiv. — 2016. — arXiv:1606.08415. — URL: https://arxiv.org/abs/1606.08415 (date of access: 24.09.2026).
11. Loshchilov, I. Decoupled Weight Decay Regularization / I. Loshchilov, F. Hutter // 7th International Conference on Learning Representations (ICLR 2019). — New Orleans, 2019. — URL: https://arxiv.org/abs/1711.05101 (date of access: 24.09.2026).

12. Dropout: A Simple Way to Prevent Neural Networks from Overfitting / N. Srivastava, G. Hinton, A. Krizhevsky [et al.] // Journal of Machine Learning Research. — 2014. — Vol. 15, No. 56. — P. 1929–1958. — URL: https://jmlr.org/papers/v15/srivastava14a.html (date of access: 24.09.2026).
13. Rethinking the Inception Architecture for Computer Vision / C. Szegedy, V. Vanhoucke, S. Ioffe [et al.] // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). — Las Vegas, 2016. — P. 2818–2826. — DOI: 10.1109/CVPR.2016.308.
14. PyTorch: An Imperative Style, High-Performance Deep Learning Library / A. Paszke, S. Gross, F. Massa [et al.] // Advances in Neural Information Processing Systems 32 (NeurIPS 2019). — Red Hook : Curran Associates, 2019. — P. 8024–8035. — URL: https://papers.nips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html (date of access: 24.09.2026).
15. Scikit-learn: Machine Learning in Python / F. Pedregosa, G. Varoquaux, A. Gramfort [et al.] // Journal of Machine Learning Research. — 2011. — Vol. 12, No. 85. — P. 2825–2830. — URL: https://jmlr.org/papers/v12/pedregosa11a.html (date of access: 24.09.2026).
16. Omohundro, S. M. Five Balltree Construction Algorithms : Technical Report TR-89-063 / S. M. Omohundro. — Berkeley : International Computer Science Institute, 1989. — 22 p.
17. Sinnott, R. W. Virtues of the Haversine / R. W. Sinnott // Sky and Telescope. — 1984. — Vol. 68, No. 2. — P. 158–159.
18. Douglas, D. H. Algorithms for the Reduction of the Number of Points Required to Represent a Digitized Line or its Caricature / D. H. Douglas, T. K. Peucker // The Canadian Cartographer. — 1973. — Vol. 10, No. 2. — P. 112–122. — DOI: 10.3138/FM57-6770-U75U-7727.
19. Cohen, J. A Coefficient of Agreement for Nominal Scales / J. Cohen // Educational and Psychological Measurement. — 1960. — Vol. 20, No. 1. — P. 37–46. — DOI: 10.1177/001316446002000104.
20. Fawcett, T. An Introduction to ROC Analysis / T. Fawcett // Pattern Recognition Letters. — 2006. — Vol. 27, No. 8. — P. 861–874. — DOI: 10.1016/j.patrec.2005.10.010.
21. The GeoJSON Format : RFC 7946 / H. Butler, M. Daly, A. Doyle [et al.]. — Internet Engineering Task Force, 2016. — 28 p. — DOI: 10.17487/RFC7946. — URL: https://www.rfc-editor.org/rfc/rfc7946.html (date of access: 24.09.2026).
22. Fette, I. The WebSocket Protocol : RFC 6455 / I. Fette, A. Melnikov. — Internet Engineering Task Force, 2011. — 71 p. — DOI: 10.17487/RFC6455. — URL: https://www.rfc-editor.org/rfc/rfc6455.html (date of access: 24.09.2026).
23. Fielding, R. T. Architectural Styles and the Design of Network-based Software Architectures : Doctoral dissertation / R. T. Fielding ; University of California, Irvine. —

Irvine, 2000. — URL: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm (date of access: 24.09.2026).
24. GEBCO Compilation Group. GEBCO 2024 Grid [Electronic resource]. — 2024. — DOI: 10.5285/1c44ce99-0a0d-5f4f-e063-7086abc0ea0f. — URL: https://www.gebco.net/data-products-gridded-bathymetry-data/gebco2024-grid (date of access: 24.09.2026).
25. NOAA National Centers for Environmental Information. ETOPO 2022 15 Arc-Second Global Relief Model [Electronic resource]. — 2022. — DOI: 10.25921/fd45-gt74. — URL: https://www.ncei.noaa.gov/products/etopo-global-relief-model (date of access: 24.09.2026).
26. OpenStreetMap contributors. Planet dump [Electronic resource]. — Data licensed under ODbL. — URL: https://planet.openstreetmap.org ; https://www.openstreetmap.org/copyright (date of access: 24.09.2026).
27. Harrower, M. ColorBrewer.org: An Online Tool for Selecting Colour Schemes for Maps / M. Harrower, C. A. Brewer // The Cartographic Journal. — 2003. — Vol. 40, No. 1. — P. 27–37. — DOI: 10.1179/000870403235002042.
28. Maritime routing and speed optimization with emission control areas / K. Fagerholt, N. T. Gausel, J. G. Rakke, H. N. Psaraftis // Transportation Research Part C: Emerging Technologies. — 2015. — Vol. 52. — P. 57–73. — DOI: 10.1016/j.trc.2014.12.010.
29. Kingma, D. P. Adam: A Method for Stochastic Optimization / D. P. Kingma, J. Ba // 3rd International Conference on Learning Representations (ICLR 2015). — San Diego, 2015. — arXiv:1412.6980. — URL: https://arxiv.org/abs/1412.6980 (date of access: 24.09.2026).
30. Holland, J. H. Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence / J. H. Holland. — Ann Arbor : University of Michigan Press, 1975. — viii, 183 p. — ISBN 0-472-08460-7.
31. Goldberg, D. E. Genetic Algorithms in Search, Optimization, and Machine Learning / D. E. Goldberg. — Reading, MA : Addison-Wesley, 1989. — xiii, 412 p. — ISBN 0-201-15767-5.
32. Frénay, B. Classification in the Presence of Label Noise: A Survey / B. Frénay, M. Verleysen // IEEE Transactions on Neural Networks and Learning Systems. — 2014. — Vol. 25, No. 5. — P. 845–869. — DOI: 10.1109/TNNLS.2013.2292894.
33. Rumelhart, D. E. Learning representations by back-propagating errors / D. E. Rumelhart, G. E. Hinton, R. J. Williams // Nature. — 1986. — Vol. 323, No. 6088. — P. 533–536. — DOI: 10.1038/323533a0.
34. Hornik, K. Multilayer feedforward networks are universal approximators / K. Hornik, M. Stinchcombe, H. White // Neural Networks. — 1989. — Vol. 2, No. 5. — P. 359–366. — DOI: 10.1016/0893-6080(89)90020-8.

35. Lamini, C. Genetic Algorithm Based Approach for Autonomous Mobile Robot Path Planning / C. Lamini, S. Benhlima, A. Elbekri // Procedia Computer Science. — 2018. — Vol. 127. — P. 180–189. — DOI: 10.1016/j.procs.2018.01.113.
36. A new weather-routing system that accounts for ship stability based on a real-coded genetic algorithm / A. Maki, Y. Akimoto, Y. Nagata [et al.] // Journal of Marine Science and Technology. — 2011. — Vol. 16, No. 3. — P. 311–322. — DOI: 10.1007/s00773-011-0128-z.
37. Prechelt, L. Early Stopping — But When? / L. Prechelt // Neural Networks: Tricks of the Trade / ed. G. B. Orr, K.-R. Müller. — Berlin ; Heidelberg : Springer, 1998. — (Lecture Notes in Computer Science ; Vol. 1524). — P. 55–69. — DOI: 10.1007/3-540-49430-8_3.
38. Müller, R. When does label smoothing help? / R. Müller, S. Kornblith, G. E. Hinton // Advances in Neural Information Processing Systems 32 (NeurIPS 2019). — 2019. — P. 4694–4703. — URL: https://papers.nips.cc/paper/2019/hash/f1748d6b0fd9d439f71450117eba2725-Abstract.html (date of access: 24.09.2026).
39. Bellman, R. Dynamic Programming / R. Bellman. — Mineola, NY : Dover Publications, 2003. — xxv, 340 p. — Reprint of the edition: Princeton : Princeton University Press, 1957.
40. Sutton, R. S. Reinforcement Learning: An Introduction / R. S. Sutton, A. G. Barto. — 2nd ed. — Cambridge, MA : The MIT Press, 2018. — 552 p. — ISBN 978-0-262-03924-6. — URL: http://incompleteideas.net/book/the-book-2nd.html (date of access: 25.09.2026).
41. Global Land One-kilometer Base Elevation (GLOBE) : Digital Elevation Model, Documentation, Version 1.0 / ed. by D. A. Hastings, P. K. Dunbar ; National Geophysical Data Center. — Boulder : NOAA, 1999. — (Key to Geophysical Records Documentation ; No. 34). — URL: https://repository.library.noaa.gov/view/noaa/13424 (date of access: 13.10.2026).
42. Karin, T. Global Land Mask : software, version 1.0.0 / T. Karin. — 2020. — DOI: 10.5281/zenodo.4066722. — URL: https://github.com/toddkarin/global-land-mask (date of access: 13.10.2026).

Appendix A. User guide

A.1 Starting the application

Run python scripts/run_server.py and open <http://127.0.0.1:8000>.

The header shows two indicators:

- live connection — the real-time channel to the server is open. While it reads reconnecting… the application still works through ordinary HTTP requests, only without live progress.
- model 96.9% acc — the loaded classifier and its test-set accuracy.

The window is split into the control panel (left) and the chart (right). The panel has five tabs: Plan, Data, Compare, Model, History.

The buttons EN, RU and 中文 at the right of the header switch the language of the interface between English, Russian and Chinese at any moment, also after a search: the results on the screen are redrawn in the new language. The choice is remembered in the browser. Result files, reports and exports are always in English (figures A.7, A.8).

A.2 Planning a route

A.2.1 Setting departure and destination

Any route can be set; the presets are only examples. Three equivalent ways:

- Click the chart. The first click sets the departure (marker A), the second the destination (marker B). The hint above the coordinate fields says which one the next click will set.
- Drag the markers. Both markers can be dragged; the coordinate fields follow.
- Type the coordinates into the four fields, or press one of the preset chips (Kiel → Tallinn, Rotterdam → Lisbon, Helsinki → Copenhagen, Odesa → Novorossiysk, Piraeus → Thessaloniki, and the complex courses Gdańsk → Bergen, Jeddah → Dubai, Piraeus → Rhodes, Singapore → Hong Kong, Rotterdam → Gdańsk). The Piraeus presets also set the grid resolution to 0.1°, which the island-dense Aegean Sea needs; the other presets restore the default of 0.25°.

Via points. A complex course — round a peninsula, through a chosen strait, past a chosen side of an island group — can be forced through intermediate points: Shift + click on the chart, or press Add via point and click as many times as needed, then press it again. Each point is drawn as a yellow numbered marker and listed under the coordinates; the route passes through the points in their order, every leg on its own lattice. A via point can be

dragged, removed with a right click or with × in the list, or all of them with Remove all. The buttons Set A on chart and Set B on chart choose which point the next click sets, and Swap A ⇄ B exchanges them (figure A.5).

A.2.2 Choosing the parameters

The data are given in table A.1.

Table A.1 – Choosing the parameters

Control	**Meaning**
Search algorithm	A* — heuristic search, the default; Dijkstra — uniform cost, same result, more nodes; Genetic algorithm — evolutionary search: a population of candidate routes is improved by selection, crossover and mutation under a fitness function; it gives no guarantee of optimality and is shown for comparison; Dynamic programming — solves the Bellman equation for the cost to the destination of every cell (value iteration) and follows it from the departure; exact, same result as A*; Great circle — the naive shortest-distance reference
Grid resolution	lattice step in degrees, 0.02–1°. Finer resolution follows narrow straits more precisely but costs more time.
Service speed	used only to convert the route length into a passage time
Show zone classification layer	overlays the classified cells on the chart
Live zone read-out	classifies the point under the pointer continuously and shows the zone, confidence and depth at the bottom-left of the chart
Algorithm parameters	panel with the group Lattice and run and the group of the selected method (table 2.12): step limits, heuristic weight, sweeps, population, generations, early stop, operators, waypoints and seed of the genetic algorithm, cell budget, margin, corrections, time limit. Hover over a label for its meaning; changed values are amber and counted in the title (figure A.3)
Reset to defaults	restores the parameters of the groups shown

A.2.3 Building the route

Press Build route. A progress block reports each stage:

1) Building geographic lattice
2) Neural network classifying grid cells — with the number of cells done
3) Cost map ready — with the share of each zone
4) A* search in progress — with nodes expanded and frontier size (for Dijkstra the same; for the genetic algorithm Genetic algorithm: evolving routes with the generation number and the best route cost so far; for dynamic programming Dynamic programming: solving the Bellman equation with the sweep number and the number of values improved in it)
5) Coastline check: correcting the route — only when the first route crosses land on the independent coastline (subsection A.2.5)
6) Route ready

Timer and Stop. The progress block shows a timer and the time limit, if one is set. The Stop button ends the search at any stage within a fraction of a second (figure A.3). A search that reaches a limit ends by itself. The outcome is shown above the route summary:

- Route found — the route is drawn;
- Step limit reached before the goal — the method made its maximum number of steps (A*, Dijkstra), sweeps (dynamic programming) or generations (genetic algorithm); raise it in the parameters, for example from 1,000 to 100,000, and build again (figure A.4);
- Time limit reached — raise the time limit or use a coarser lattice;
- Stopped by the user — the Stop button was pressed;
- No navigable route between the points — try via points, a finer lattice or a larger margin.

If no route is found — because a strait is narrower than one lattice cell, or because the only passage lies outside the rectangle around the two ports — the message Adapting the lattice appears: when the water around both points reaches the edge of the lattice, the search area is widened; otherwise the lattice is refined (up to three times). A departure or destination is moved onto the nearest navigable cell only if that cell is within 60 km; a point further inland is reported as lying on land.

The genetic algorithm may fail to find a navigable route where A* succeeds (it can converge on a strait that the lattice closes). In that case the lattice is not refined, because A* has proved that a route exists; choose A* instead.

A.2.4 Reading the result

The chart shows:

- the optimised route, coloured per zone along its length;
- the great-circle route as a red dashed line;
- the zone classification layer, if enabled.

The Route summary panel reports the route length, the great-circle distance, the detour ratio, the weighted traversal cost, the estimated passage time, the planning time, the number of expanded nodes and the number of waypoints. Below it, the Zone profile bar shows how much of the route lies in each zone.

How the route was found opens a table of every lattice the planner tried — cell size, cells, steps, seconds, outcome — and what it did next: a finer lattice, a wider area, the corridor of the two-level search (⊥), the next leg of a route with via points. Above the summary a green box shows the absolute path of the result file, with Copy path and Download JSON (subsection A.2.6, figure A.4).

A.2.5 Coastline check

Every route is checked automatically against an independent coastline — the 1 km GLOBE land mask of the package global-land-mask, which plays no part in the classification. The route is sampled every kilometre; the first and the last 5 km (the quays) are not checked. A banner above the route summary shows the outcome:

- green, Coastline check passed — no land on the route, or it touches the coast for less than 3 km;
- amber, the first route crossed land … and was corrected automatically — the neural network had taken some land for water; the cells under that land were blocked and the route searched again, if necessary on a finer lattice; the route shown passes the check (figure A.2);
- red, The route crosses land … — the automatic correction could not remove the land; the places are marked on the chart with red circles. Such a course must not be followed: try a finer grid resolution or check the passage on an official chart.

A port given by a city-centre coordinate lies on land, and the straight connector from it to the open water may cross the shore: land touching the quay is counted as the port approach and is not an error. The result of the check is also written to the PDF report (line Coastline check), to the GeoJSON export and to the database. Without the package global-land-mask, installed with requirements.txt, the banner is not shown and the application works as before.

A.2.6 Result files

Every search — found, not found, step limit, time limit or stopped — and every comparison of the methods is saved as a JSON file in the folder outputs/results of the application (another folder can be set with the environment variable MR_RESULTS_DIR).

The folder and the latest files are listed on the Data tab. The name of the file gives the points, the method, the outcome and the search time, for example route_N54.3233E10.1228_N59.4370E24.7536_via1_astar_found_3.72s_20261014-134827.json.

The file holds the request, all parameters used, the outcome, the timing, the lattices tried, the route with its coastline check and every point with latitude, longitude, zone, cumulative distance and course (table 2.14). It is opened by Python (json.load), Excel (Power Query), MATLAB (jsondecode) or any GIS without a special parser.

A.3 Comparing with the shortest route

The Compare tab is filled automatically after every route. It reports the length of both routes, the additional distance accepted by the optimised route, how many sample points of the great circle fall on land, and a verdict: a great-circle route that crosses land is not navigable at all, so the extra distance is the price of a route that can actually be sailed.

Press Compare algorithms to run A*, Dijkstra, dynamic programming, the genetic algorithm and the great circle on one shared cost map of the same course, with the parameters set on the Plan tab; the time limit applies to each method separately, and the timer and Stop work as for one route. The routes of all methods are drawn on the chart in the colours of the table (figure A.6). The first table reports for each method the outcome, route length, cost, how far above the optimum it lies, the steps made (for dynamic programming the Bellman updates, for the genetic algorithm the routes evaluated), the step limit and the search time. The second table puts the measured work next to the theoretical complexity — $O((V + E) \log V)$ for A* and Dijkstra, $O(S \cdot E)$ for dynamic programming, $O(G \cdot P \cdot L)$ for the genetic algorithm — and its worst-case bound for this lattice. The node reduction of A* against Dijkstra and the cost differences follow. The reported times exclude the shared cost-map construction. The comparison is saved to a compare_….json file.

A.4 Working with AIS data

Open the Data tab and drop a .geojson, .csv or .json file onto the drop zone (or click it to browse).

- CSV files must contain latitude and longitude columns; the usual spellings (lat, lon, lng, y, x, …) are recognised automatically.
- GeoJSON files must be a FeatureCollection of points.

The application validates the file, classifies the points and stores them in the database. The report shows how many features were read, how many were valid, how many

were dropped and how the points are distributed over the four zones. If the file carries class_code labels, the agreement between the model and those labels is reported as well.

The classified points are drawn on the chart and can be downloaded as Points GeoJSON or Points CSV.

The Database block at the bottom of the tab shows how many sessions, points, routes and waypoints are stored and how large the database file is.

A.5 Exporting the results

After a route has been built, three buttons appear under the route summary:
The data are given in table A.2.

Table A.2 – Exporting the results

Button	**File**	**Content**
GeoJSON	route_<id>.geojson	route line, every waypoint with its zone, and the great-circle reference line
CSV	route_<id>.csv	waypoint table: sequence, coordinates, zone, weight, cumulative distance, bearing
PDF report	route_<id>.pdf	voyage summary (with the coastline check and its warning, if any), zone profile, comparison with the shortest route, model information and the waypoint list

The GeoJSON file carries styling properties (stroke, marker-color) and opens directly in QGIS, geojson.io and similar tools.

A.6 Inspecting the model

The Model tab reports the architecture, the number of input features and parameters, the size of the reference index, and the full test-set metrics: accuracy, macro and weighted F1, Cohen's κ and macro ROC-AUC. Below them the confusion matrix is shown cell by cell, together with the training and validation curves of the run that produced the deployed weights.

A.7 Stored routes

The History tab lists the routes saved in the database, newest first, with the algorithm, length, creation time, number of waypoints, search time and detour ratio. Clicking an entry draws that route on the chart again.

A.8 Keyboard and mouse summary

The data are given in table A.3.

Table A.3 – Keyboard and mouse summary

Action	**Result**
Click on the chart	set departure, then destination
Drag marker A / B	move departure / destination
Shift + click on the chart	add a via point
Drag a via point / right click on it	move it / remove it
Stop button in the progress block	stop the running search
Scroll on the chart	zoom
Move the pointer over the chart	live zone read-out (when enabled)
Layer control, top-left	switch between the online chart and the on-board chart
EN / RU / 中文 in the header	switch the language of the interface

A.9 Views of the interface

The state of the application after start-up is shown in figure A.1.



Figure A.1 – The application after start-up, on-board chart selected

The banner of the coastline check after a route that was corrected automatically (preset Helsinki → Copenhagen) is shown in figure A.2.

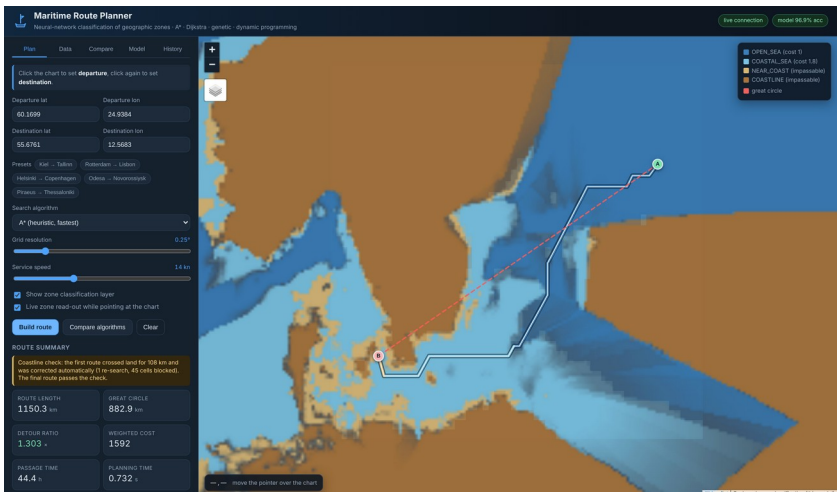


Figure A.2 – A route corrected by the coastline check: the amber banner above the route summary

The parameter panel, the timer and the Stop button during a long search are shown in figure A.3.

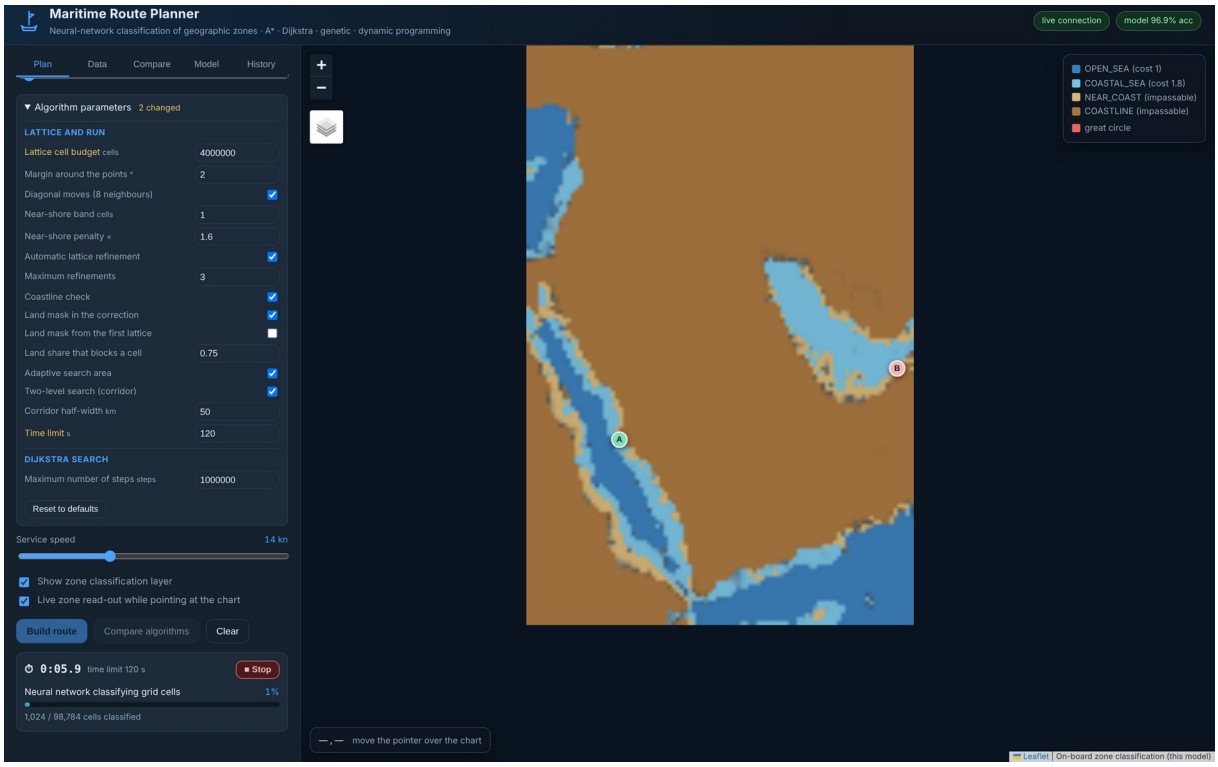


Figure A.3 – Parameters of Dijkstra and of the lattice, the timer and the Stop button during a long search (Jeddah → Dubai, 0.04°)

The outcome of a search ended by the step limit and the location of its result file are shown in figure A.4.

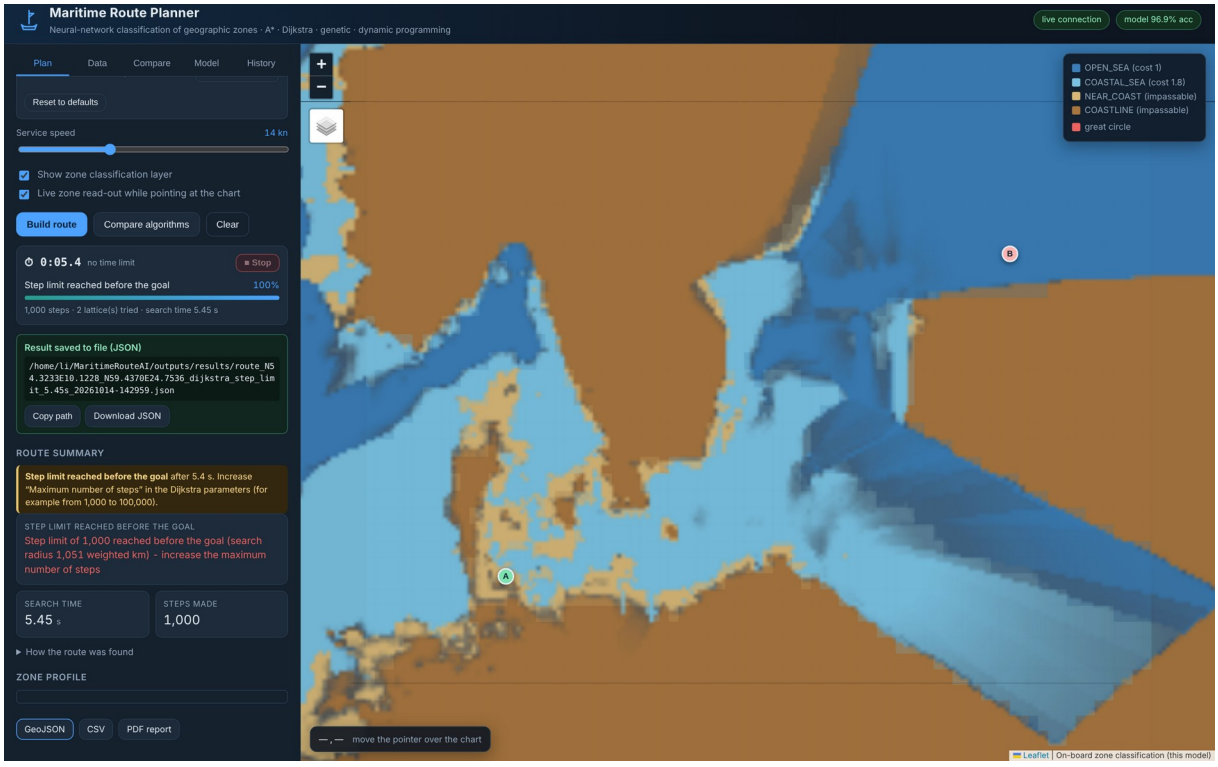


Figure A.4 – A search ended by the step limit (Dijkstra, 1,000 steps) and the location of its result file

A route through a via point set with Shift + click is shown in figure A.5.



Figure A.5 – Own route with a via point (Kiel → via point east of Gotland → Tallinn)

The comparison of the methods on one course is shown in figure A.6.

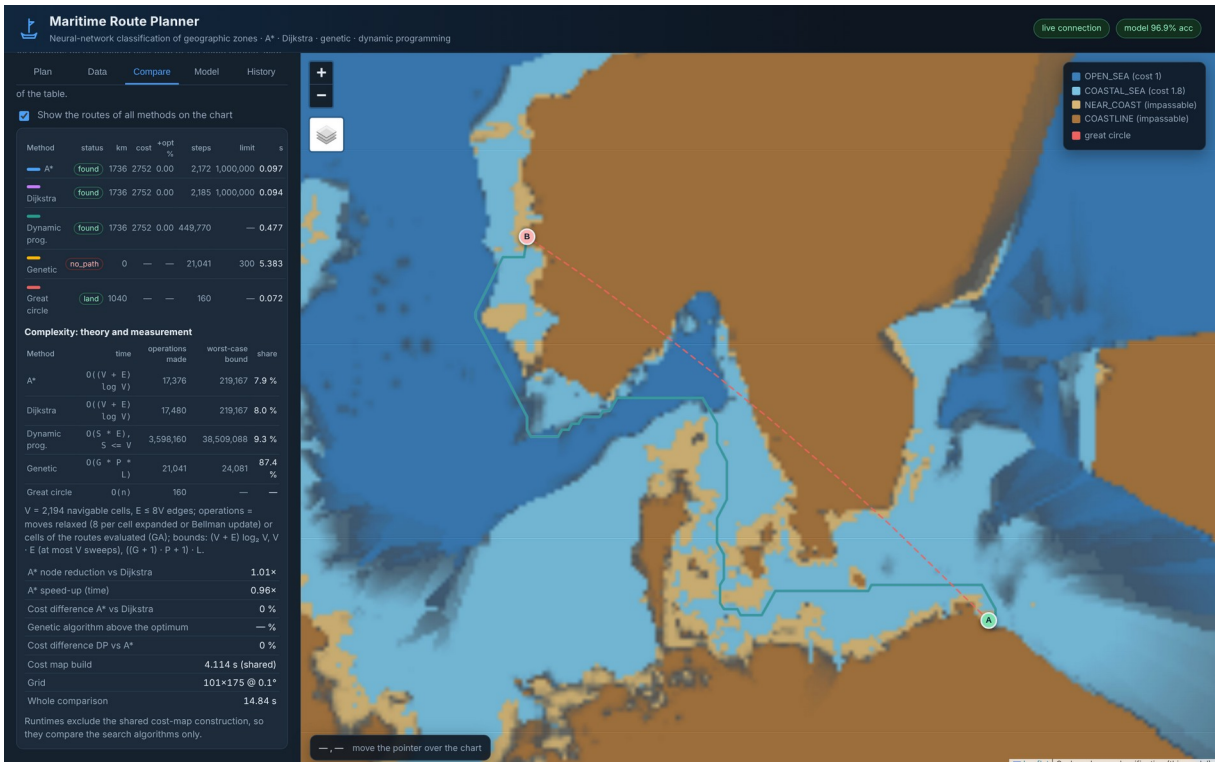


Figure A.6 – Comparison of the methods on one course: routes, outcomes, steps and complexity (Gdańsk → Bergen)

The interface in Russian is shown in figure A.7.

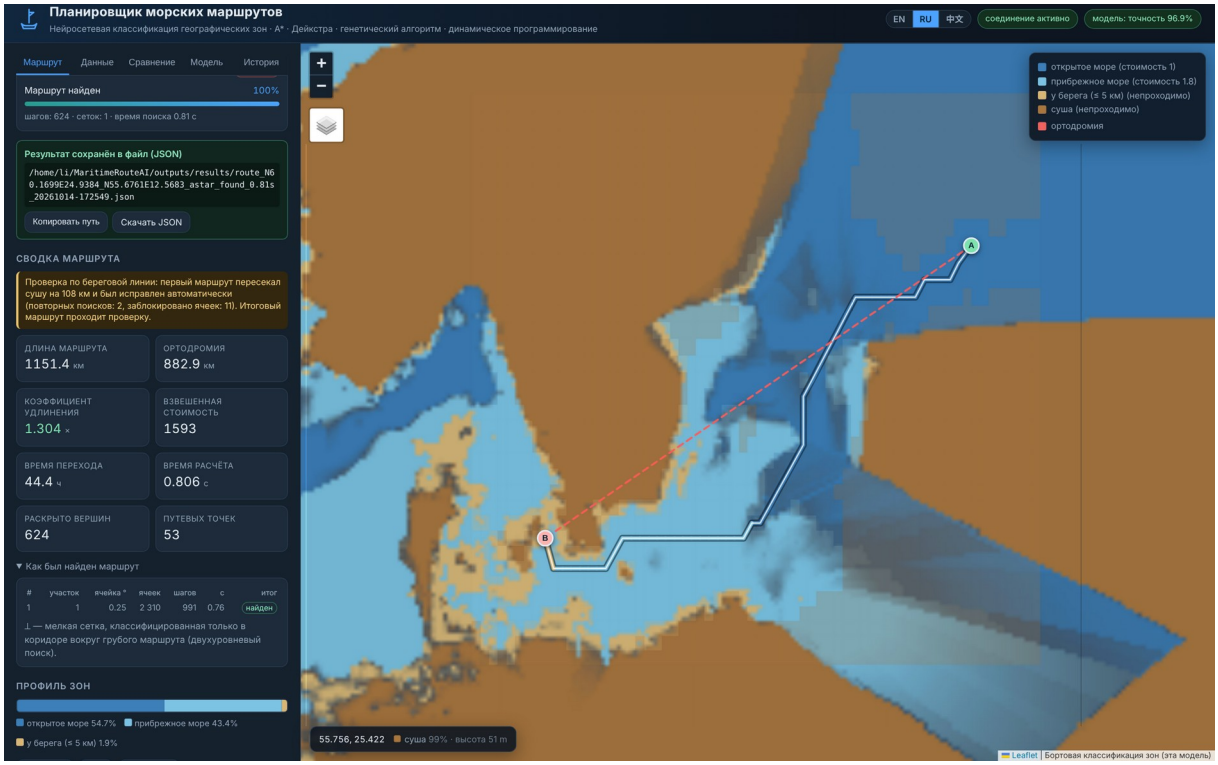


Figure A.7 – The interface in Russian: a route corrected by the coastline check (Helsinki → Copenhagen)

The interface in Chinese is shown in figure A.8.

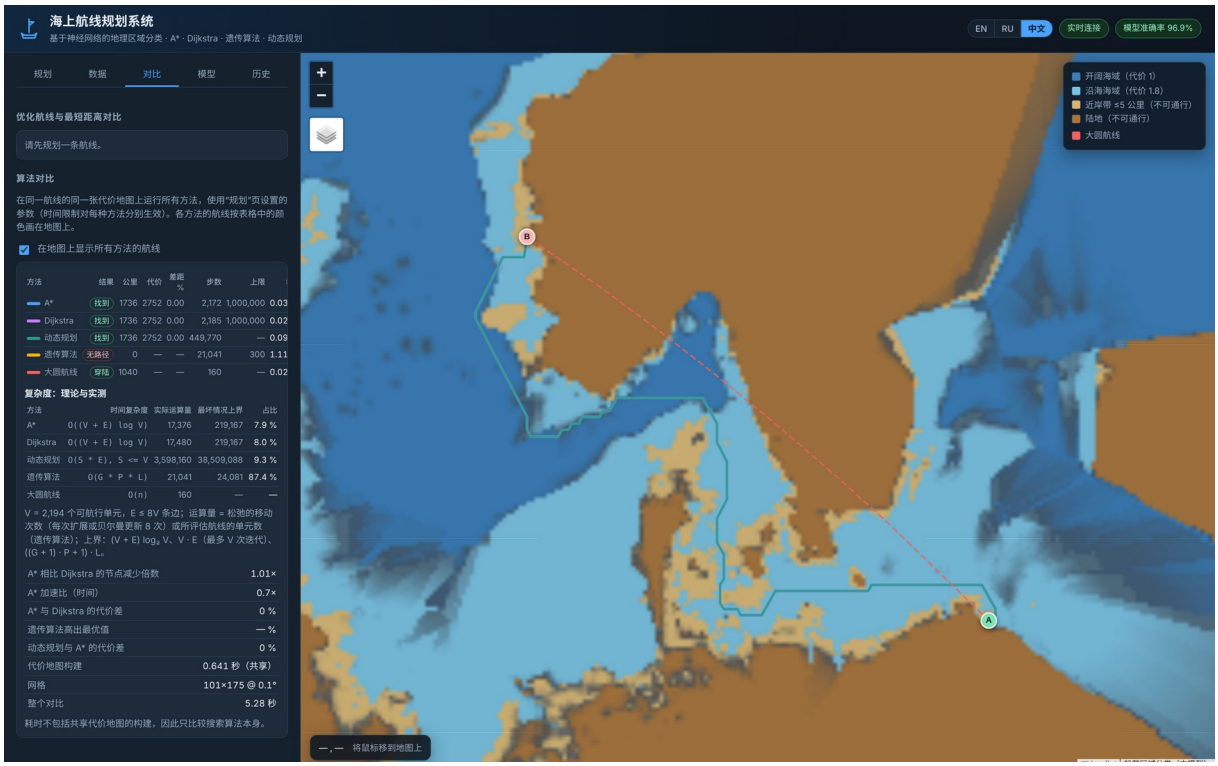


Figure A.8 – The interface in Chinese: comparison of the methods (Gdańsk → Bergen)

Appendix B. Installation and configuration guide

B.1 System requirements

The data are given in table B.1.

Table B.1 – System requirements

Item	Minimum	Recommended
Operating system	Windows 10 / Ubuntu 20.04 / macOS 12	any 64-bit, current
Python	3.10	3.11
RAM	4 GB	8 GB
Free disk space	3 GB	5 GB
CPU	2 cores	4 cores or more
GPU	not required	not required (CPU inference is sufficient)
Browser	any with WebSocket support	Chrome / Firefox / Edge, current

Training the model from scratch takes about five minutes on four CPU cores; no GPU is needed at any stage.

B.2 Installation

B.2.1 Obtain the sources

Clone the repository from GitHub (or unpack the delivered archive):

```
git clone https://github.com/EDXiaoM/MaritimeRouteAI.git
cd MaritimeRouteAI
```

On Windows without Git, use Code → Download ZIP on the GitHub page, unpack the archive and open a command prompt in the unpacked folder.

B.2.2 Create an isolated environment

```
python -m venv .venv
```

```
source .venv/bin/activate        # Linux / macOS
.venv\Scripts\activate           # Windows
```

B.2.3 Install the dependencies

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

If pip cannot reach the default PyTorch wheels, install the CPU build explicitly:

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
```

B.2.4 Verify the installation

```
pip install -r requirements-dev.txt
pytest
```

All tests must pass. Tests that need the trained model are skipped automatically when models/zone_classifier.pt is absent.

B.2.5 Updating the project from GitHub

When a new version is published in the repository, the copy on the computer is updated in place; nothing has to be installed again. Stop the server (Ctrl + C) and run in the project folder:

```
git pull                         # the new version from GitHub
.venv\Scripts\activate           # Windows
source .venv/bin/activate        # Linux / macOS
pip install -r requirements.txt  # new dependencies, if any
python scripts/run_server.py     # start again, reload the page
```

The command git pull leaves alone the files that are not part of the repository: the database (data/maritime_route.db), the result files (outputs/results), the loaded data and the environment .venv. If it refuses because files of the repository were changed locally, the changes are discarded with git checkout -- . or set aside with git stash, and git pull is repeated. The installed version is shown by git log -1.

A copy unpacked from a ZIP archive has no link to the repository: the new archive is downloaded (Code → Download ZIP), unpacked into a new folder, the folders outputs and data are copied into it from the old copy, and steps B.2.2–B.2.3 are repeated.

B.3 Preparing the data and the model

The archive already contains a trained model (models/), so this section is needed only to retrain or to use a different corpus.

The labelled corpus is attached to the GitHub release v1.0 as

merged_geojson_v2.zip

(https://github.com/EDXiaoM/MaritimeRouteAI/releases/download/v1.0/merged_geojson_v2.zip); unpack it into data/raw/.

```
# 1. put the labelled corpus here:
#    data/raw/merged.geojson
python scripts/prepare_dataset.py            # ~16 s, cleaning + features,
writes data/processed/
python scripts/train_model.py                # ~5 min on CPU, writes models/
python scripts/evaluate_regions.py           # optional: accuracy per sea
region
python scripts/benchmark_routing.py          # optional: 6 European voyages,
all methods
python scripts/regional_examples.py          # optional: 11 regional voyages +
map figure
python scripts/make_figures.py               # optional: regenerate the
figures
```

prepare_dataset.py removes the points whose class contradicts their measured height or depth. Both measurements of a point, noaa_depth and elevation, are checked; at the default tolerance of 5 m (LABEL_CHECK_TOLERANCE_M in

config.py) 5 427 of the 156 838 points are removed and 151 411 remain. Pass

--no-clean to keep them.

prepare_dataset.py writes:

The data are given in table B.2.

Table B.2 – Preparing the data and the model

File	**Content**
data/processed/zone_dataset.npz	feature matrices and labels for the three splits
data/processed/dataset_statistics.json	corpus statistics, cleaning report and per-region counts
models/reference_index.npz	label-free bathymetry / OSM index

train_model.py writes:

The data are given in table B.3.

Table B.3 – Preparing the data and the model

File	Content
models/zone_classifier.pt	network weights and architecture
models/feature_scaler.joblib	fitted standardiser
models/model_metadata.json	hyper-parameters, metrics and training history

B.4 Running the application

```
python scripts/run_server.py
```

Then open <http://127.0.0.1:8000>.
Options:

```
python scripts/run_server.py --host 0.0.0.0 --port 8080
python scripts/run_server.py --reload          # development: auto-reload
```

B.5 Configuration

All tunable values live in src/maritime_route/config.py. The most useful ones:
The data are given in table B.4.

Table B.4 – Configuration

Setting	Default	Meaning
ZONE_COST	1.0 / 1.8 / ∞ / ∞	traversal weight of each zone
ROUTING.grid_resolution_deg	0.25	default lattice step, degrees
ROUTING.max_grid_cells	400 000	lattice budget; larger areas are coarsened
ROUTING.safety_margin_cells	1	width of the penalised belt along the shore
ROUTING.proximity_penalty	1.6	extra cost of a water cell next to land
GENETIC.*	see file	default population size, generations, rates and seed of the genetic algorithm
routing/parameters.py	see file	defaults, ranges and help texts of every parameter set in the

Setting	Default	Meaning
		interface (version 1.3)
LABEL_CHECK_TOLERANCE_M	5.0	tolerance of the data-cleaning check, metres
REGIONS	eleven boxes	the sea regions used for per-region evaluation
TRAINING.*	see file	network size, epochs, learning rate, seed
SERVER.progress_interval_s	0.15	minimum interval between progress frames

Several settings can also be overridden with environment variables, which is convenient for deployment:

```
export MR_HOST=0.0.0.0
export MR_PORT=8080
export MR_DB_PATH=/var/lib/maritime/routes.db
export MR_MODEL_DIR=/opt/maritime/models
export MR_OUTPUT_DIR=/var/lib/maritime/exports
export MR_RESULTS_DIR=/var/lib/maritime/results
```

B.6 The database

SQLite is created automatically at data/maritime_route.db on first start; the schema is in src/maritime_route/storage/schema.sql. No database server has to be installed. To reset the stored history, stop the application and delete the

data/maritime_route.db* files.

B.7 Offline operation

The client requests OpenStreetMap tiles when the machine is online. If the tile service is unreachable, after four failed tiles the client switches by itself to the on-board chart — tiles rendered by the server from the model's own classification — and the application stays fully usable. The layer can also be selected manually in the layer control at the top-left corner of the map.

B.8 Troubleshooting

The data are given in table B.5.

Table B.5 – Troubleshooting

Symptom	**Cause and remedy**
503 Model weights not found	the model has not been trained — run scripts/train_model.py
The status pill shows reconnecting…	the server was restarted; the client reconnects by itself with exponential back-off
Address already in use	another process holds port 8000 — start with --port 8081
Planning reports no navigable path	departure or destination is landlocked; the lattice is already refined and widened three times automatically
Planning reports lies on land	the port is more than 60 km from any navigable cell, often because it lies outside the eleven sea regions covered by the training data
The genetic algorithm reports no navigable route evolved	the evolutionary search converged on a closed strait; use A*
A red banner The route crosses land …	the automatic coastline check could not correct the route (the network takes too much land for water there); set the grid resolution to 0.1° and build again, or check the passage on an official chart
No coastline-check banner appears	the package global-land-mask is not installed — pip install -r requirements.txt
Step limit reached before the goal	the maximum number of steps of the method is too small for this course; raise it in Algorithm parameters, for example ×100, or use A* instead of Dijkstra
The search takes too long	press Stop or set a time limit; use a coarser lattice, via points, or a heuristic weight of A* above 1
Where are the result files?	in outputs/results of the project or in MR_RESULTS_DIR; the path is shown after every search and on the Data tab
The map is blank	no internet access — switch to On-board chart (offline) in the layer control
ModuleNotFoundError: maritime_route	run the scripts from the project root, or set PYTHONPATH=src
Activate.ps1 cannot be loaded because running scripts is disabled (Windows PowerShell)	allow local scripts once with Set-ExecutionPolicy -Scope CurrentUser RemoteSigned, or skip the activation and call .venv\Scripts\python.exe directly
git pull reports “Your local changes would be overwritten”	files of the repository were edited locally: git stash (keep them) or git checkout -- . (discard them), then git pull again

Appendix C. Results of the test on 100 random voyages

Table C.1 lists every voyage of the test described in subsection 3.5.6: the two ends (latitude, longitude in degrees), the great-circle distance, the length of the planned route, the verdict of the check against the independent land mask, the longest continuous land run and, for an error, its cause. The same data are written by the script route_audit.py to the files docs/figures/route_audit.csv and route_audit.json. Verdicts: correct — no sample on land; grazing — land runs shorter than 3 km; crosses land — a land run of 3 km or more; no route — the planner found none. Causes: classifier — land taken for water; lattice — a diagonal step across land narrower than a cell; no sea link — no open sea connection between the ports; closed on map — a connection exists but the classified lattice closes it.

Table C.1 – Voyages of the test

No.	Region	From	To	GC, km	Route, km	Verdict	Land run, km	Cause
1	North Europe	56.95, 17.54	57.95, 10.78	420	976	crosses land	13.4	classifier
2	Red Sea, Aden	11.28, 43.55	12.56, 49.17	628	669	correct	0.0	–
3	Aegean Sea	35.45, 28.09	35.42, 22.98	463	508	correct	0.0	–
4	Black Sea, E. Med.	42.66, 38.16	43.43, 33.94	353	385	correct	0.0	–
5	Persian Gulf	25.18, 57.32	16.49, 62.34	1099	1169	correct	0.0	–
6	Bay of Bengal	17.41, 83.00	16.99, 88.28	562	656	correct	0.0	–
7	Malacca, Andaman	2.35, 100.99	4.28, 98.36	363	391	correct	0.0	–
8	South China Sea	7.69, 116.15	8.99, 109.94	698	742	correct	0.0	–
9	Japan, NW Pacific	41.88, 138.72	41.89, 151.43	1051	1137	crosses land	27.9	classifier
10	Southern Africa	-38.44, 17.00	-37.98, 15.34	154	168	correct	0.0	–
11	Central America	10.70, -79.27	8.73, -79.62	222	–	no route	–	no sea link
12	North Europe	54.08, 10.95	50.64, 0.27	818	1745	grazing	2.9	–
13	Red Sea, Aden	25.47, 51.59	27.86, 50.96	273	301	correct	0.0	–
14	Aegean Sea	40.46, 22.81	35.85, 25.56	566	651	grazing	2.0	–
15	Black Sea, E. Med.	42.52, 34.10	43.89, 31.65	251	274	correct	0.0	–
16	Persian Gulf	26.24, 56.00	24.85, 58.69	311	349	crosses land	10.0	lattice
17	Bay of Bengal	17.42, 83.00	20.36, 90.09	814	889	correct	0.0	–
18	Malacca, Andaman	7.15, 104.58	5.35, 95.05	1073	2040	crosses land	6.9	classifier
19	South China Sea	11.60, 115.28	14.32, 111.06	548	587	correct	0.0	–
20	Japan, NW Pacific	41.06, 145.29	37.86, 153.33	776	857	correct	0.0	–
21	Southern Africa	-37.73, 17.84	-34.79, 22.59	537	578	correct	0.0	–

No.	Region	From	To	GC, km	Route, km	Verdict	Land run, km	Cause
22	Central America	8.25, -80.24	6.08, -83.94	474	–	no route	–	closed on map
23	North Europe	54.08, 11.38	55.94, 15.52	335	383	correct	0.0	–
24	Red Sea, Aden	18.34, 39.74	11.13, 43.63	904	978	crosses land	4.0	classifier
25	Aegean Sea	39.79, 24.23	36.56, 20.42	490	791	crosses land	17.7	classifier
26	Black Sea, E. Med.	41.30, 28.97	43.00, 38.52	809	877	correct	0.0	–
27	Persian Gulf	24.61, 58.43	26.99, 53.09	597	696	crosses land	7.0	lattice
28	Bay of Bengal	21.18, 89.99	16.17, 87.66	608	656	correct	0.0	–
29	Malacca, Andaman	6.26, 97.65	8.40, 96.55	268	290	correct	0.0	–
30	South China Sea	17.24, 117.45	15.29, 122.11	542	1138	crosses land	14.6	classifier
31	Japan, NW Pacific	36.39, 142.16	37.09, 137.83	393	–	no route	–	closed on map
32	Southern Africa	-38.50, 15.38	-35.05, 22.37	731	799	correct	0.0	–
33	Central America	8.23, -80.25	10.70, -81.95	333	–	no route	–	no sea link
34	North Europe	54.94, 18.26	54.09, 11.34	457	529	correct	0.0	–
35	Red Sea, Aden	14.77, 41.01	18.31, 39.72	417	446	correct	0.0	–
36	Aegean Sea	34.46, 28.12	35.05, 19.87	756	782	correct	0.0	–
37	Black Sea, E. Med.	32.35, 32.71	42.35, 30.04	1137	–	no route	–	no sea link
38	Persian Gulf	25.12, 58.92	23.23, 61.50	336	349	correct	0.0	–
39	Bay of Bengal	19.59, 89.89	16.66, 90.23	328	341	correct	0.0	–
40	Malacca, Andaman	5.54, 97.67	4.61, 103.96	704	1388	grazing	2.9	–
41	South China Sea	4.48, 106.63	5.11, 105.22	172	198	correct	0.0	–
42	Japan, NW Pacific	43.87, 137.38	35.66, 135.65	925	974	correct	0.0	–
43	Southern Africa	-35.97, 19.71	-36.41, 17.59	196	209	correct	0.0	–
44	Central America	9.35, -80.38	6.08, -83.69	516	–	no route	–	no sea link
45	North Europe	54.94, 18.23	56.14, 16.48	173	192	correct	0.0	–
46	Red Sea, Aden	20.08, 39.34	21.20, 38.08	181	182	correct	0.0	–
47	Aegean Sea	36.93, 23.71	38.25, 25.50	216	259	crosses land	9.7	classifier
48	Black Sea, E. Med.	44.18, 31.63	43.92, 35.94	346	359	correct	0.0	–
49	Persian Gulf	26.19, 54.16	25.02, 60.51	650	740	crosses land	4.0	classifier
50	Bay of Bengal	14.48, 84.89	21.71, 90.57	1003	1048	correct	0.0	–
51	Malacca, Andaman	1.12, 103.60	6.20, 97.55	877	936	crosses land	7.9	classifier

No.	Region	From	To	GC, km	Route, km	Verdict	Land run, km	Cause
52	South China Sea	7.95, 107.39	5.53, 111.80	556	607	correct	0.0	–
53	Japan, NW Pacific	35.93, 146.75	37.94, 144.80	282	283	correct	0.0	–
54	Southern Africa	-36.25, 18.96	-34.47, 18.63	200	211	correct	0.0	–
55	Central America	6.11, -83.66	8.24, -80.28	442	–	no route	–	closed on map
56	North Europe	54.60, 16.49	54.25, 12.11	286	332	correct	0.0	–
57	Red Sea, Aden	18.30, 39.72	12.54, 44.85	845	981	correct	0.0	–
58	Aegean Sea	35.26, 30.49	34.54, 25.37	474	505	correct	0.0	–
59	Black Sea, E. Med.	45.42, 30.50	42.06, 31.55	383	400	correct	0.0	–
60	Persian Gulf	16.76, 54.23	16.27, 58.49	458	477	correct	0.0	–
61	Bay of Bengal	11.41, 81.71	17.38, 82.98	678	718	correct	0.0	–
62	Malacca, Andaman	5.73, 99.90	4.83, 104.04	469	1246	grazing	1.0	–
63	South China Sea	6.21, 107.64	9.63, 115.46	942	1023	correct	0.0	–
64	Japan, NW Pacific	37.77, 146.12	40.97, 139.34	683	1638	crosses land	51.0	classifier
65	Southern Africa	-35.05, 19.37	-33.06, 17.88	260	298	correct	0.0	–
66	Central America	8.24, -80.21	11.88, -80.02	405	–	no route	–	no sea link
67	North Europe	58.46, 21.62	58.53, 10.53	644	1411	crosses land	19.7	classifier
68	Red Sea, Aden	26.96, 50.96	29.27, 48.15	377	387	correct	0.0	–
69	Aegean Sea	34.62, 22.23	38.21, 21.59	403	461	crosses land	24.0	lattice
70	Black Sea, E. Med.	43.04, 27.95	44.51, 33.21	453	506	correct	0.0	–
71	Persian Gulf	25.35, 58.13	15.88, 53.92	1141	1491	correct	0.0	–
72	Bay of Bengal	17.42, 83.00	19.84, 89.27	714	781	correct	0.0	–
73	Malacca, Andaman	7.57, 96.89	6.91, 98.79	223	247	correct	0.0	–
74	South China Sea	7.67, 113.59	17.27, 118.25	1181	1266	correct	0.0	–
75	Japan, NW Pacific	42.75, 151.65	39.60, 154.76	436	438	correct	0.0	–
76	Southern Africa	-34.40, 18.47	-34.51, 21.93	318	437	correct	0.0	–
77	Central America	10.69, -79.36	9.38, -80.31	179	186	correct	0.0	–
78	North Europe	61.55, -27.23	57.67, -25.32	445	460	correct	0.0	–
79	Red Sea, Aden	9.84, 51.22	11.16, 48.09	372	447	crosses land	184.6	classifier
80	Aegean Sea	40.02, 23.32	36.18, 20.31	502	946	grazing	2.0	–
81	Black Sea, E. Med.	43.79, 30.48	35.35, 35.38	1028	–	no route	–	no sea link

No.	Region	From	To	GC, km	Route, km	Verdict	Land run, km	Cause
82	Persian Gulf	24.80, 58.97	20.22, 58.23	514	686	crosses land	30.3	classifier
83	Bay of Bengal	19.98, 89.23	21.33, 90.90	230	263	correct	0.0	–
84	Malacca, Andaman	7.22, 96.76	5.21, 104.04	836	1667	crosses land	10.6	classifier
85	South China Sea	19.02, 118.42	19.18, 120.83	254	268	correct	0.0	–
86	Japan, NW Pacific	36.39, 142.16	36.29, 146.58	395	398	correct	0.0	–
87	Southern Africa	-33.71, 27.13	-35.52, 18.83	785	861	correct	0.0	–
88	Central America	9.65, -79.47	8.03, -80.29	202	–	no route	–	no sea link
89	North Europe	57.00, 17.57	54.95, 18.25	232	248	correct	0.0	–
90	Red Sea, Aden	11.22, 43.61	18.32, 39.77	891	973	crosses land	5.0	classifier
91	Aegean Sea	34.69, 27.74	36.78, 20.16	723	788	correct	0.0	–
92	Black Sea, E. Med.	42.07, 31.90	43.82, 31.73	196	202	correct	0.0	–
93	Persian Gulf	16.04, 59.12	23.84, 58.57	870	980	correct	0.0	–
94	Bay of Bengal	11.71, 83.95	11.90, 80.10	419	429	correct	0.0	–
95	Malacca, Andaman	6.18, 97.42	6.68, 104.28	760	1632	crosses land	5.0	classifier
96	South China Sea	8.02, 114.34	10.47, 117.25	420	437	correct	0.0	–
97	Japan, NW Pacific	39.48, 146.94	36.07, 150.30	481	483	correct	0.0	–
98	Southern Africa	-37.06, 15.24	-34.25, 22.85	755	837	correct	0.0	–
99	Central America	9.54, -82.47	11.54, -79.23	419	448	correct	0.0	–
100	North Europe	56.17, 17.88	54.09, 11.34	476	547	correct	0.0	–

Table C.2 lists the 19 voyages whose route was changed when the test was repeated with the automatic coastline check (subsection 3.5.7): the verdict, the longest land run and the route length without and with the check, the lattice finally used and the number of additional searches. All other voyages kept their route.

Table C.2 – Voyages changed by the coastline check

No.	Region	Without: verdict	land run, km	route, km	With: verdict	land run, km	route, km	lattice, °	extra searches
1	North Europe	crosses land	13.4	976	grazing	2.0	976	0.1375	2
9	Japan & North-West Pacific	crosses land	27.9	1137	grazing	1.0	1460	0.055	2
16	Persian Gulf & Arabian Sea	crosses land	10.0	349	correct	0.0	396	0.25	1
18	Strait of Malacca & Andaman Sea	crosses land	6.9	2040	grazing	2.9	2035	0.0756	2
24	Red Sea & Gulf of Aden	crosses land	4.0	978	correct	0.0	978	0.25	1

No.	Region	Without: verdict	land run, km	route, km	With: verdict	land run, km	route, km	lattice, °	extra searches
25	Aegean Sea	crosses land	17.7	791	grazing	3.0	956	0.25	3
27	Persian Gulf & Arabian Sea	crosses land	7.0	696	correct	0.0	754	0.25	1
30	South China Sea	crosses land	14.6	1138	grazing	2.0	1138	0.25	4
47	Aegean Sea	crosses land	9.7	259	grazing	3.0	399	0.25	2
49	Persian Gulf & Arabian Sea	crosses land	4.0	740	correct	0.0	758	0.25	1
51	Strait of Malacca & Andaman Sea	crosses land	7.9	936	correct	0.0	934	0.25	1
64	Japan & North-West Pacific	crosses land	51.0	1638	grazing	1.9	1683	0.0756	2
67	North Europe	crosses land	19.7	1411	correct	0.0	1400	0.1375	2
69	Aegean Sea	crosses land	24.0	461	grazing	1.0	462	0.1	0
79	Red Sea & Gulf of Aden	crosses land	184.6	447	correct	0.0	769	0.25	2
82	Persian Gulf & Arabian Sea	crosses land	30.3	686	correct	0.0	782	0.25	3
84	Strait of Malacca & Andaman Sea	crosses land	10.6	1667	grazing	1.9	1650	0.1	0
90	Red Sea & Gulf of Aden	crosses land	5.0	972	correct	0.0	973	0.25	1
95	Strait of Malacca & Andaman Sea	crosses land	5.0	1632	grazing	2.9	1632	0.0756	1

The maps of figures C.1–C.11 show the voyages of each region on the land of the independent mask (grey): green — correct, orange — grazing, red — crosses land; a voyage without a route is shown by its two ends in black. The number next to a route is its number in table C.1.

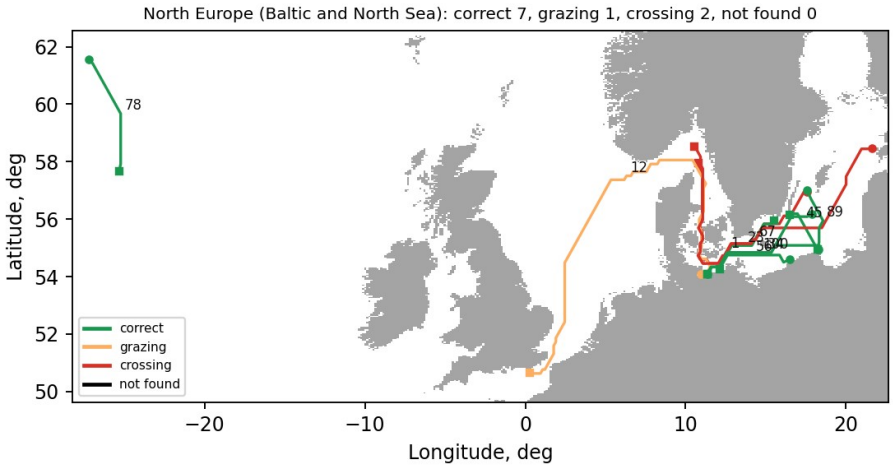


Figure C.1 – Test voyages in the region North Europe (Baltic and North Sea)

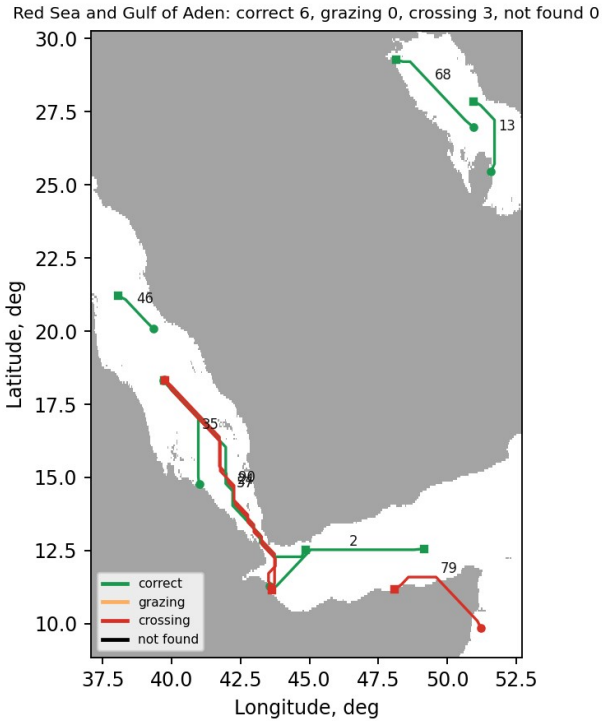


Figure C.2 – Test voyages in the region Red Sea and Gulf of Aden

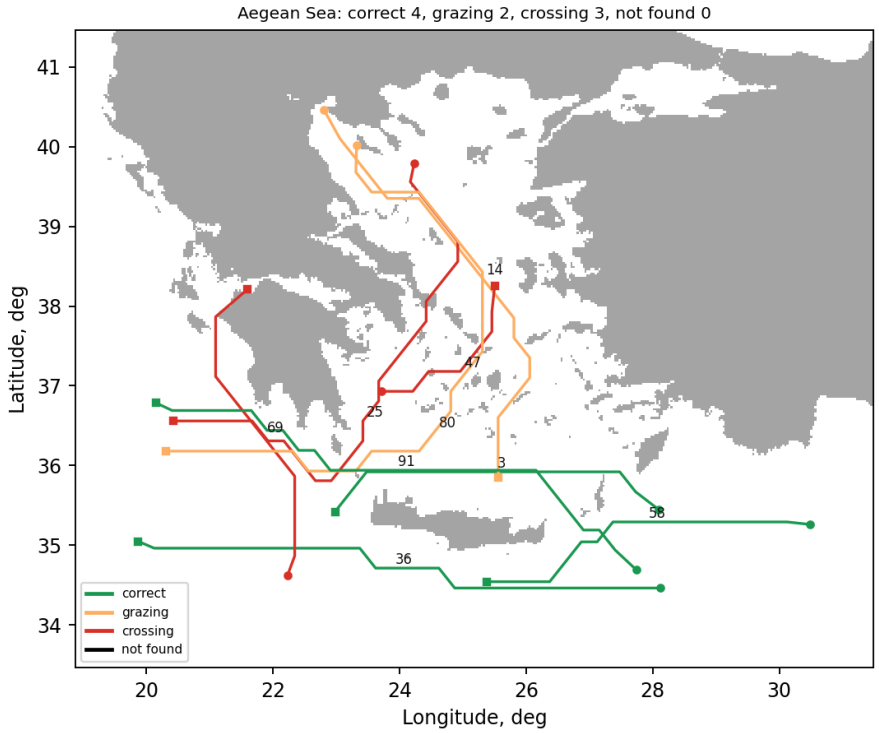


Figure C.3 – Test voyages in the region Aegean Sea

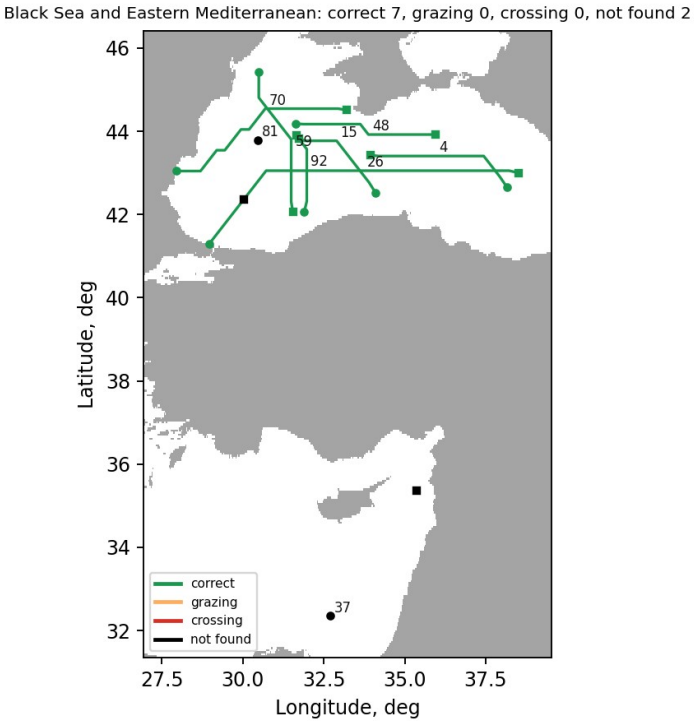


Figure C.4 – Test voyages in the region Black Sea and Eastern Mediterranean

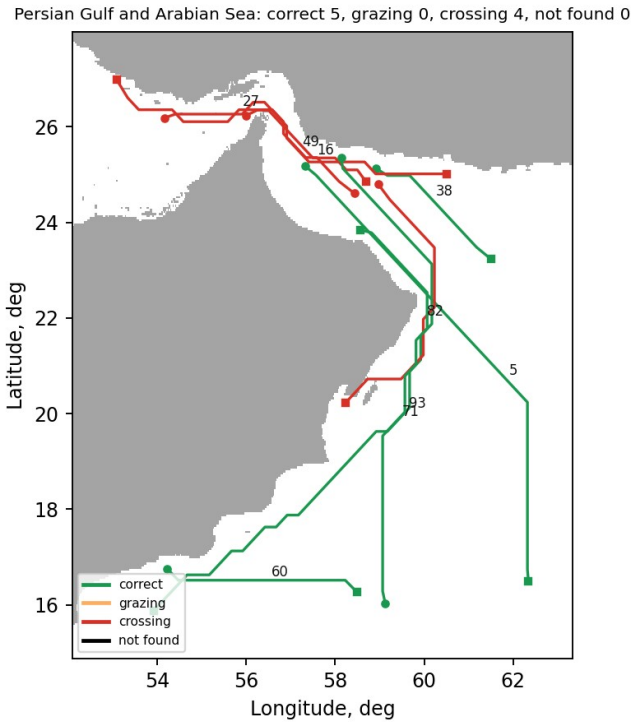


Figure C.5 – Test voyages in the region Persian Gulf and Arabian Sea

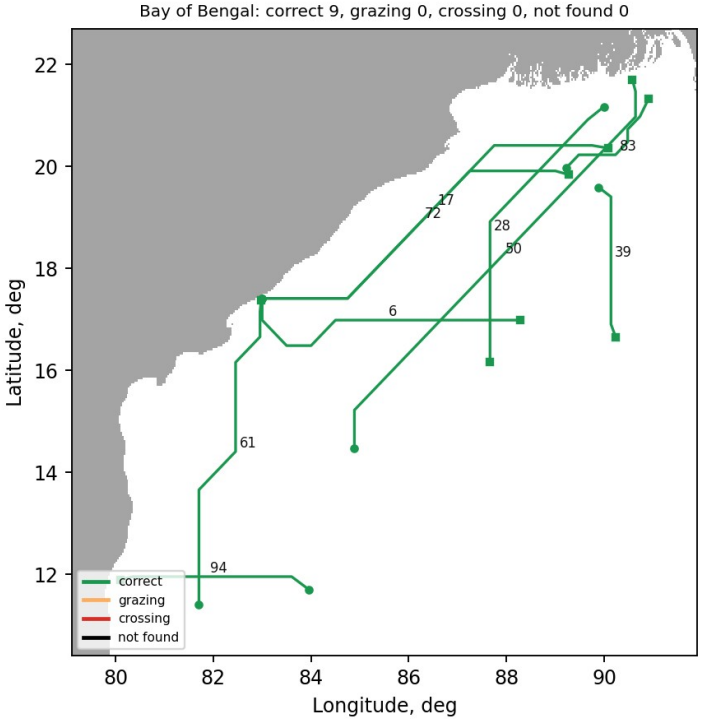


Figure C.6 – Test voyages in the region Bay of Bengal

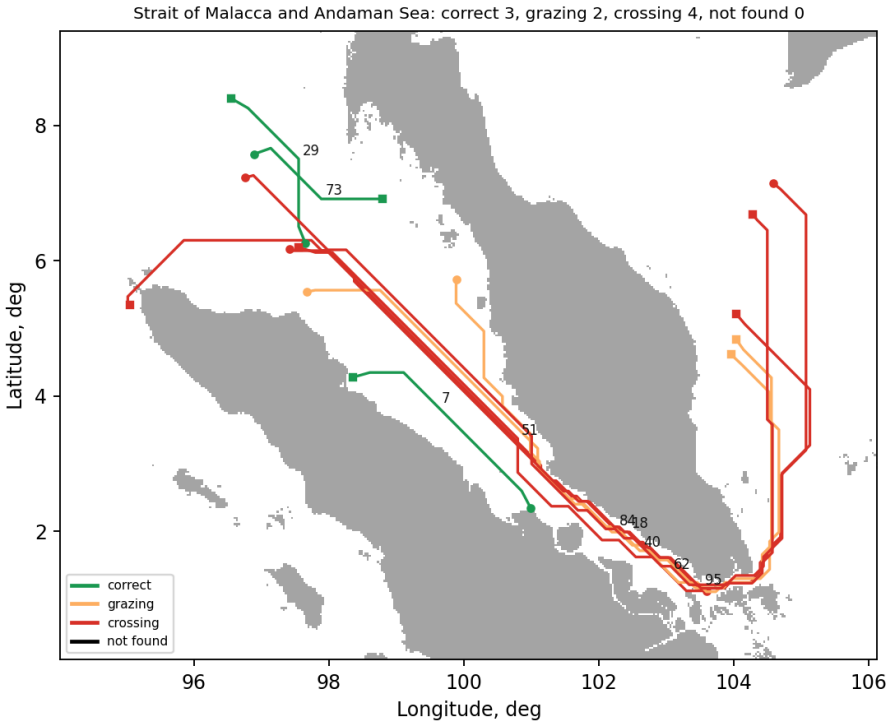


Figure C.7 – Test voyages in the region Strait of Malacca and Andaman Sea

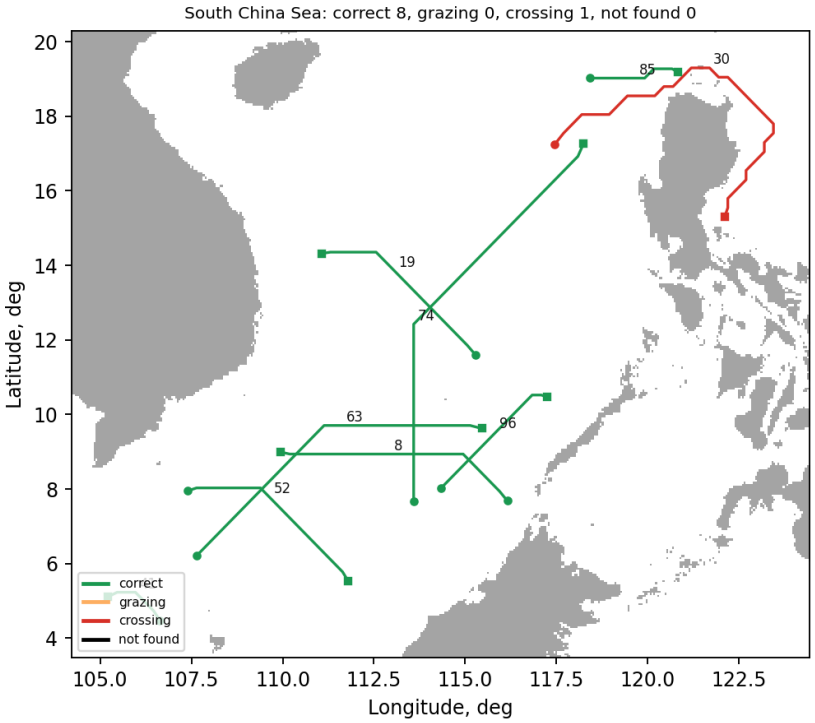


Figure C.8 – Test voyages in the region South China Sea

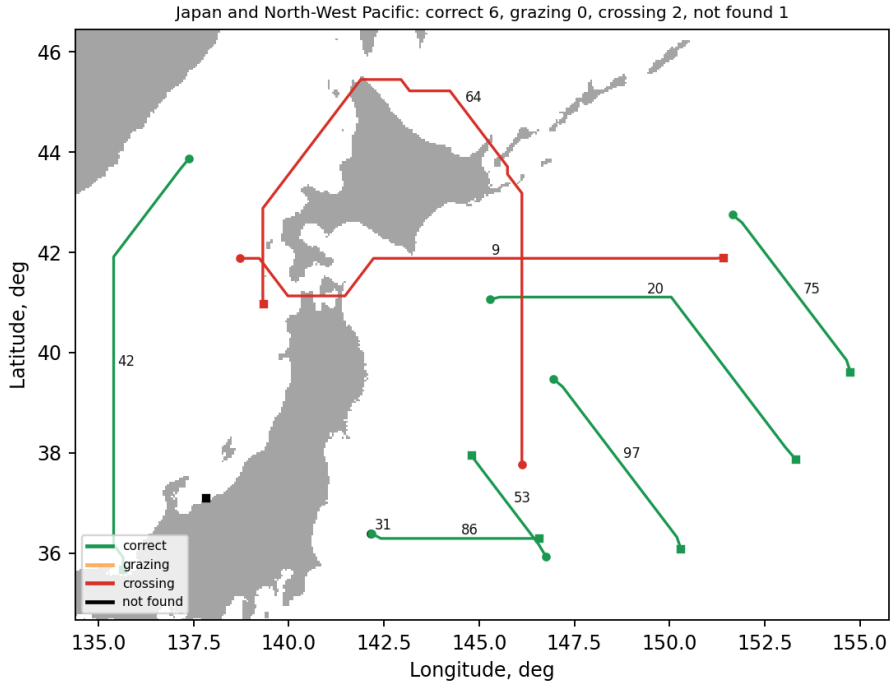


Figure C.9 – Test voyages in the region Japan and North-West Pacific

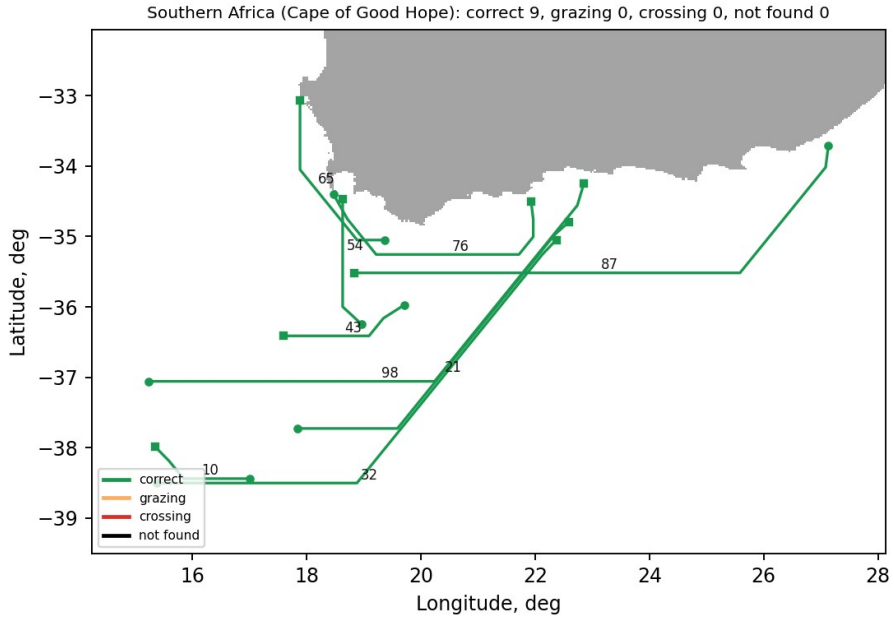


Figure C.10 – Test voyages in the region Southern Africa (Cape of Good Hope)

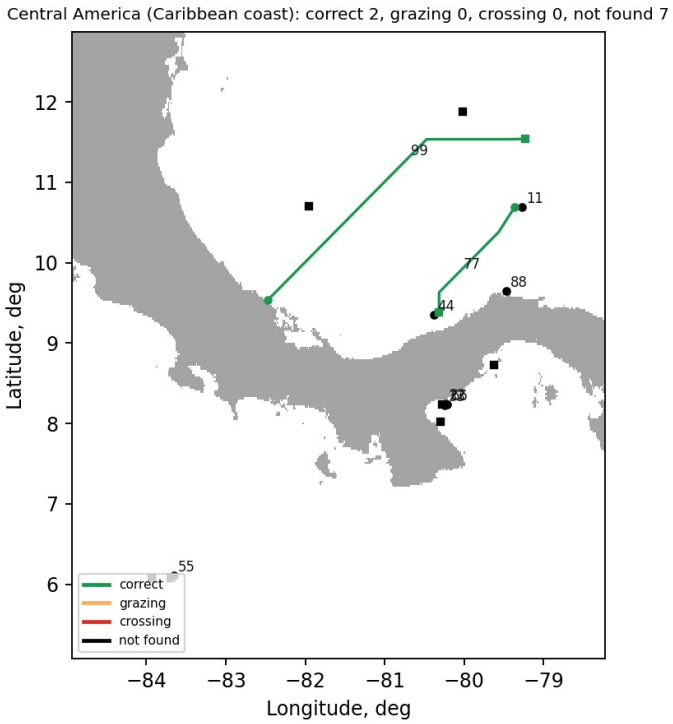


Figure C.11 – Test voyages in the region Central America (Caribbean coast)

Appendix D. Program text

The text of the program is published in the public repository https://github.com/EDXiaoM/MaritimeRouteAI